ETAS ISOLAR-A 12.7.0

ISOLAR-A Getting Started Guide

www.etas.com

Copyright

ETAS ISOLAR-A 12.7.0 - ISOLAR-A Getting Started Guide R01 EN – 04.2025

Contents

1 Introduction

This Getting Started Guide provides you with the information you'll require to install and start using ISOLAR-A (Integrated SOLution for AutosaR – Application Software). It also contains instructions for building an example application supplied with your distribution. A reference to other useful manuals is also provided.

1.1 Purpose

The purpose of this manual is to describe how to use ISOLAR-A. Its ease of use is illustrated through an example project that demonstrates an inter ECU Communication use case.

ISOLAR-A is the ETAS AUTOSAR Authoring Tool that assists users in designing application software to AUTOSAR standards.

Abstract: AUTOSAR is an evolving standard, which defines modular software architecture for automotive electronic control units. It is designed to increase the ease of software re-use and the deployment on different hardware. AUTOSAR increases the ease of software integration, but in a way mandates the use of tools to effectively handle its complexity.

Who Should Read this Manual?

This manual will be of interest to anyone who wishes to install and use ISOLAR-A to develop AUTOSAR compatible systems and/or AUTOSAR compatible application software. You should read this Getting Started Guide before installing ISOLAR-A.

1.2 Document Conventions

`OCI_CANTxMessage msg0`	Code snippets are shown in a monospaced Courier typeface.
Choose **File→Open**.	Menu commands are shown in boldface.
Click **OK**.	Buttons are shown in boldface.
Press <ENTER>.	Keyboard commands are shown in angled brackets.
The 'Open File' dialog box is displayed.	Names of program windows, dialog boxes, fields, etc. are shown in quotation marks.
Select the file *setup.exe*	Text in drop-down lists on the screen, program code, as well as path- and file names are shown in italics.
A *distribution* is always a one-dimensional table of sample points.	General emphasis and new terms are set in italics.

1.3 Privacy Statement (GDPR)

Your privacy is important to ETAS so we have created the following Privacy Statement that informs you which data are processed in ISOLAR-A, which data categories ISOLAR-A uses, and which technical measures you have to take to ensure

users' privacy.

Where relevant, we also provide additional information on where ISOLAR-A stores personal or personal-related data, and how it can be deleted.

1.3.1 Data Processing

Personal or personal-related data or data categories are processed if using the ETAS License Manager (LiMa). The purchaser of this product is responsible for the legal conformity of processing the data in accordance with Article 4 No. 7 of the General Data Protection Regulation (GDPR). As the manufacturer, ETAS GmbH is not liable for any mishandling of this data.

When using the ETAS License Manager in combination with user-based licenses, the following personal or personal-related data or data categories are recorded on the license server for the purposes of license management:

- Communication data: IP address
- User data: UserID, WindowsUserID

1.3.2 Technical and organizational measures

This product does not encrypt the personal or personal-related data or data categories that it records. Ensure that the data recorded are secured by means of suitable technical or organizational measures in your IT system. Personal or personal-related data in log files can be deleted by tools in the operating system.

1.4 Safety Notice

WARNING

Any configuration(s) described in this Getting Started Guide are provided as an example only and must be reviewed and adapted for specific project requirements before use.

For any safety related development that uses the ISOLAR-A tool you should apply an applicable safety standard such as ISO 26262:2018.

The ETAS_Safety_Advice.pdf provides additional safety information.

2 Documentation

2.1 Getting Started Guides

- ISOLAR-A Getting Started Guide (This manual)
- ISOLAR-B Getting Started Guide

2.2 Reference Manuals

- ISOLAR-A Help Reference Manual
- ISOLAR-B Help Reference Manual

2.3 Online Help

Detailed help for all the features in ISOLAR-A.

3 System Requirements

Please refer to the latest ISOLAR-A Release Notes for more details.

4 Installation

The Installation and Licensing processes for ISOLAR-A are described in the Release Notes document (Section 2).

5 Working with ISOLAR-A

5.1 Introduction

ISOLAR-A (Integrated SOLutions for AutosaR - Asw) is an AUTOSAR Architecture tool that is built on the Eclipse technology which uses the Artop framework to enable easy integration into existing development environments.

Some of the benefits of AUTOSAR, such as authoring of AUTOSAR compliant software systems, the design of Software Components (SWCs), SWC Compositions and integration to software systems, can be fully exploited. The following sections in this manual will help you to understand some of the common use cases that can be designed with ISOLAR-A.

Example Project

The example project described in this Getting Started Guide gives you a guided tour through ISOLAR-A and its main features. It demonstrates the typical tasks required to build an AUTOSAR-compatible software system, including the authoring of a system of application software components and compositions, and the mapping of systems onto technical platforms of ECUs, real-time operating systems, and Inter-ECU communication networks (e.g. CAN, LIN or FlexRay buses).

The diagram below illustrates the setup and configuration of our “Wiper Control” example project, which involves the mapping of a software system onto two ECUs, “Wiper Control” and “Debug”. Note that the Network communication between these two ECUs is via a CAN bus.

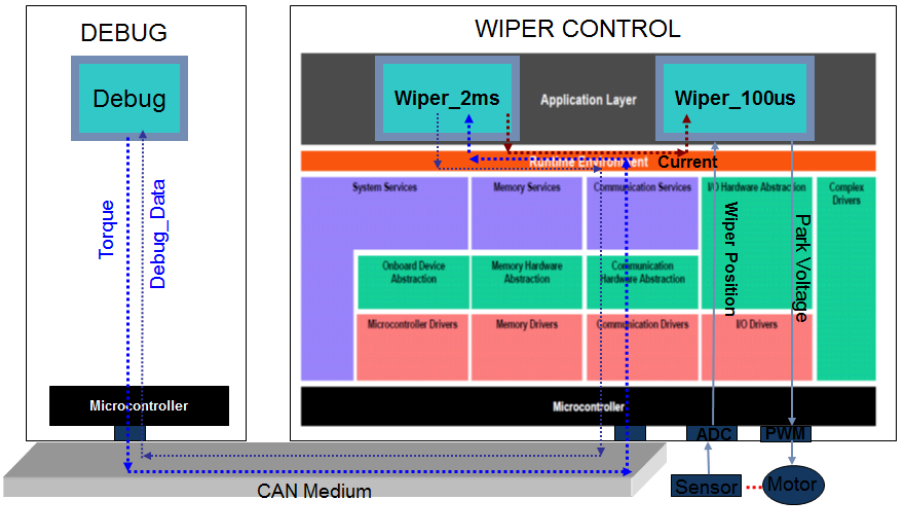

Figure 5.1.Inter-ECU Example

The application software components in our example project (and their main properties) are as follows:

1. The **Debug** component (running on the “Debug” ECU) provides torque data every 2ms. This data is transmitted as Frames via the CAN bus network.

2. The **Wiper_2ms** component (running on the "WiperControl" ECU) receives, via the AR BSW Com stack, the Torque Frames from the CAN bus. It reads this torque data every 2ms and converts it into Current data, which it then sends to the Wiper_100us component (which is also running on the "Wiper-Control" ECU).
3. The **Wiper_100us** component reads the Current data every 100us and converts it into a Park Voltage, which is used for controlling an electrical motor.
4. The electrical motor is controlled through the pulse width modulated signal from PWM. The Sensor senses the motor position and sends an Analog Signal to an ADC (Analogue to Digital Converter), which converts the analog signal to a digital signal and then sends it to the **Wiper_100us** component.

Note: The PWM, the electrical motor and ADC are not covered in this documentation.

5. Debug information is fed back to the Debug component from Wiper_2ms via the CAN bus and AUTOSAR-compatible Basic Software (BSW) in both ECUs, which is configured to the communication needs of the two components.

The remainder of this section will guide you through the steps required for using ISOLAR-A V11.0 (or later) to create the example project.

Once finished, you will have successfully created your own copy of the example project, including all the associated artifacts.

As part of the above process, you will also learn how to import an example project ("InterECU_4x") into your ISOLAR-A workspace, and how to import files from other projects into your own project.

NOTE

1. This example project is available as part of the installation process of ISOLAR-A. The entire project's contents can be loaded into an open workspace by selecting **File** > **New** > **Example…** > **ISOLAR-A** > **AUTOSAR Examples** > **InterECU_4x** (See the Hints and Advice section for more information).
2. The image provided for the BSW Module Description in this document is not the latest, it will be updated with new image in the further releases.
3. In the AUTOSAR Adaptive Support section there are two icons at the top of the AR Explorer labelled CP and AP. By default, CP [classic platform] is enabled, but AP [adaptive platform] can be enabled by clicking on the AP icon.

5.2 Creating a new Project

5.2.1 Starting ISOLAR-A

ISOLAR-A can be started from wherever it has been installed, which is typically under *C:\Programs\ETAS*, or you can start it via the **Windows Start Menu** > **ETAS** > **ISOLAR-A**.

5.2.2 Creating a new AUTOSAR Project

ISOLAR-A offers various options for creating a new AUTOSAR project. The ones shown below are via the "AR Explorer".

As shown below, you can right-click anywhere in the AR Explorer tab and then select **New** > **AUTOSAR Project**, or just click on the "New Autosar Project" link.

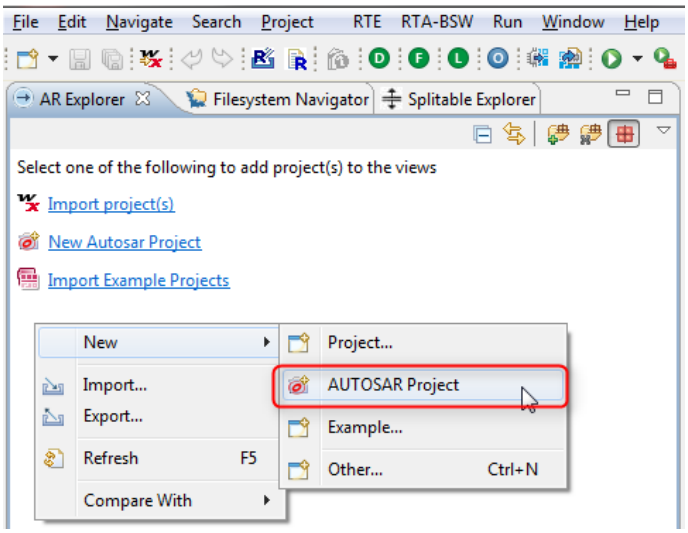

Figure 5.2.Creating a new AUTOSAR Project

Whichever option you use, you'll be taken to the "AUTOSAR Project" wizard.

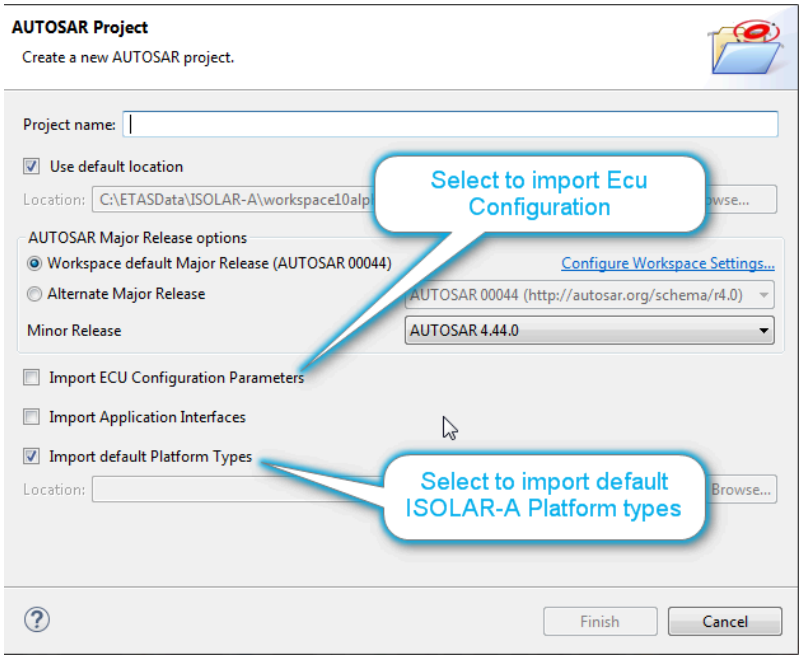

Figure 5.3.New AUTOSAR Project Wizard

1. Enter the “Project name” as *WiperControl_4x*.
2. Set the “Release Version” as *AUTOSAR 00044*.
3. Tick the “Import ECU Configuration Parameters” and “Import Default Platform Types” options.

Note: These parameters can also be loaded later by right-clicking on the project and selecting ISOLAR-A > Import EcuParamDef. You can also import Application Interfaces of AUTOSAR if you want to use them in your project.

4. When you click **Finish**, your new project will be created and it will be visible in the AR Explorer.

5.2.2.1 Importing AR Interfaces, Data Types and Components

AUTOSAR specifies a set of standardized Interfaces and Data Types, and these will be available to you in ISOLAR-A as follows:

- First, you need to import into your workspace an example project called **InterECU_4x**. This existing project contains some files that we will later on be copying into our newly created **WiperControl_4x** project. (See the Hints and Advice section for more information on importing an existing project into your workspace).
- Then, you need to import and copy the *01_TypesAndInterface.arxml* file, which is also available in the **InterECU_4x** example project. This import can be done either through **File** > **Import**, or by copying the file from the example project and pasting it into the **WiperControl_4x** project. (Note: When

copying files in this way, it is recommended that you use the "File System Navigator", which lists all the files contained in a project).

Our example project also requires the *ISOLAR_PlatformTypes.arxml* file from the **InterECU_4x** project. If you ticked the "Import Default Platform Types" option in the AUTOSAR Project Wizard, this file will already be imported into your **Wiper-Control_4x** project. If not, you will need to copy/paste it using the same method as described above.

The *01_TypesAndInterface.arxml* and *ISOLAR_PlatformTypes.arxml* files contain the following elements that are required in our example project:

01_TypesAndInterface.arxml

- Application Primitive Data Types
- Data Constraints
- ImplementationDataTypes
- Port Interfaces
  - Current_SRI [Contains DEP_Current – Variable Data Prototype]
  - Torque_SRI [Contains DEP_Torque – Variable Data Prototype]
  - Debug_SRI [Contains Variable Data Prototypes] (DEP_Debug_SINT8, DEP_Debug_SINT16, DEP_Debug_SINT32)

ISOLAR_PlatformTypes.arxml

- Data Constraints
- Sw Base Types
- Implementation Data Types

5.3 Naming Rules

5.3.1 Introduction

ISOLAR-A allows you to create naming convention rules that can be used to automatically determine the names of elements that are auto-generated within your projects. Before you can create any of these rules, you first need to enable the rules feature itself.

- Go to **Window** > **Preferences** > **ISOLAR-A** > **User Naming Conventions**
- Tick the "Enable User Defined Naming Convention Rules" option, then click **Apply**

You're now ready to start creating naming rules.

5.3.2 Creating a Naming Rule for AR Elements

Follow these steps to create a new naming rule for AR Elements.

1. Go to **Window** > **Preferences** > **ISOLAR-A** > **User Naming Conventions** > **Rule Configurator**.

Note: On the left-hand side of the Rule Configurator you can select the Profile into which you wish to add your new naming rule. As you can see in the example below, you can create additional profiles if you wish, but in our example we're using the "Default" profile.

2. Click the **Add** button on the right-hand side to add your new naming rule.

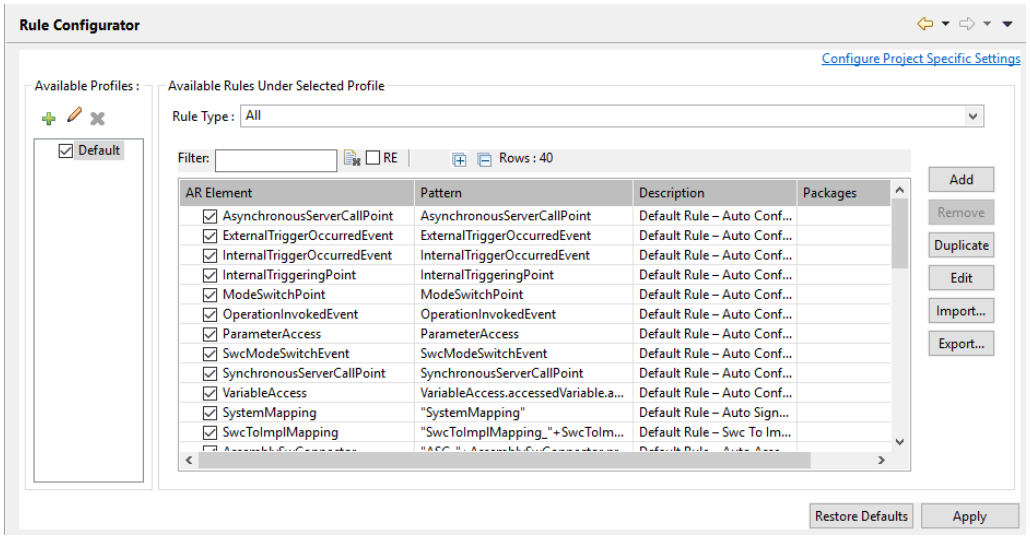

Figure 5.4.Add a new Naming Rule

3. Set the following fields in the "Naming Rule Configurator" dialog:

- **AR Element:** Enter RP, then press <CTRL+Space> and select RPortPrototype.
- **Description:** Enter "RPortPrototype interface name rule".
- **Rule Pattern:** Press <CTRL+Space>, then click on "Prefix" and replace it with a suitable prefix that will be used for the automatic naming of RPortProtoype interface elements.

Then, as shown in the example below, follow it with +RPortPrototype.requiredInterface. (Note: Some rules might enforce a prefix, others might enforce a suffix, or both).

4. Click **Finish** to exit the "Naming Rule Configurator" dialog.
5. Click **Apply** and then **OK** to save your changes and exit the Preferences.

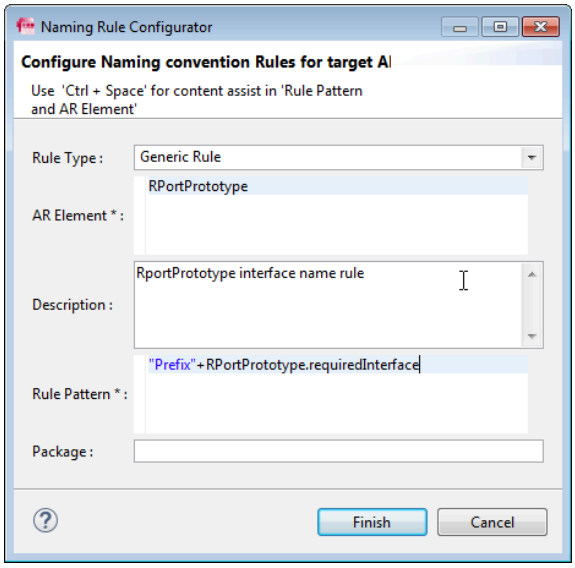

Figure 5.5.Configure a Naming Rule

5.3.3 Configuring Project Specific Rules

With one or more naming rules set up, you can use them to enforce the configured naming conventions in a project. To do this, you must first enable the rule(s) within the project.

1. At the top of the Rule Configurator (see Figure 5.5), you'll see there is a link for **Configure Project Specific Settings**.
2. Click on that link, select the project in which you want to use the rule, then click **OK**.

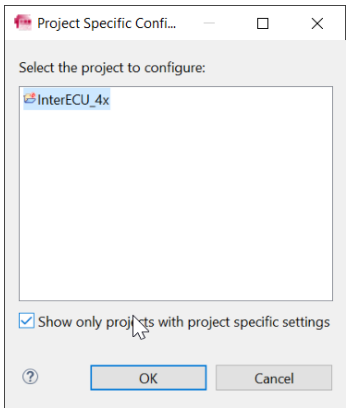

Figure 5.6.Select the Project

The current properties for the selected project will be displayed.

3. Tick the "Enable project specific settings" option.

As we saw earlier, rules can be saved to a profile. In the example below, we've selected the "Sample" profile, and we can see the rules that are available within that profile.

4. You can now select the rule(s) that you wish to have specifically enabled in the project.
5. Then click **Apply and Close** to save your changes and exit the dialog.

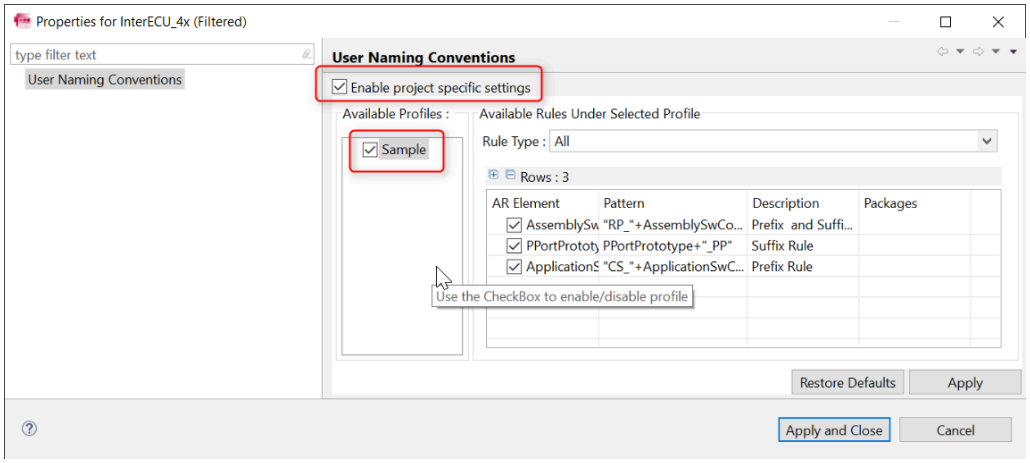

Figure 5.7.Enable Project Specific Settings

5.3.4 Applying the Naming Rule for AR Elements

Having enabled one or more rules in a project, you can then use those rules to apply the required naming convention to AR Elements that you create.

1. In AR Explorer, expand the *Components* folder, right click on **Debug**, then select **New Child** › **Rport prototype**.
2. Click the newly created **RPort RPortPrototype_0** and set the Required Interface to "Current_SRI"
3. Then, as shown below, click RPort RPortPrototype_0 again and select **ISOLAR-A** › **Apply Naming Rule**.

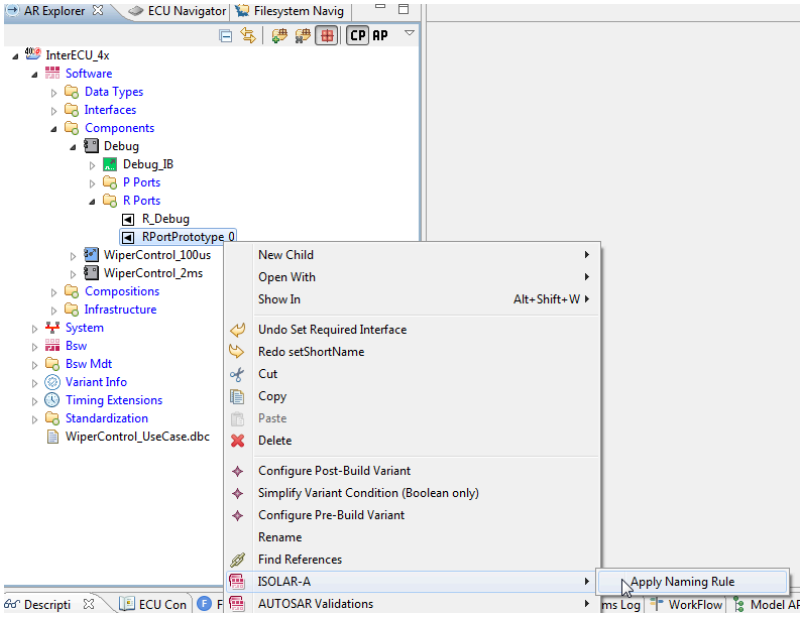

Figure 5.8.Apply the Naming Rule

Any element created from any of the editors in ISOLAR-A will also, by default, inherit the naming rule. For example, double-click on the **Debug** component to open it in the Component Editor, click on **PPorts** and create a PPortPrototype, select the interface and click **OK**.

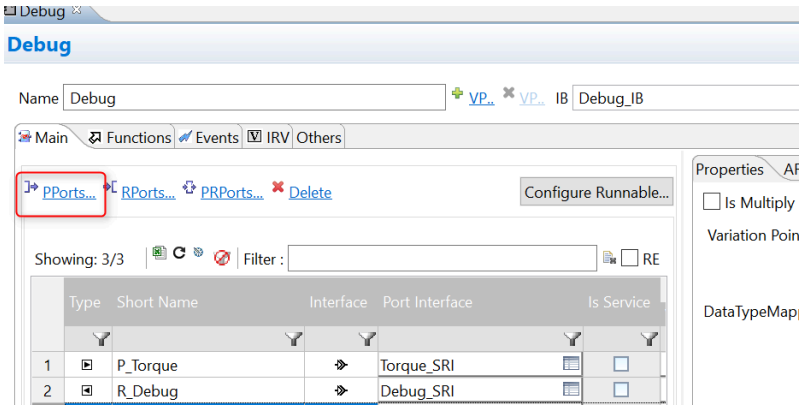

Figure 5.9.Create PPortPrototype

5.4 Create an AUTOSAR Software Template

5.4.1 Introduction

This section describes how to create an AUTOSAR Software Template comprising an AUTOSAR **Software Component (SWC)** and a **Composition**.

5.4.2 Creating an AUTOSAR Software Component (SWC)

We'll begin by creating the AUTOSAR Software Component (SWC), together with other related elements, such as Ports and Internal Behaviours.

1. As shown below, we first create a new AUTOSAR XML-formatted file (.arxml) and name it as *02_WiperControl_100us.arxml*.
2. Tick the "With an ARPackage" option, and set the package name as *WiperControl_100us*.

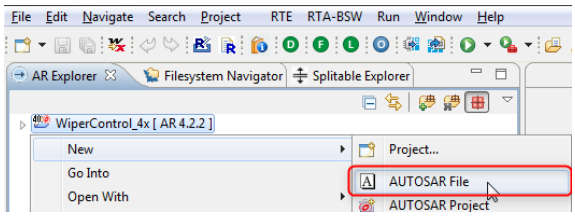

Figure 5.10.Create new AUTOSAR file

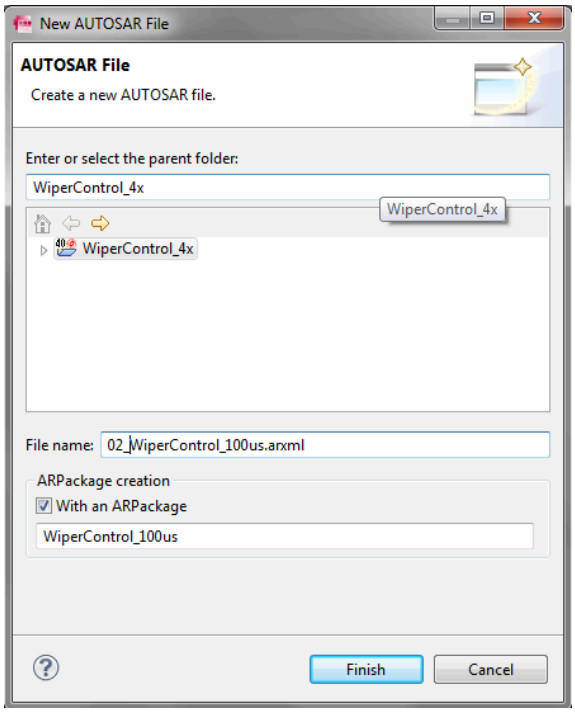

Figure 5.11.New AUTOSAR file dialog

One or more AUTOSAR objects (e.g. SWCs) can be stored in the *02_WiperControl_100us.arxml* file. By specifying an AR Package name, all its contents (e.g. SWC objects) inherit a name space.

Now we're ready to create the AUTOSAR Software Component (SWC).

3. First we create a **Sensor Actuator Sw Component Type**.

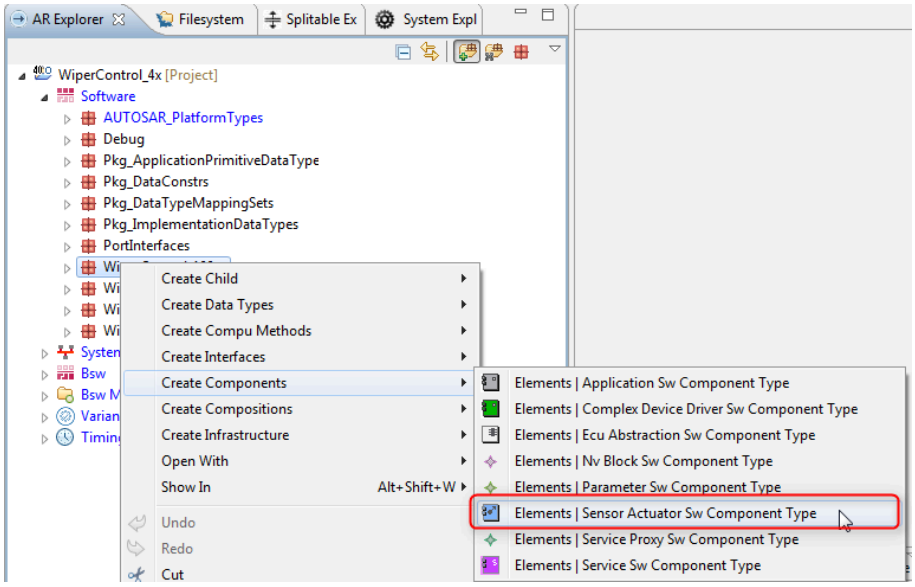

Figure 5.12.Create Components menu

4. And then we rename the SWC type to *WiperControl_100us*.

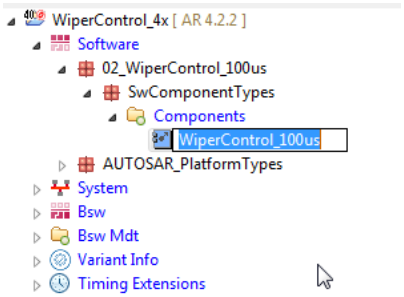

Figure 5.13.Rename the SWC Type

5. We can then open **WiperControl_100us** in the "Component Editor", from where we can configure its properties, including adding Ports.

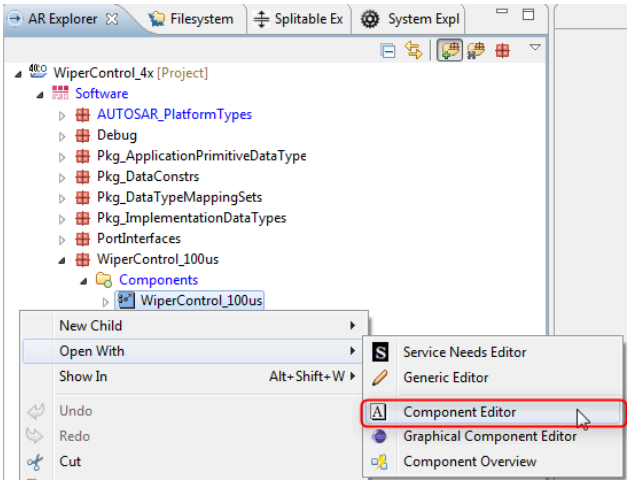

Figure 5.14.Open WiperControl_100us in the Component Editor

5.4.2.1 Adding Ports to the SWC

SWCs exchange information through require (receive) or Provide (send) Ports, and these Ports are connected to Ports of other SWCs (Assembly Connection) or Compositions. Ports are “typed” by their Interfaces, which can be either Sender-Receiver (SRI) or Client-Server (CSI).

1. Click the **RPorts...** link to begin the creation of the Requestor Port.

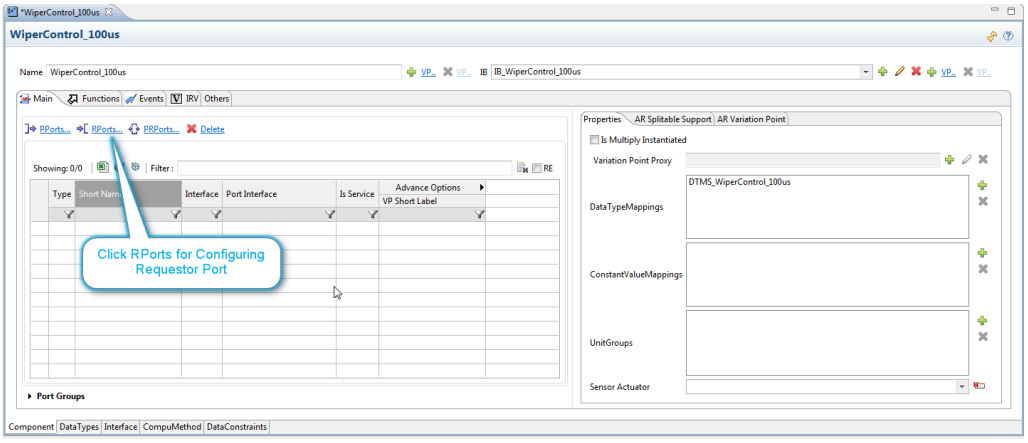

Figure 5.15.Component Editor (Component Page)

2. In the “Port Creation” dialog, select **Current_SRI** (Sender-Receiver-Interface), and then click **OK** to create the Requester Port.

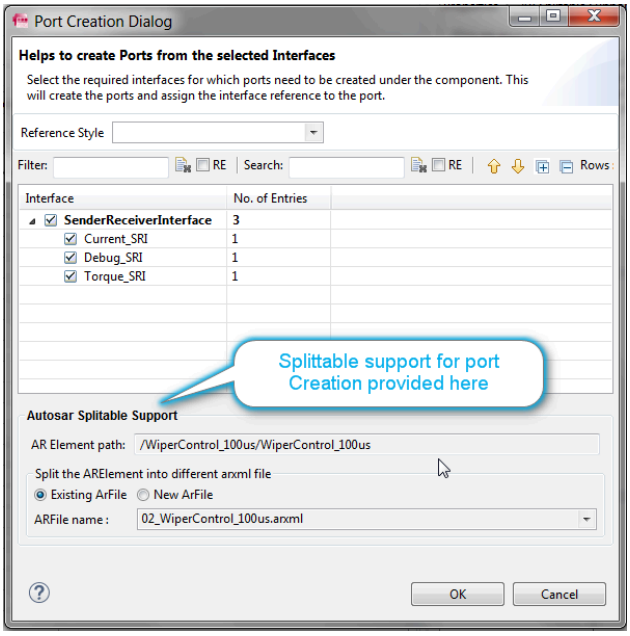

Figure 5.16.Port Creation dialog

3. Back in the “Component Editor”, rename the Require Port from *RPortPrototype_0* to *R_Current*.

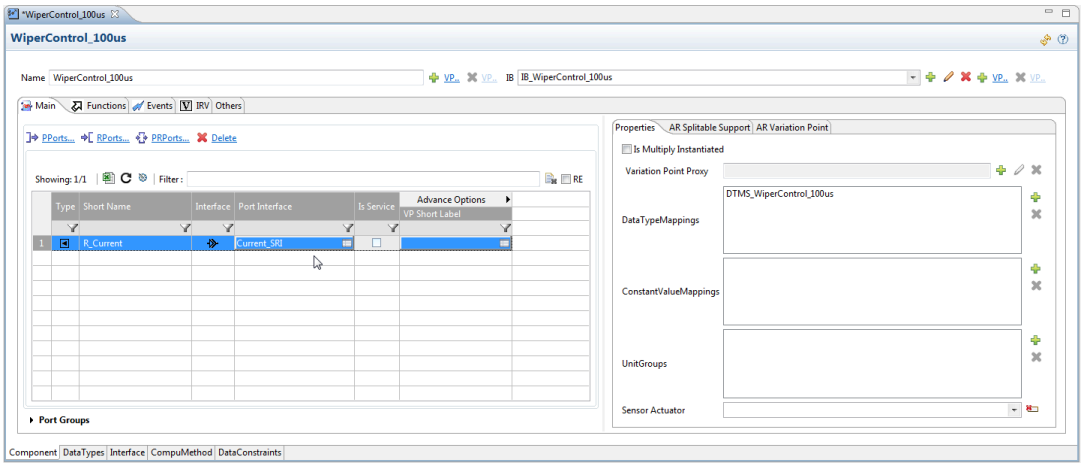

Figure 5.17.Rename the RPort

5.4.2.2 Adding an Internal Behavior to the SWC

We’re now going to add an Internal Behavior to the **WiperControl_100us** SWC, which will describe its “dynamic”/timed behavior. (Note: Calibration Parameter SWCs do not have an Internal Behavior).

1. Click on the **Add Internal Behavior** hyperlink and enter *IB_WiperCon-*

trol_100us as the name of the Internal Behavior that you wish to create.

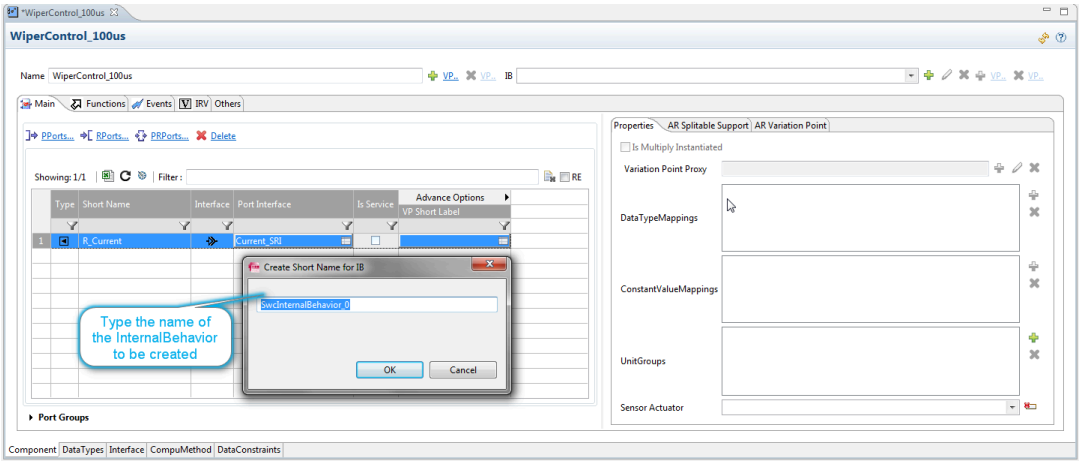

Figure 5.18.Create and Configure the Internal Behavior

In the following steps we're going to:

- Add a **Runnable Entity** to the Internal Behavior.
- Add a **DataAccessPoint** (Data Read Access) to the Runnable.
- Create a **Timing Event** to trigger the Runnable.
- Add a **DataTypeMappingSet** to the Internal Behavior.

Runnable Entities

An SWC may contain one or more Runnable Entities (often referred to as "Runnables"), each of which contains executable pieces of code.

All the Runnables of an SWC are mapped onto one core of a multi-core microcontroller of one ECU in a system.

(Note: Calibration Parameter SWCs do not have Runnable Entities because they do not have an Internal Behavior).

1. The **Functions** tab in the "Component Editor" is where we create and configure an Internal Behavior's Runnable Entities.

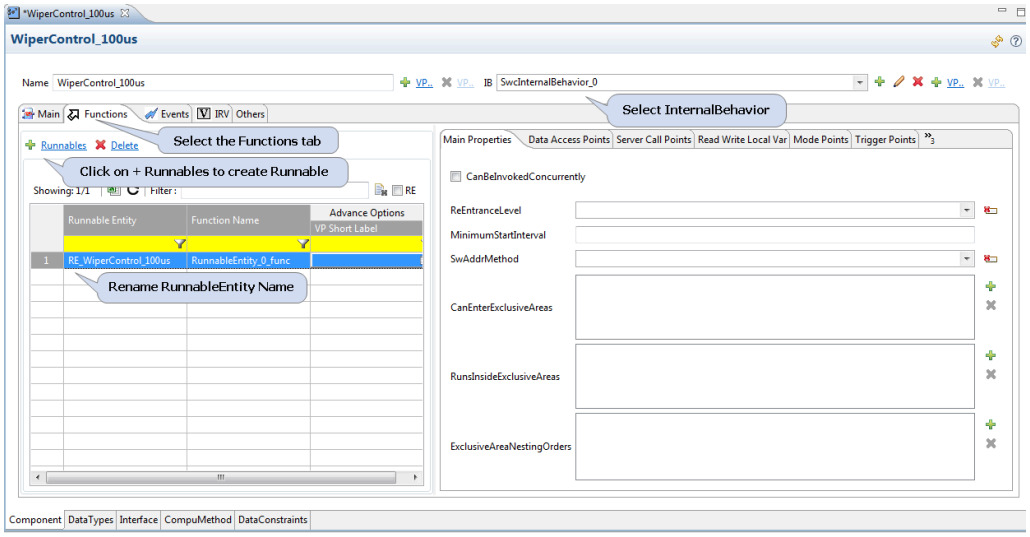

Figure 5.19.Create and Configure IB's Runnable Entities

2. The **Data Access Points** sub-tab is where we create and configure Data Access Points inside the Runnable. In our example we're adding a DataReadAccess data access point.

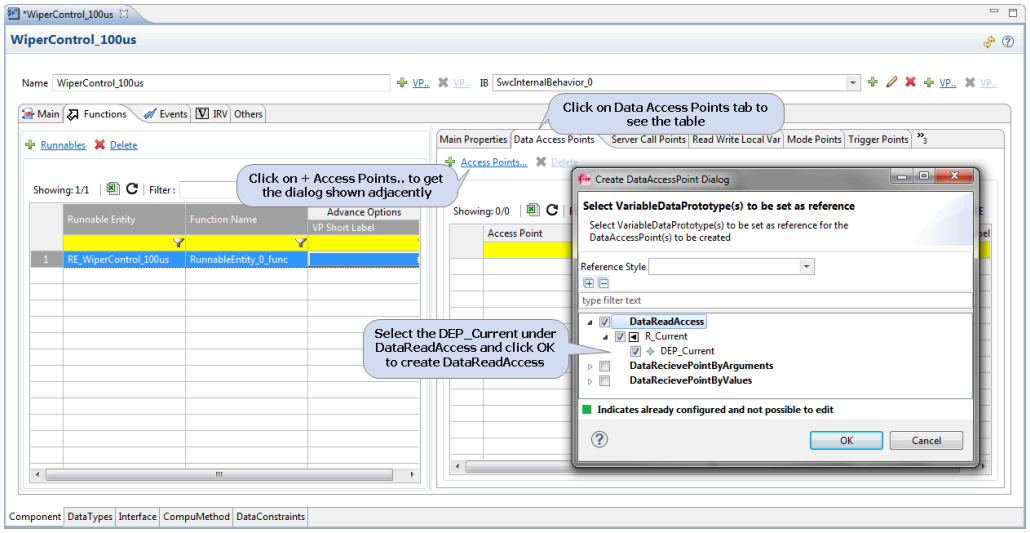

Figure 5.20.Create and Configure Runnable's Data Access Points

3. The next step is to create a **Timing Event** to trigger the Runnable, which is done in the "Events" tab.

As shown in the two images below, click the "+" to open the "TimingEvent Creation" dialog and select the Runnable Entity that is to be the subject of this Timing Event.

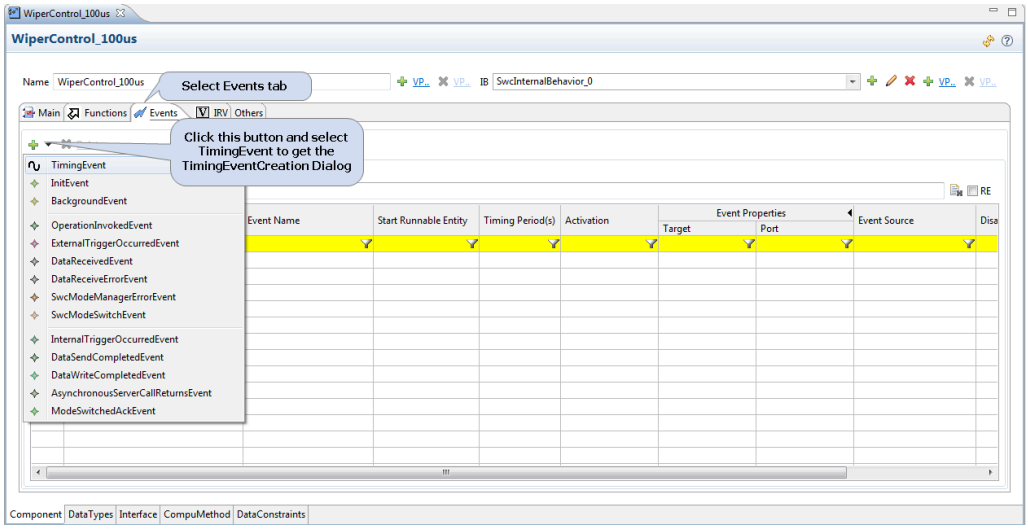

Figure 5.21.Create a Timing Event

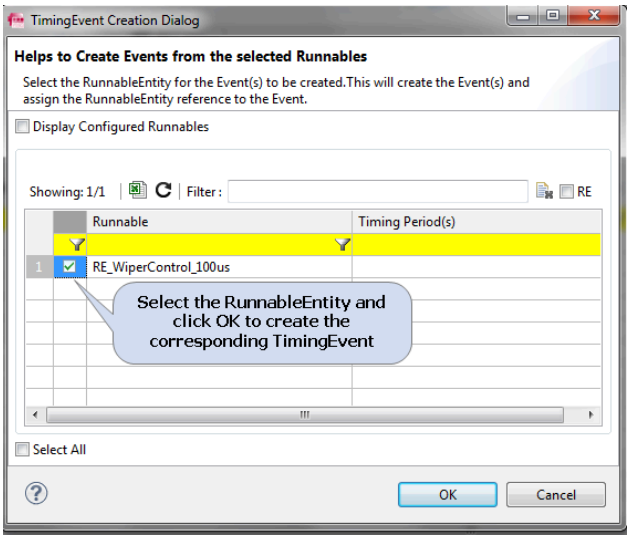

Figure 5.22.Component Editor (Events tab)

4. The final step is to add the **Data Type Mapping Sets** under the Internal Behavior, as shown in the images below.

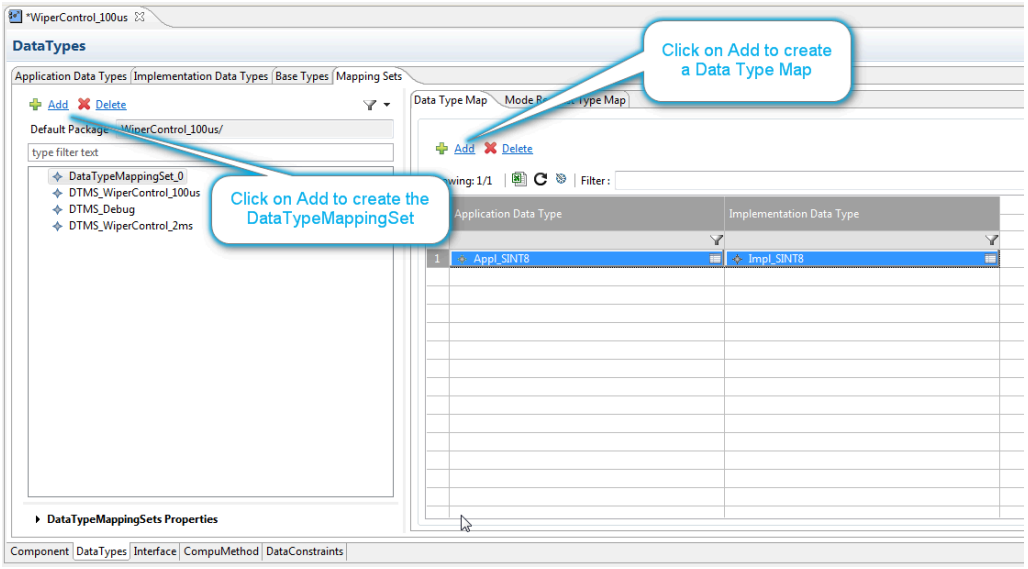

Figure 5.23.Create IB's Data Type Mapping Sets

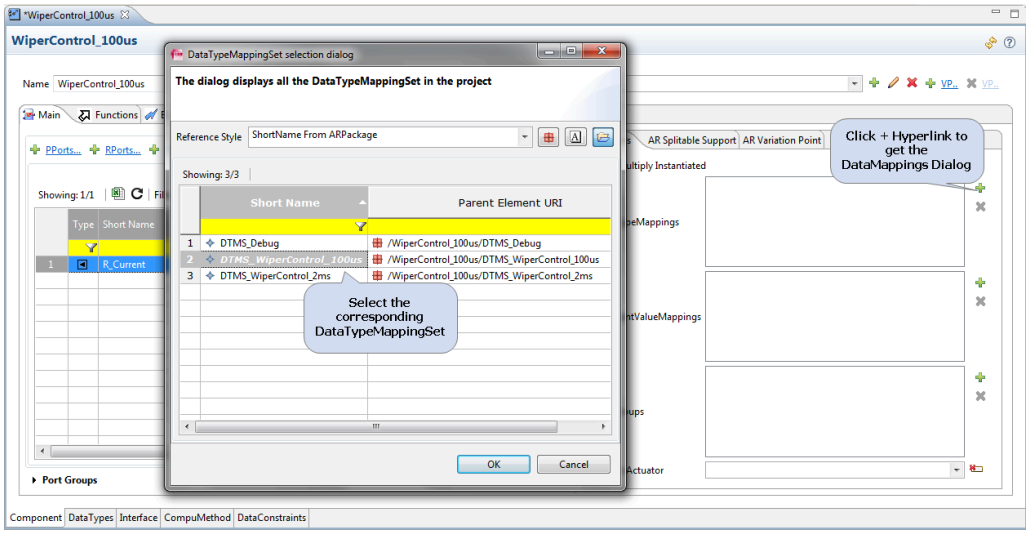

Figure 5.24.Adding multiple References

5.4.2.3 Automatic creation of Access Points and Events

Access Points and Events can be created automatically, and the steps below show you how to do this for the **WiperControl_2ms** component.

1. Right-click on the component in the AR Explorer and select the “Configure Runnable” option.

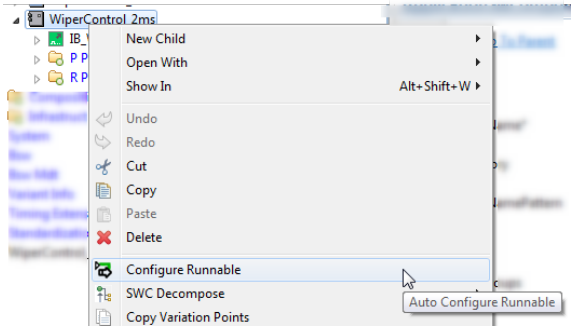

Figure 5.25.Configure Runnable from AR Explorer

2. Or you can just open the Component Editor for the desired component, and click on the **Configure Runnable** button in the top right-hand corner.

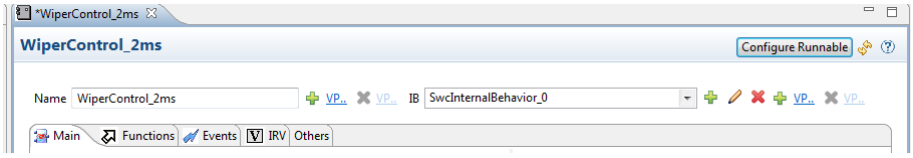

Figure 5.26.Configure Runnable from Component Editor

3. Whichever of the above two methods you choose, you'll be taken to the Component page where you can configure the Runnables.

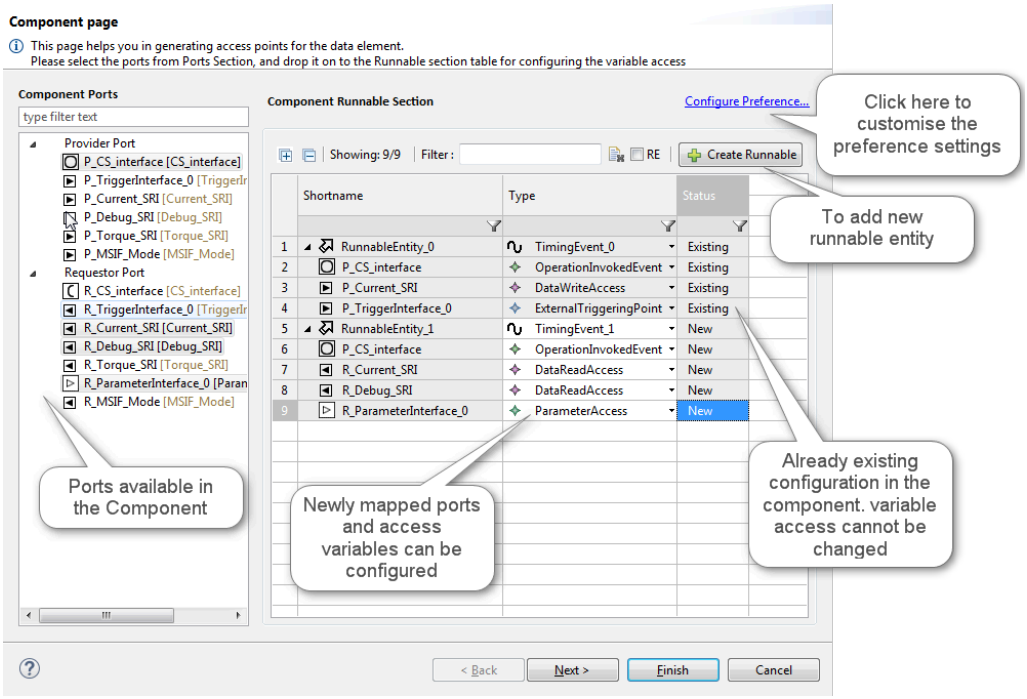

Figure 5.27.Component page

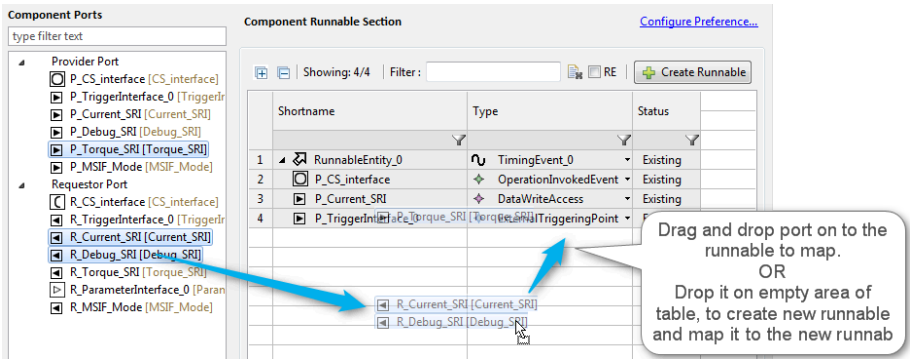

Figure 5.28.Map Port to Runnable to create Access Points

4. When you click **Next**, you'll be taken to the "DataTypeMappingSet" page.

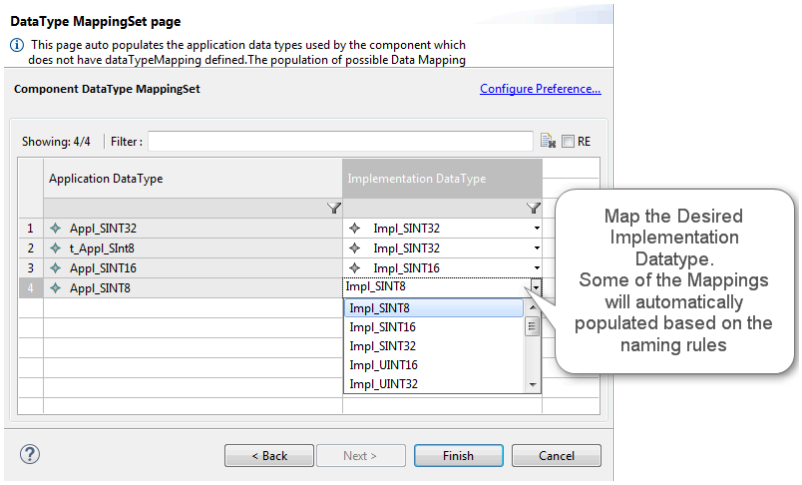

Figure 5.29.DataTypeMappingSet populated

5. When you click **Finish**, the Access points for the ports in the respective Runnable will be created, and the DataTypeMappingSet will be generated.

5.4.2.4 Using the SWC Editor

The SWC Editor can be used to edit various properties of a SWC. This editing is based on a specific use case, and these use cases are enabled/customized based on the type of Component on which the Editor is opened. See the ISOLAR-A Reference Guide for more details.

5.4.3 Importing SWCs into your Project

To complete our “Wiper Control” example, and to complement the **WiperControl_100us** SWC that we created above, we now need to create SWCs for **Debug** and **WiperControl_2ms**. Rather than create these SWCs from scratch, you can import them from another project.

So, from the InterECU_4x example project that you’ve already imported into your workspace in Section 5.2.2.1, import into your **WiperControl_4x** project the *03_WiperControl_2ms.arxml* and *04_Debug.arxml* files.

After the import is done, and with the **Abstract ArPackage** option enabled, you should see all 3 SWCs in the AR Explorer.

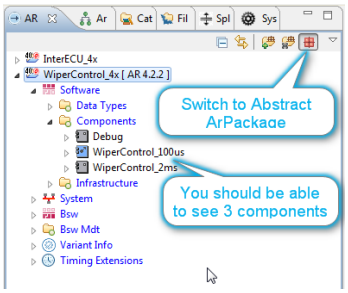

Figure 5.30.AR Explorer (with Abstract ArPackage enabled)

5.4.4 Creating a Composition (and instances of SWC)

Having successfully created our SWCs and configured them with Ports and Internal Behaviors, the next step is to create a Composition. The steps below explain how to create and configure a composition called **WiperControl_Composition**.

5.4.4.1 Create a new Composition (using AR Packages)

1. Right click on **Software**, then select **Create Compositions** > **Elements I Composition Sw Component Type**.

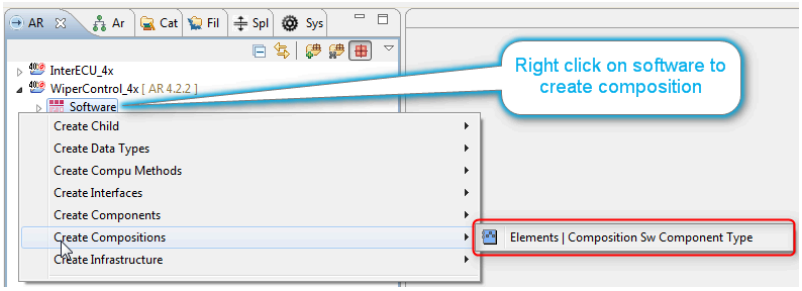

Figure 5.31.Create Composition

2. This will take you to the “New AR Element Creation” wizard for the CompositionSwComponentType element.

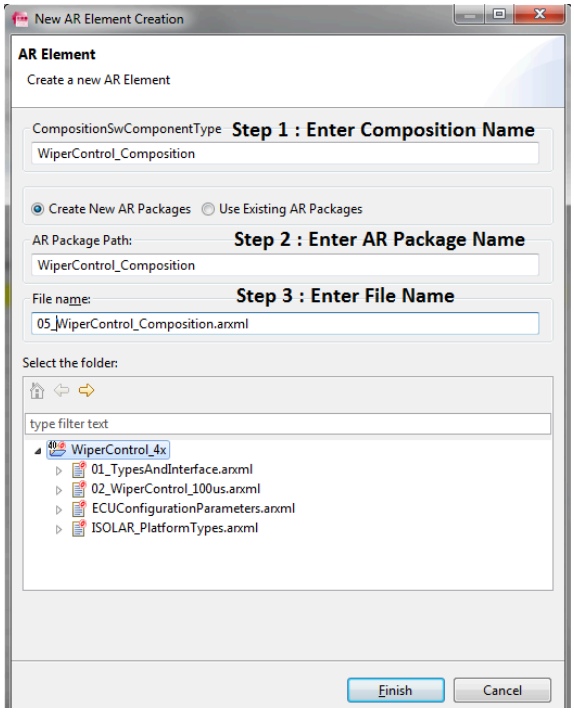

Figure 5.32.Create new AR Element

3. Configure the new AR Element as follows:

   - **CompositionSwComponentType:** *WiperControl_Composition*
   - **AR Package Path:** *WiperControl_Composition*
   - **File name:** *05_WiperControl_Composition.arxml*

4. The *05_WiperControl_Composition.arxml file* will create a new ARPackage called “WiperControl_Composition”, which will in turn create a **CompositionSwComponentType**, also named “WiperControl_Composition”.

5.4.4.2 Graphical Composition using the Composition Editor

Now that a composition has been created, it can be filled with atomic SWCs and /or other compositions. ISOLAR-A provides several editors for this, one of which is the “Composition Editor”.

1. Open the **WiperControl_Composition** with the “Composition Editor” as shown here.

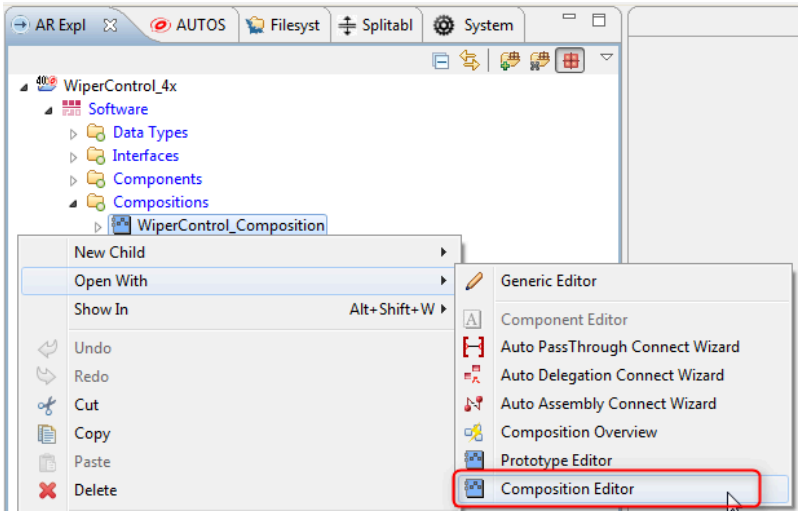

Figure 5.33.Open the Composition Editor

2. Within the “Composition Editor”, the Component Prototypes can be created and connected.

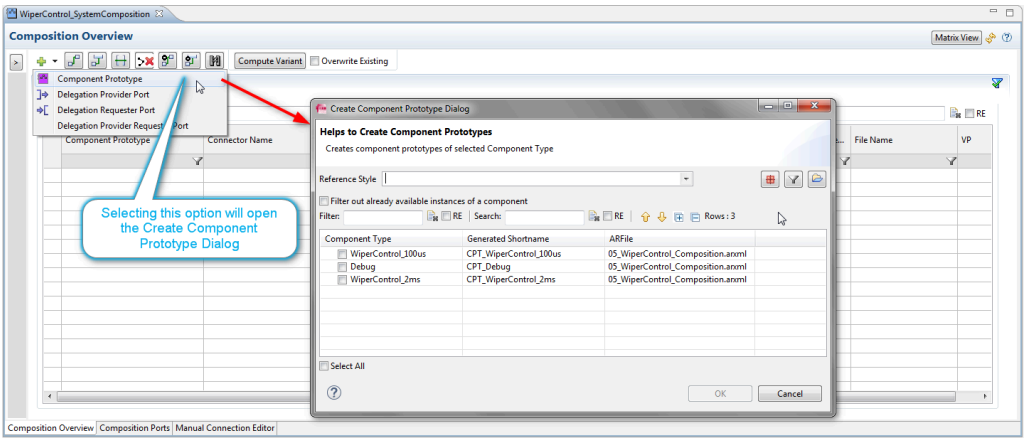

Figure 5.34.Create and Connect Component Prototypes

3. For each **Component Type** selected in the dialog, a corresponding **SWC Prototype** will be created (e.g. *CPT_WiperControl_2ms* for the WiperControl_2ms Component Type). Note that CPT_<Component Name> is not mandatory, but it is a common naming for prototypes. To select all 3 Component Types, tick the “Select All” option, then click **OK**.

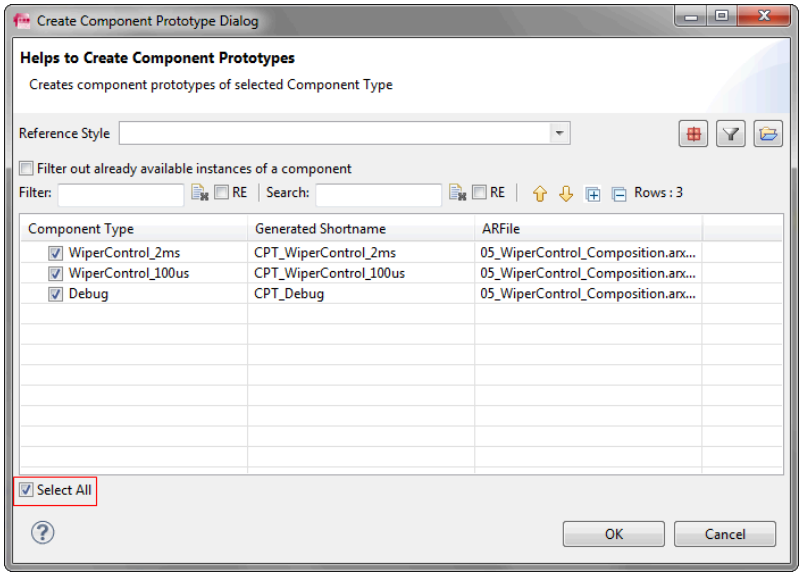

Figure 5.35.Create Component Prototype

4. Once all the three SWC Prototypes have been created, the “Composition Editor” will look like this.

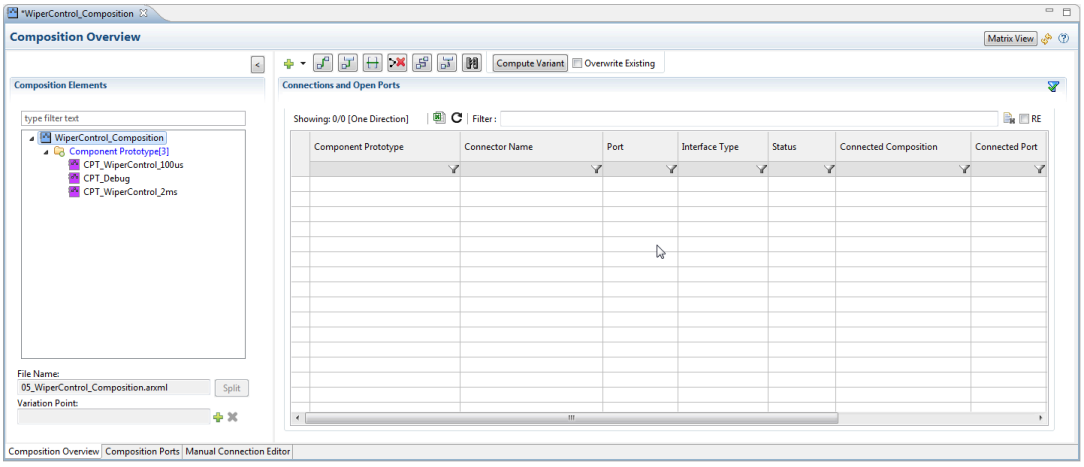

Figure 5.36.Component Editor with Prototypes created

5.4.4.3 Creating Assembly Connections between SWCs

The next step is to create Assembly Connections, which can be created either manually or automatically, as described in the two sub-sections below.

Manual creation of Assembly Connections

1. Select the “Manual Connection Editor” tab.

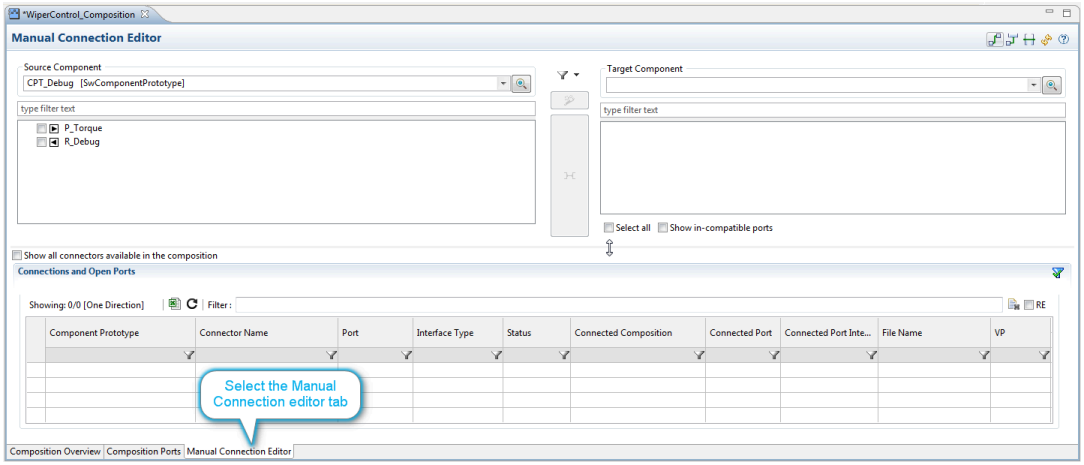

Figure 5.37.Manual Connection Editor

2. Selecting the Port in the “Source Component” pane displays the compatible port in the Target Component pane. Select the **target port** and click the **Connect** button to create the connection.

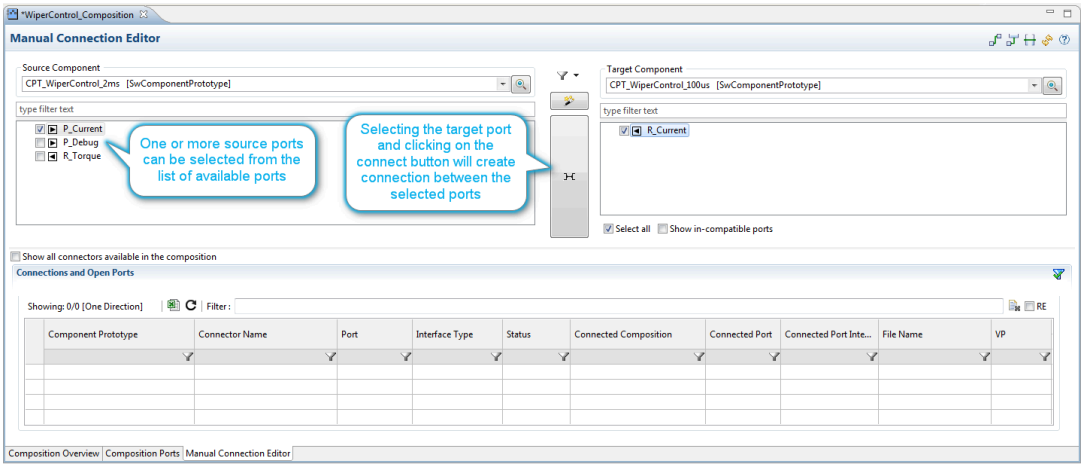

Figure 5.38.Create the Connection

3. Once the Connection has been created it will be displayed in the “Connections and Open Ports” table at the bottom of the Manual Connection Editor.

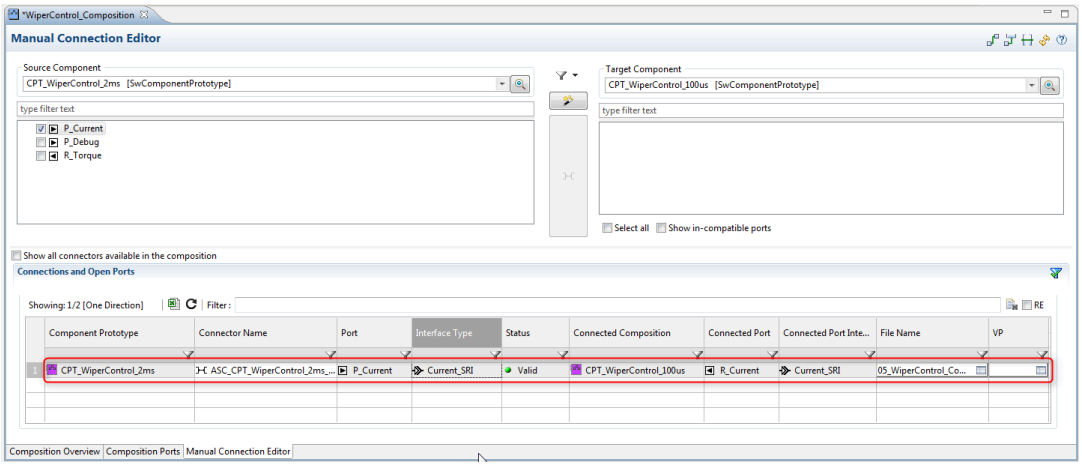

Figure 5.39.Assembly Connection created

Automatic creation of Assembly Connections

1. We can create all the remaining possible connections automatically through the “Connection” wizard, which is accessed via the button shown here.

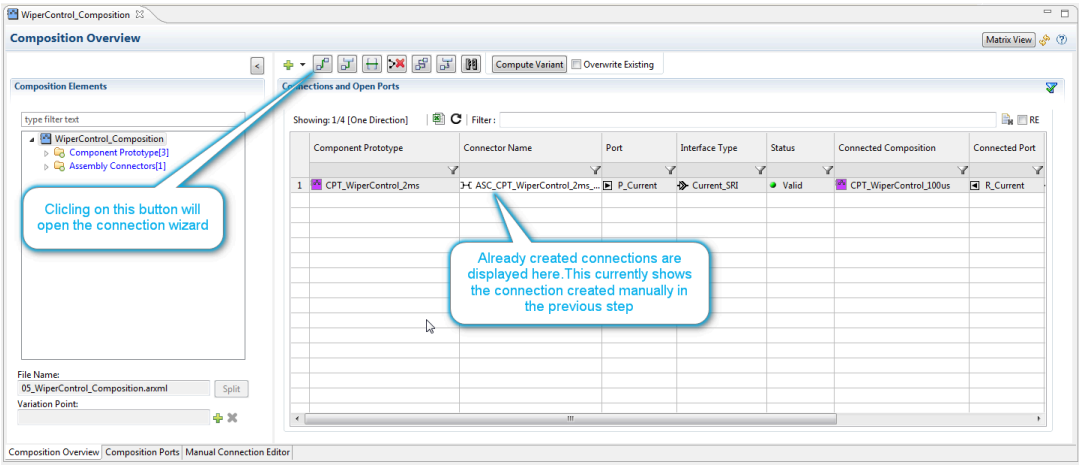

Figure 5.40.The Connection wizard

2. The “Connection” wizard displays all the possible connections. In the “Auto-Connection” page, tick the “Select all connections” option, and then click **Next**.

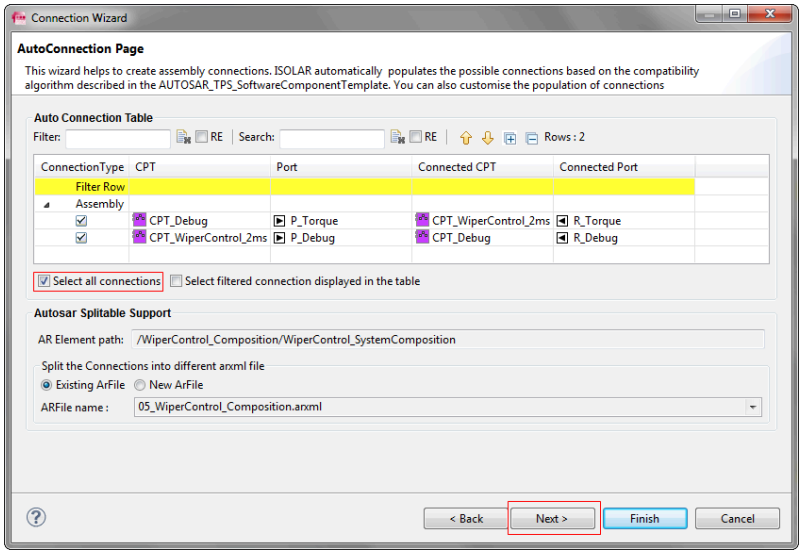

Figure 5.41.The Connection wizard (AutoConnection page)

3. The “Connection Overview” page displays all the connections selected in the “AutoConnection” page. When you click **Finish**, all the selected connections will be automatically created.

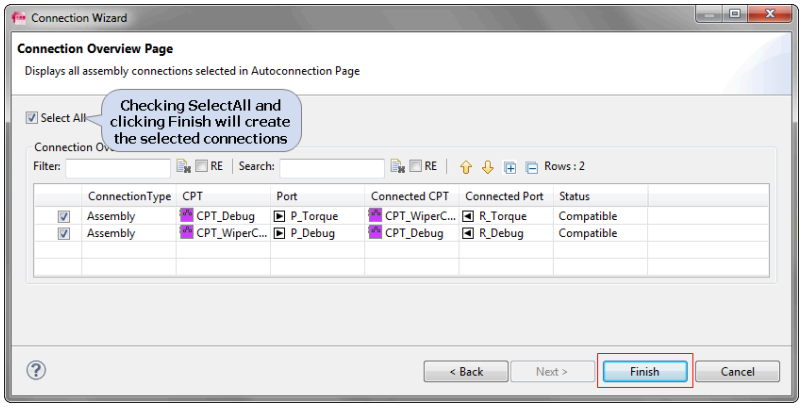

Figure 5.42.The Connection wizard (Connection Overview page)

Note: Connections can also be created in the command line mode. See the ISOLAR-A Help Reference Manual for more information.

5.4.5 Auto Configure Composition

The “Auto Configure Composition” dialog automates the configuration of a Composition by providing a dialog in which you can do the following:

- Create new component prototypes.
- Create Assembly and Delegation connections.
- Delegate open inner ports.

- Create/update SwcToEcuMapping by selecting preferences.

The dialog also helps you to locate and delete erroneous connections.

5.4.5.1 Invocation

The Auto Configure Composition dialog can be invoked from the “Compositions” Context Menu

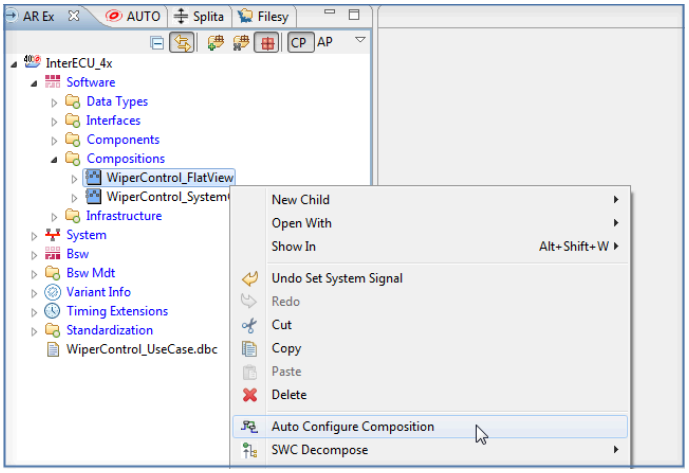

Figure 5.43.Auto Configure Composition dialog (from Compositions context menu)

Or it can be invoked using the Action Button in the Composition Editor.

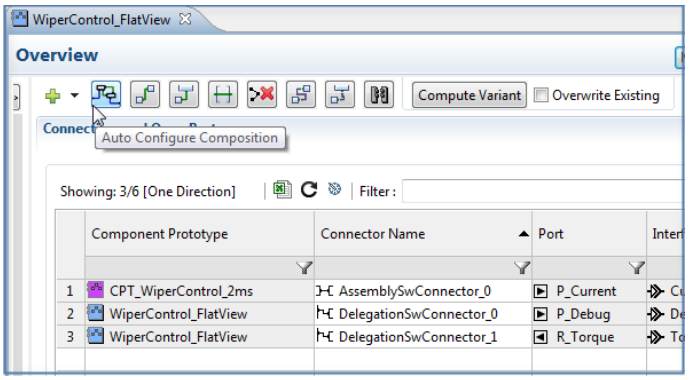

Figure 5.44.Auto Configure Composition dialog (from Composition Editor)

5.4.5.2 Composition Customization Page

The “Composition Customization” page consists of two sections.

- **Available Components:** Contains all components for active projects.
- **Component Prototypes:** Contains all component prototypes for the selected Composition.

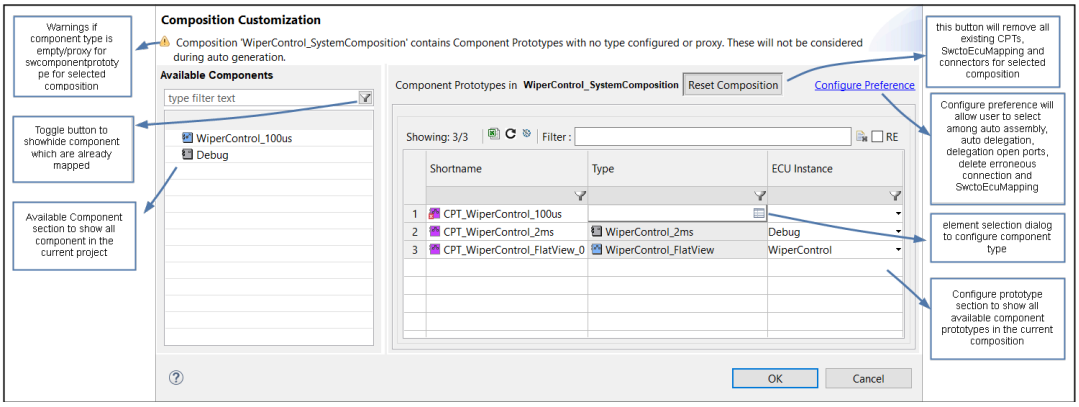

Figure 5.45.Composition Customization page

A component can be dragged from the “Available Components” pane to the “Component Prototypes” section, and when you click **OK** it creates all the new Component Prototypes.

The **Reset Configuration** button will clear all SwComponentPrototype, Connectors and Data Mappings for the selected Composition.

The Ecu Instance for Component Prototypes can be updated, but the update will only take place after you click **OK**, and only if the respective preference option is selected.

5.5 Configure an AUTOSAR System Template

5.5.1 Introduction

This section shows you how to configure an AUTOSAR System Template.

5.5.2 Basic Approach

Once the Software has been designed, the next step in the AUTOSAR Methodology is getting the System Description. In an AUTOSAR System Template configuration, there are the following configuration parameters:

1. System Description Information [Communication Matrix]

- Configuration of Clusters, ECU Instances, Frames, PDUs, and Signals.
- Association of Frames and Signals to ECUs.
- Association of ECUs to Clusters.

2. System Mapping [SYSTEM_EXTRACT]

- SWC to ECU Mapping - Mapping of designed Software Components to ECUs.
- Data Mapping - Mapping of Signals (from a System Description) to Data Elements within Ports of SWCs.

3. ECU Extract

- A process that “flattens” the Hierarchical System into a Flat System by creating a Flat View, which also generates Flat Maps for the Component Prototypes, Variables etc.

5.5.3 Importing Communication Matrix Information

The first step is to create the Communication Matrix, which can be done in one of two ways:

1. Configure a System **manually** by using the AR Explorer and Properties views (this can be very time consuming).
2. **Import** an existing system in a legacy format like DBC (CAN®), FIBEX (CAN®, FLEXRAY®), and LDF (LIN®) to create the AUTOSAR System Description. ISOLAR-A supports DBC, FIBEX v2.0, 2.0+, 3.0, 4.0 and LDF v2.0, 2.1 and 2.2

5.5.3.1 DBC Import Example

The steps below demonstrate how to import a DBC file to create a System Description [Communication Matrix] configuration:

1. Launch the DBC Importer by clicking the “D” Button from tool bar.

Figure 5.46.Invoking the DBC Importer

2. In the “Select File” page of the Import DBC wizard, locate and select the DBC file that you wish to import.

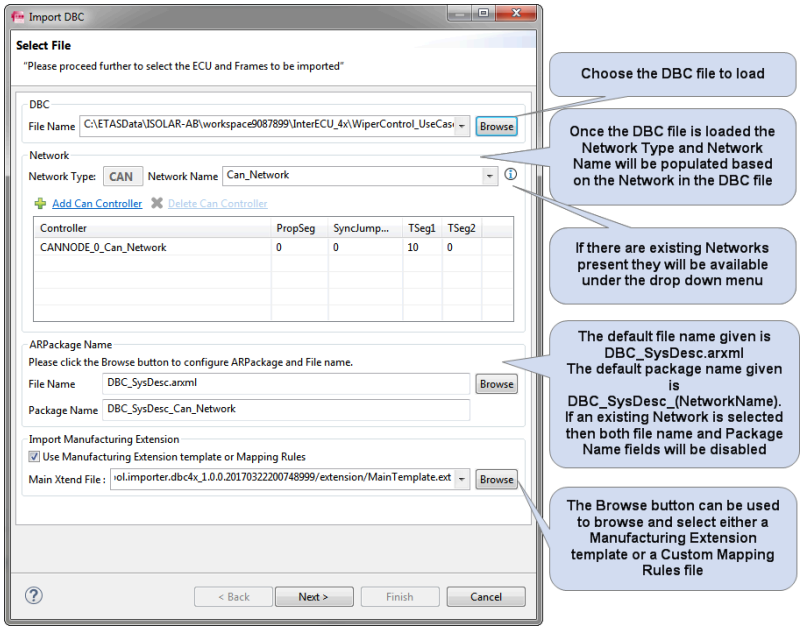

Figure 5.47.DBC Importer - Select File

3. If the System Description (.arxml) is anything other than the default, it can be located and selected manually, as shown here.

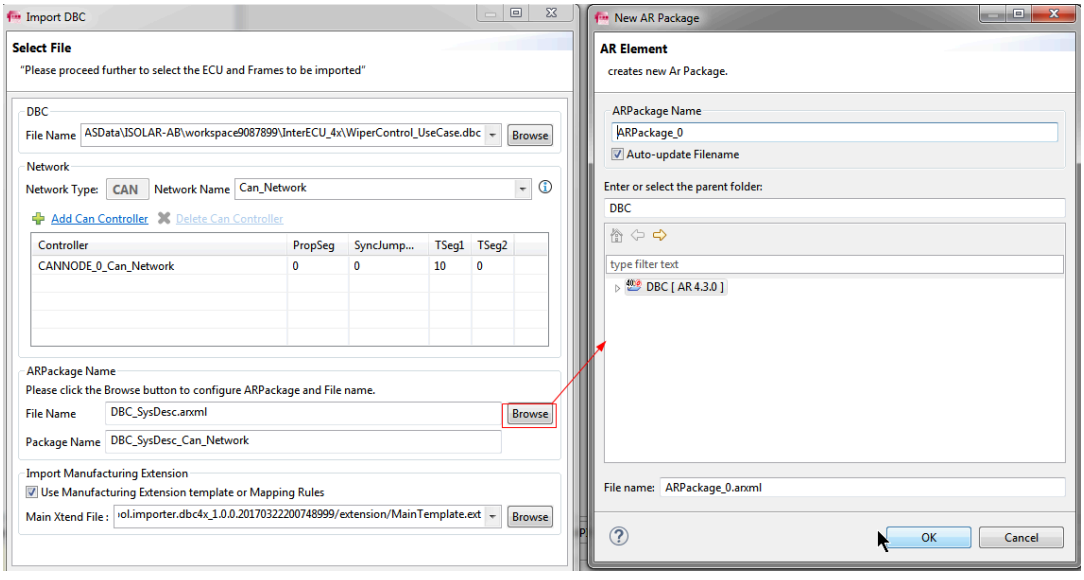

Figure 5.48.DBC Importer - New AR Package

4. Once you've selected the DBC file, the **Next** button will be enabled, and when you click it, the "Select ECUs" page will appear. This is where you select the ECU(s) to be imported from the DBC file.

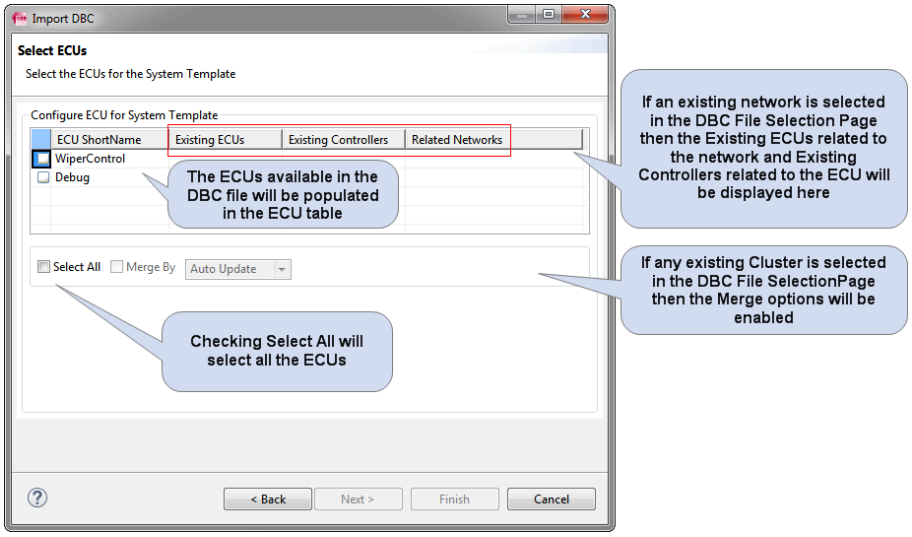

Figure 5.49.DBC Importer - Select ECU

5. When you click **Next**, the "Select Frames" page will appear, and from here you can select the required Frames for the ECU.

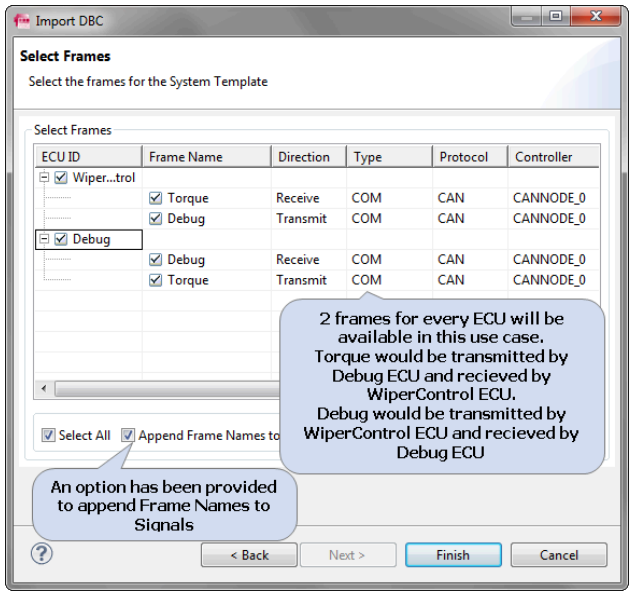

Figure 5.50.DBC Importer - Select Frames

6. When you click **Finish**, the System Description will be generated.

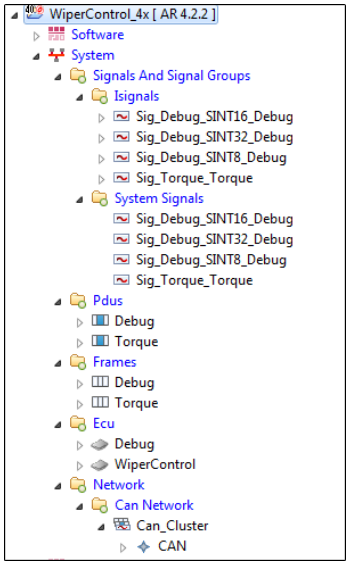

Figure 5.51.DBC Importer - Imported Elements

Simplified Import

The ISOLAR-A DBC Importer also provides an alternative way to import a DBC file, which simplifies the DBC workflow even further. If a DBC file is present inside the project from which the DBC Importer was invoked, then:

- The DBC file will be automatically detected and loaded
- The DBC “Select File” page will be filled with default values
- All the ECUs in the DBC file will be selected
- The **Finish** button will be enabled so that you can complete the import with just one click

If a DBC file is present in a project, and if you want the DBC Import Wizard to take custom values instead of the default values, this can be achieved through a *DBC.properties* file, which needs to be placed inside the project. The values that can be defined through attributes in the *DBC.properties* file are as follows:

- If more than one DBC file is present in the project, the **DBC_File_Name** attribute can be used to indicate the exact DBC file to be loaded.
- The arxml file name to be created can be specified through the **File_Name** attribute.
- The arpackage name to be created can be specified through the **Package_-Name** attribute.
- The CanCluster or J1939Cluster name to be created can be specified through the **CanCluster_Name** or **J1939Cluster_Name** attributes respectively.
- The ECU to be selected during the DBC import can be specified through the **Selected_ECUs** attribute.

An example *DBC.properties* file is available in the **VendorExtensionTemplate** example. If both the properties file and the DBC file are present inside the project, then the DBC importer will give preference to the values defined in the *DBC.properties* file.

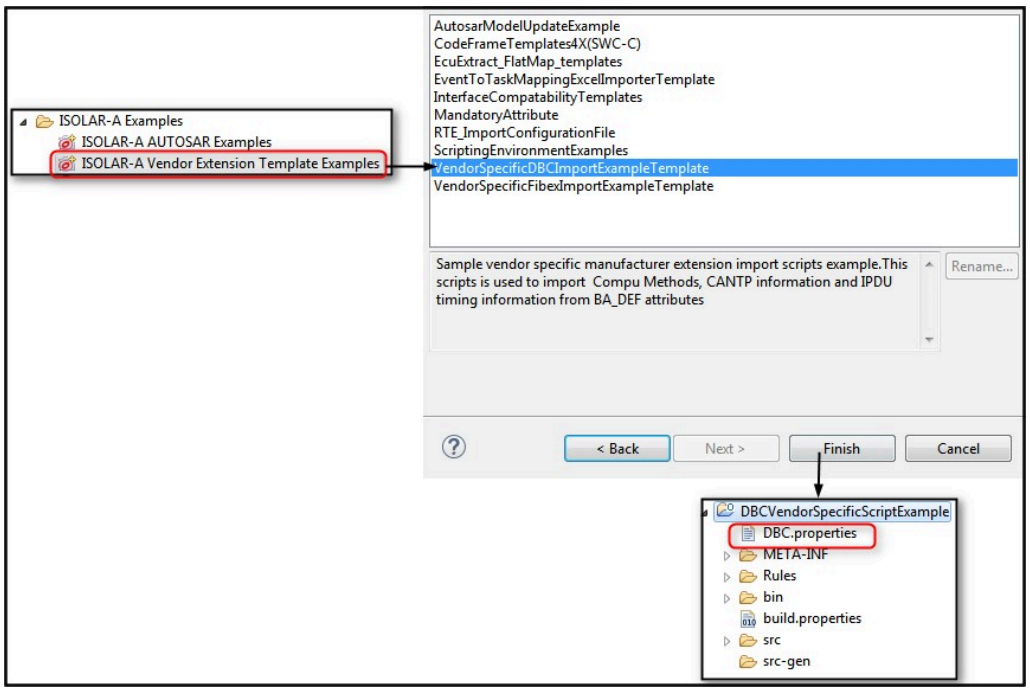

Figure 5.52.DBC Importer - Import Vendor Specific DBC Import example

5.5.3.2 LDF Import Example

The steps below demonstrate how to import an LDF file to create a System Description [Communication Matrix] configuration:

1. Launch the LDF Importer by clicking the “L” Button from tool bar.

Figure 5.53.Invoking the LDF Importer

2. In the “Select File” page of the Import wizard, locate and select the LDF file that you wish to import.

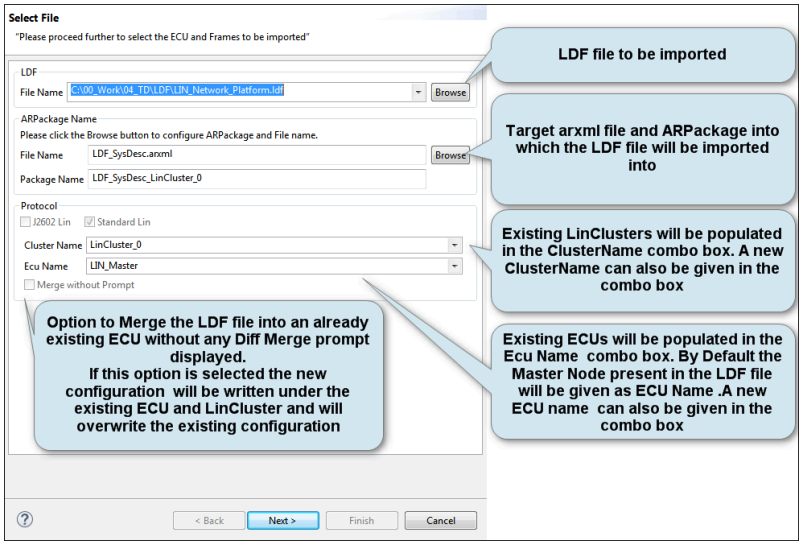

Figure 5.54.LDF Importer - Select File

3. When you click **Next**, the “Select Controllers” page will appear, and this is where you can select the Controllers to be imported from the LDF file.

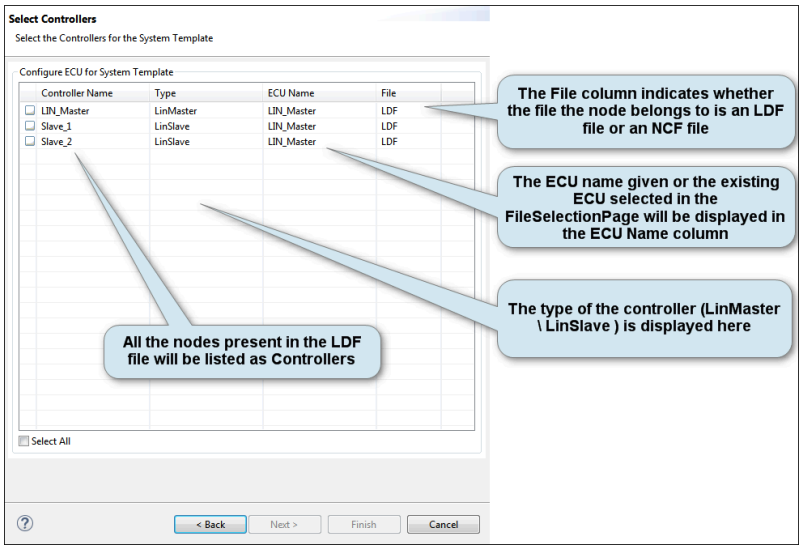

Figure 5.55.LDF Importer - Select Controllers

4. When you click **Next**, the “Select Frames” page will appear, and from here you can select the required Frames for the ECU.

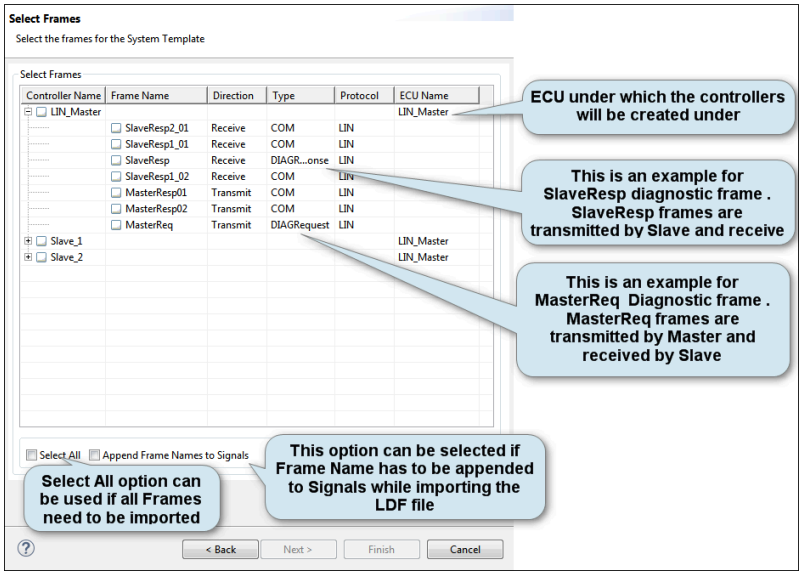

Figure 5.56.LDF Importer - Select Frames

5. When you click **Finish**, the System Description is generated.

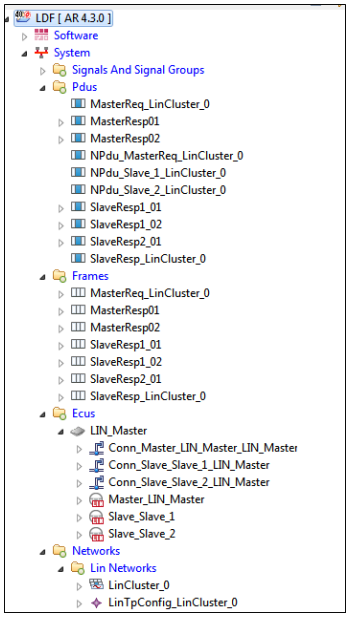

Figure 5.57.LDF Importer - Imported Elements

5.5.3.3 Fibex Import Example

The steps below demonstrate how to import a Fibex file to create a System Description [Communication Matrix] configuration:

1. Launch the Fibex Importer by clicking the “F” Button from tool bar.

Figure 5.58.Invoking the Fibex Importer

2. In the “Select File” page of the Import wizard, locate and select the Fibex file that you wish to import.

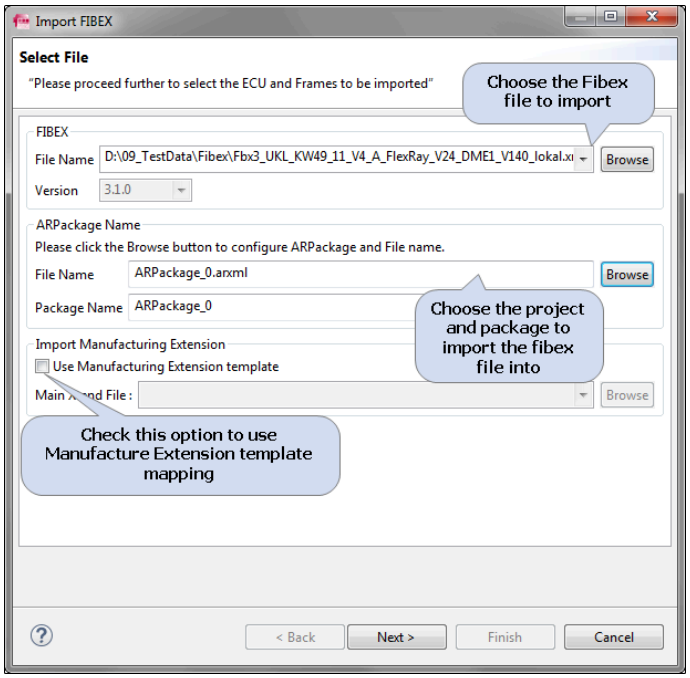

Figure 5.59.Fibex Importer - Select File

3. Once you’ve selected the Fibex file, the **Next** button will be enabled, and when you click it, the “Select ECUs” page will appear. This is where you select the ECU(s) to be imported from the Fibex file.

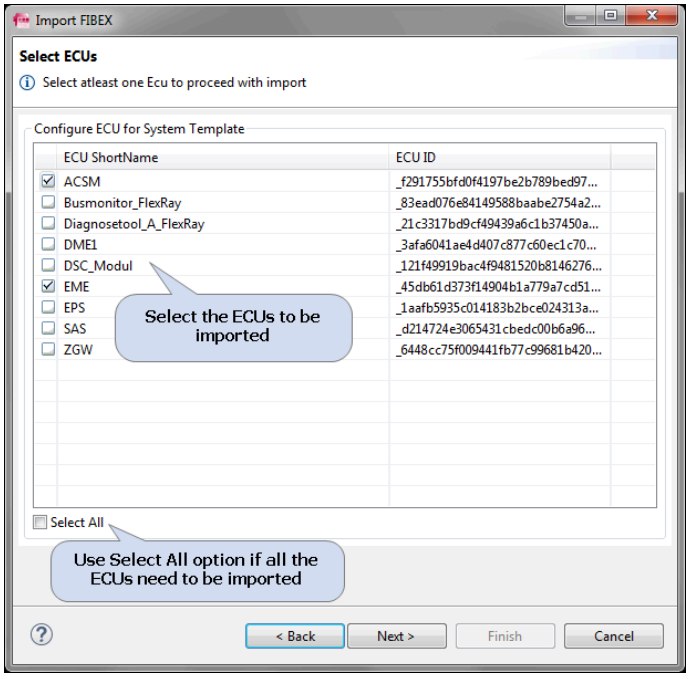

Figure 5.60.Fibex Importer - Select ECUs

4. When you click **Next**, the “Select Frames” page will appear, and from here you can select the required Frames for the ECU.

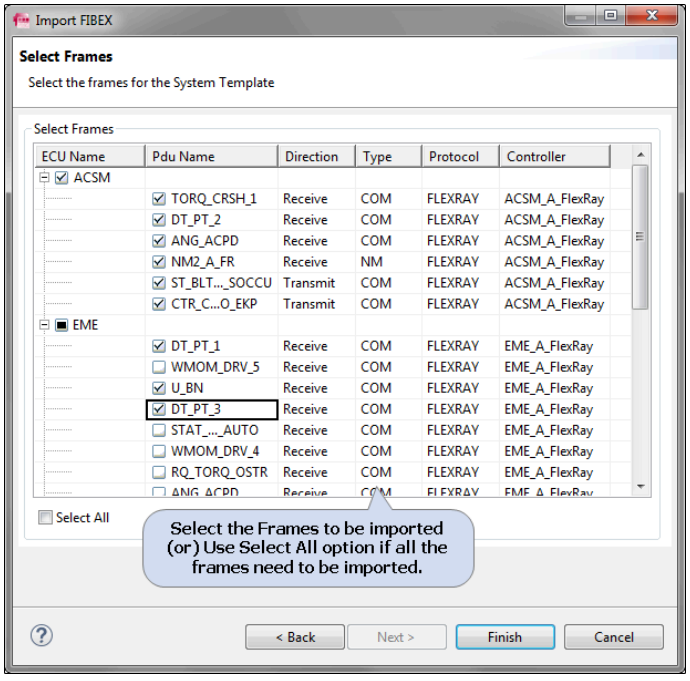

Figure 5.61.Fibex Importer - Select Frames

5. When you click **Finish**, the System Description is generated.

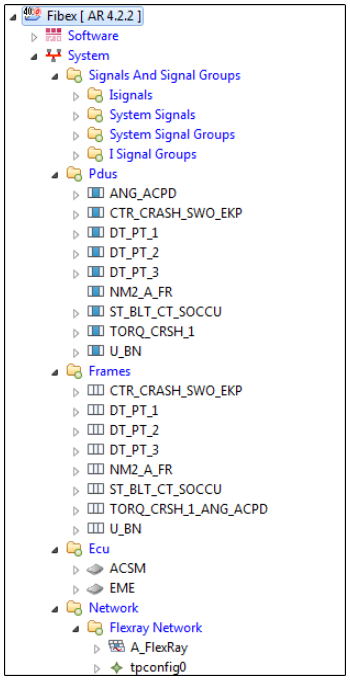

Figure 5.62.Fibex Importer - Imported Elements

5.5.4 ARXML Importer

The ARXML Importer can be invoked by either by clicking on the “A” icon in the toolbar or, as shown below, via **File** > **Import** > **ISOLAR-A** > **ARXML Import**.

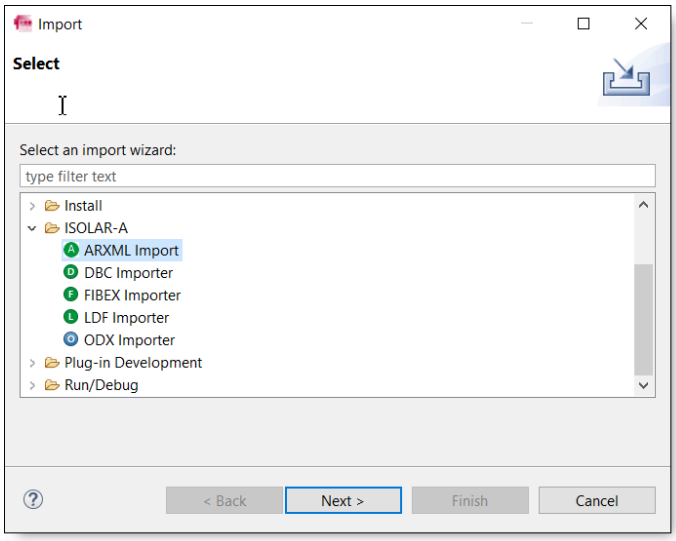

Figure 5.63.Invoking the ARXML Importer

Whichever method you choose to invoke the ARXML Importer, you'll be taken to the "ARXML Importer" UI.

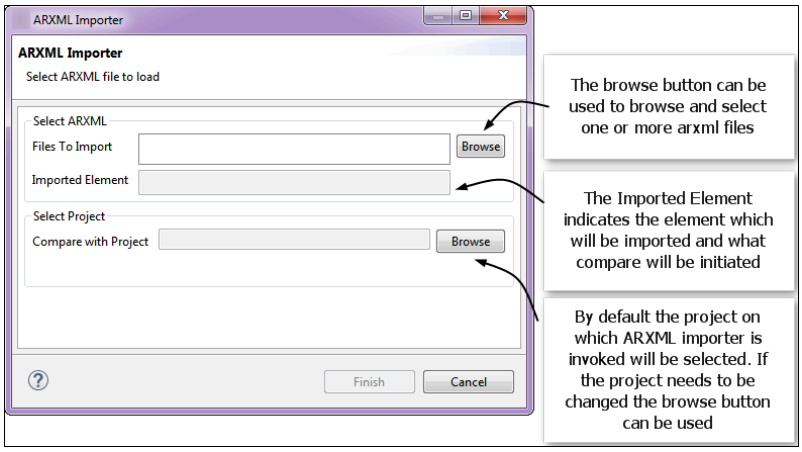

Figure 5.64.The ARXML Importer UI

5.5.4.1 Use Cases

The ARXML Importer supports the following use cases, which are described in more detail in the sections below:

Single File Import

Any ARXML file can be imported into a target project using the ARXML Importer. A single file import may involve any of the following compare and import scenarios:

- File Compare and Import
- Component Compare and Import
- Composition Compare and Import
- Diagnostic Extract Compare and Import

Multiple File Import

The ARXML Importer also supports multiple ARXML file imports, which may involve either of the following two scenarios:

- EcuExtract Import
- Files Import

Use Case 1: File Compare and Import

If the project already contains an ARXML file with the same name as the file being imported, the "ARXML Importer" dialog would look as below. When you click **Finish**, the file comparison will be initiated.

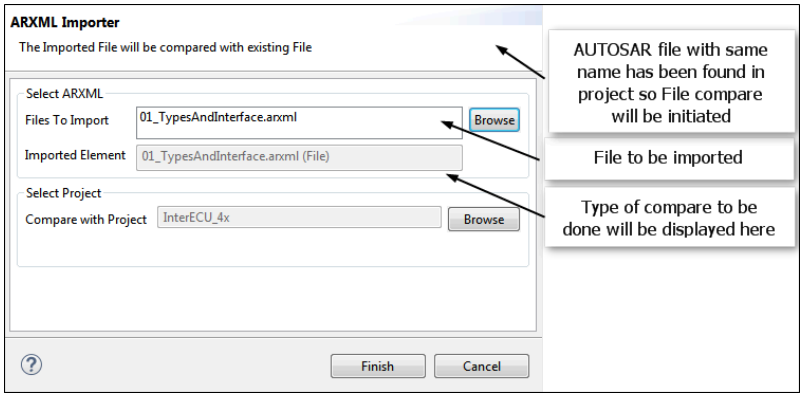

Figure 5.65.ARXML Importer - Use Case 1: File Compare and Import

If the project does not already contain an ARXML file with the same name as the one being imported, the “ARXML Importer” dialog will look like this.

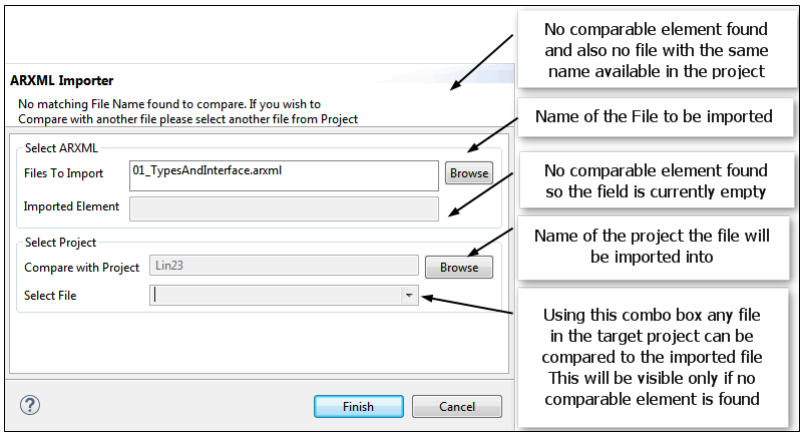

Figure 5.66.ARXML Importer - File Compare and Import (Options)

Use Case 2: Component Compare and Import

If the ARXML file being imported contains a software component with the same name as an existing software component in the project, a Component Level comparison will be invoked when you click **Finish**.

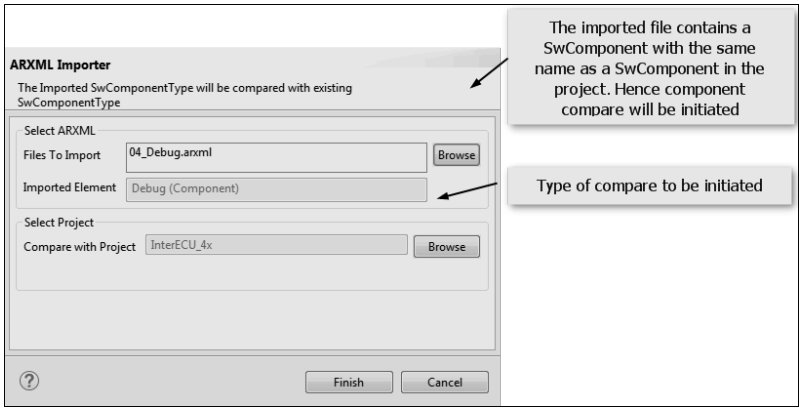

Figure 5.67.ARXML Importer - Use Case 2: Component Compare and Import

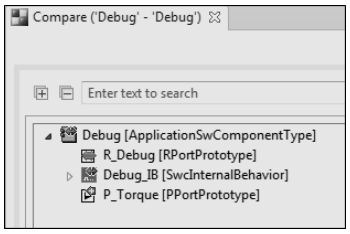

Figure 5.68.ARXML Importer - Use Case 2: Component Compare

Use Case 3: Composition Compare and Import

If the ARXML file to be imported contains a software composition with the same name as an existing software composition in the project, a Composition Level comparison will be invoked.

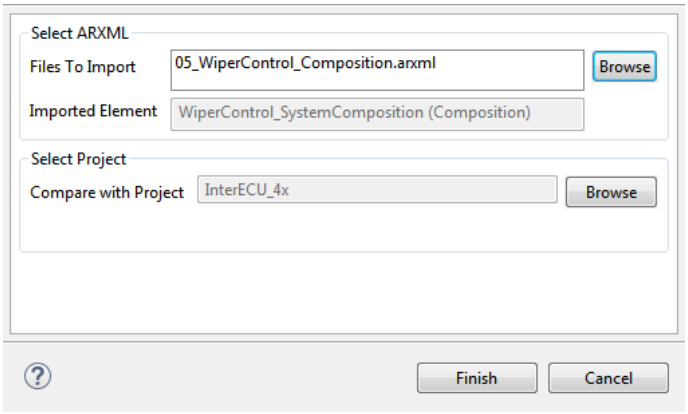

Figure 5.69.ARXML Importer - Use Case 3: Composition Compare and Import

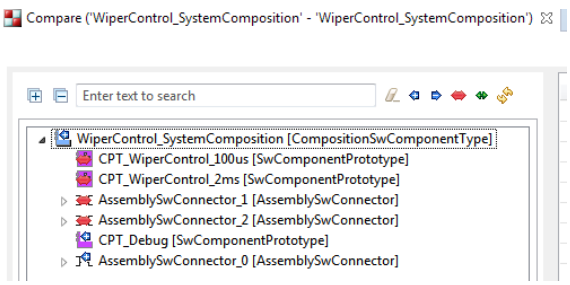

Figure 5.70.ARXML Importer - Use Case 3: Composition Compare

Use Case 4: Diagnostic Extract Compare and Import

If the ARXML file to be imported contains a diagnostic extract with the same name as an existing diagnostic extract in the project, a Diagnostic Extract Comparison will be invoked.

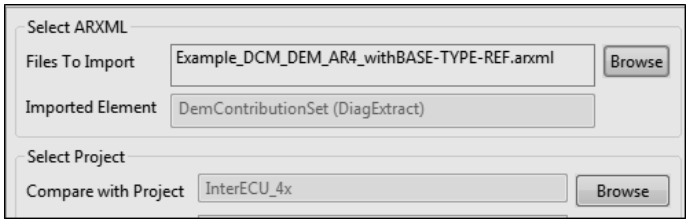

Figure 5.71.ARXML Importer - Use Case 4: Diagnostic Compare and Import

Use Case 5: EcuExtract Compare and Import

If multiple ARXML files are being imported, and if they contain an ECU extract with the same name as an existing ECU extract in the project, the ECU Extract Comparison will be invoked.

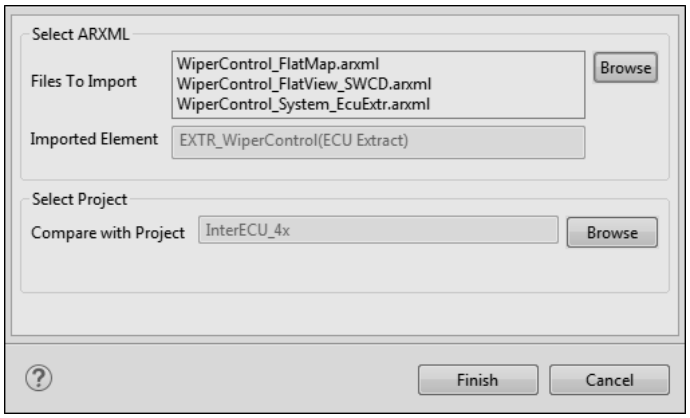

Figure 5.72.ARXML Importer - Use Case 5: EcuExtract Compare and Import

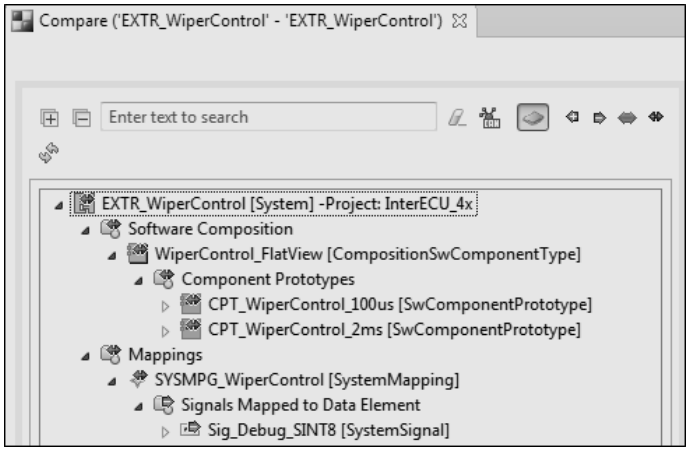

Figure 5.73.ARXML Importer - Use Case 5: EcuExtract Compare

Use Case 6: Files Import

If *multiple* ARXML files are being imported, but they do not contain an ECU extract, all the files will be imported into the project as it is.

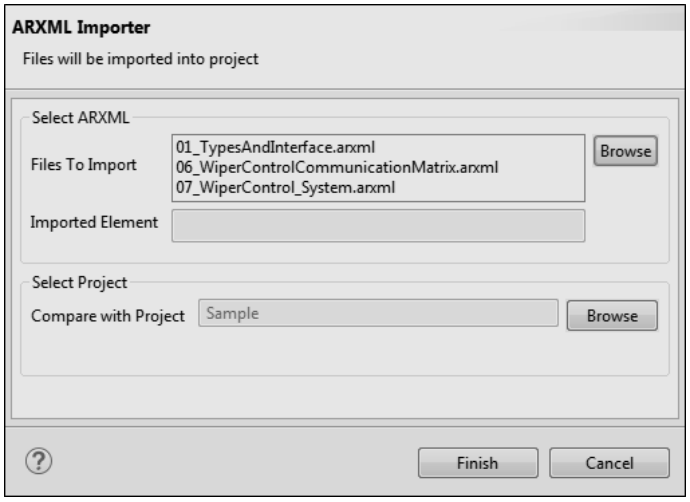

Figure 5.74.ARXML Importer - Use Case 6: Files Import

5.5.5 DiffMerge

There might be cases where you'll need to update a configuration based on new updates received from the OEMs. In these cases, the DiffMerge feature allows you to update the existing configuration without needing to delete your existing configured project.

5.5.5.1 Merge/update to previously imported Communication Matrix

In this section we look at several use cases supported by ISOLAR-A.

Use Case 1: Same message transmitted from one ECU into two different CAN networks

Expected Result: The message must not be duplicated because the same message is transmitting to the different networks.

Step 1. Initial import (Import the DBC file for the first time)

- Click the “D” icon in the tool bar for the DBC Importer.

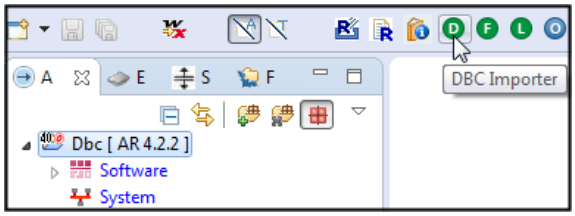

Figure 5.75.DBC Importer invocation

- In the “File Name” box, select the DBC file that you wish to import.
- The “Network” section displays the Network type, new/existing Clusters and Controllers
- The “ARPackage Name” section shows the File name and Package Name

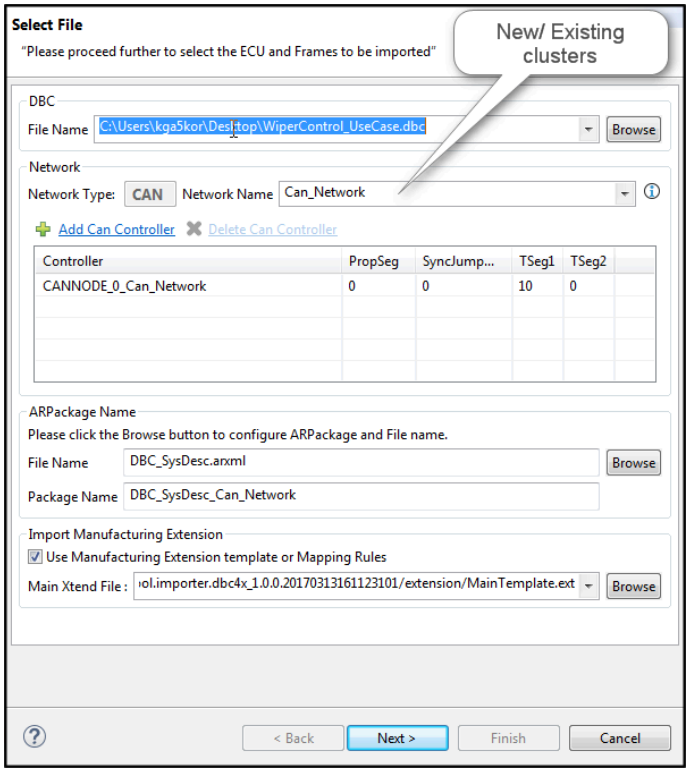

Figure 5.76.Import the DBC file

Step 2. In the “Select ECUs” page, select the ECUs to be configured in the project. (Note: The “Merge By” option will be disabled because this is a first time import into the project).

Step 3. In the “Select Frames” page, select the frame to be transmitted in the current network, then click **Finish**. The result will be one frame (“Torque”) transmitted from 'Can_Network'

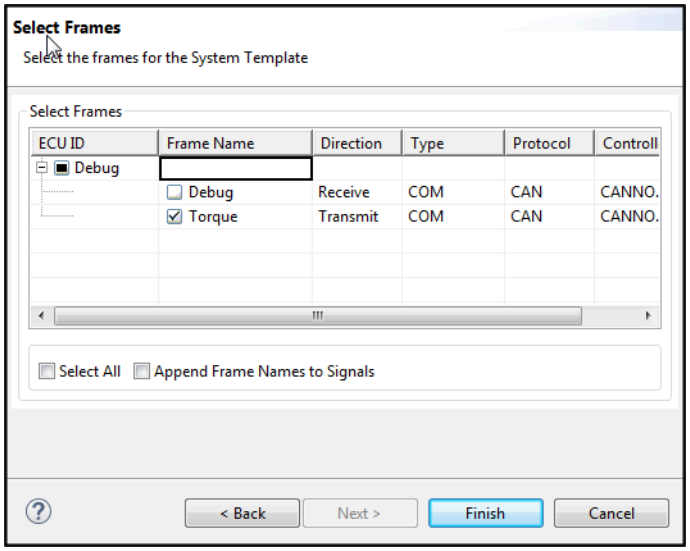

Figure 5.77.DBC Importer - Select Frames

Step 4. Now import the **updated DBC**, which has the same "Debug" ECU transmitting the same "Torque" Frame, but this time from a different network.

As shown below in the "Network Name" field, when you import the updated DBC file, the new cluster ("Can_Network_0") will be created by default, and the drop-down will show the existing networks available in the project. (Note: Merging the same ECU in a different network will not complete the **Gateway Configuration**).

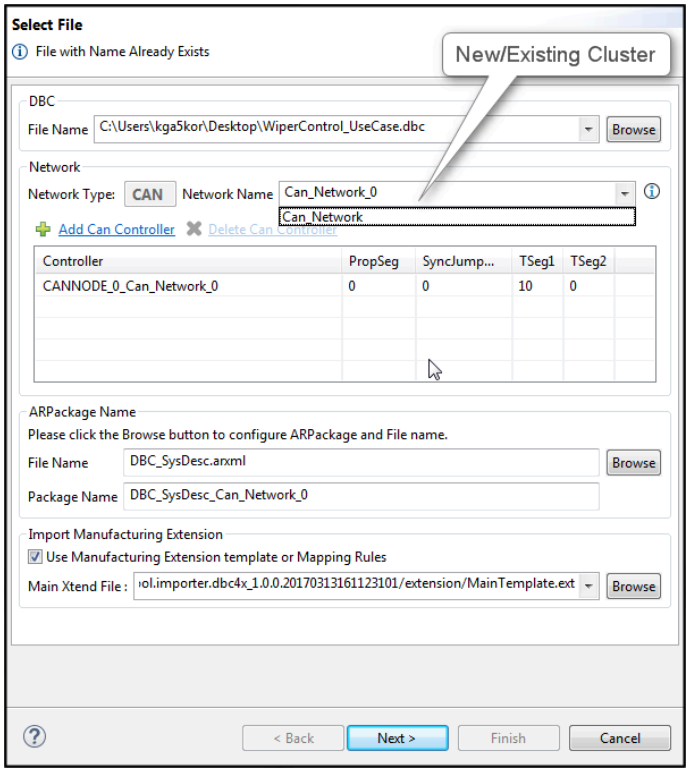

Figure 5.78.DBC Importer - Import updated DBC

Step 5. In the “Select ECUs” page, the existing ECU is automatically selected and mapped to the updated ECU. Select “Auto Update”, which will update only the newly added elements to the existing configuration, then click **Finish**.

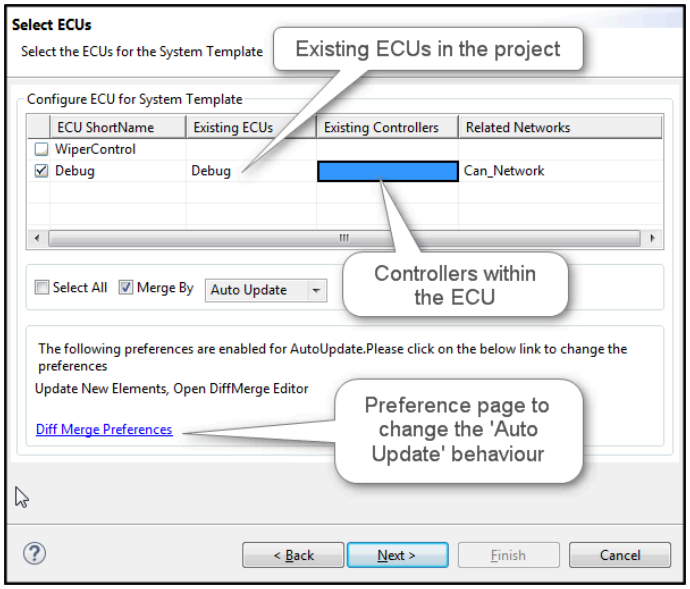

Figure 5.79.DBC Importer - Select ECUs

Step 6. The behavior of the Auto Update can be altered in the DBC Importer Preferences.

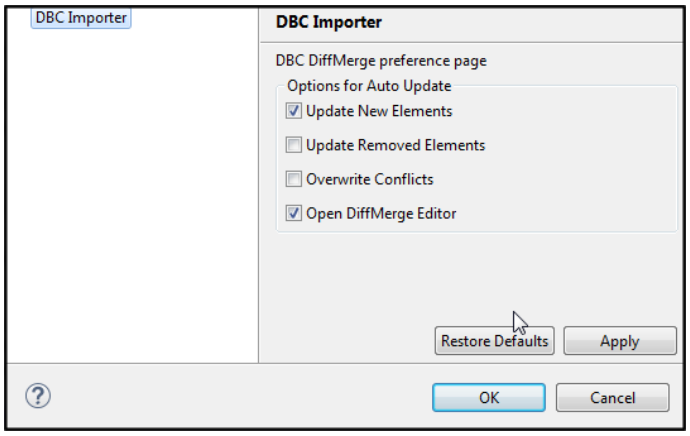

Figure 5.80.DBC Importer - Preferences

Step 7: Select the frames that need to be added to the ECU and, after you click **Finish**, the Torque Frame will not be copied twice, but is instead referred to by two triggerings in the different networks.

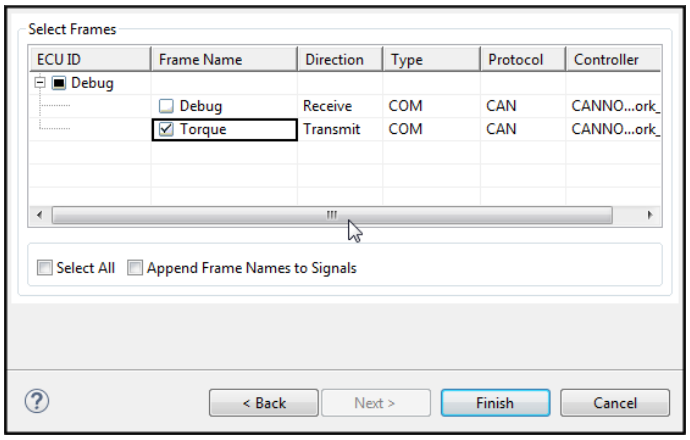

Figure 5.81.DBC Importer - Select Frames

Use Case 2: Same message transmitted from one ECU, received by another ECU in the same CAN network

Step 1: Follow Steps 1 to 3 of Use Case 1.

Step 2: Import the updated DBC file, which has the same “Torque” message received by the new ECU, but in the same network. Select the existing network from the “Network Name” drop-down.

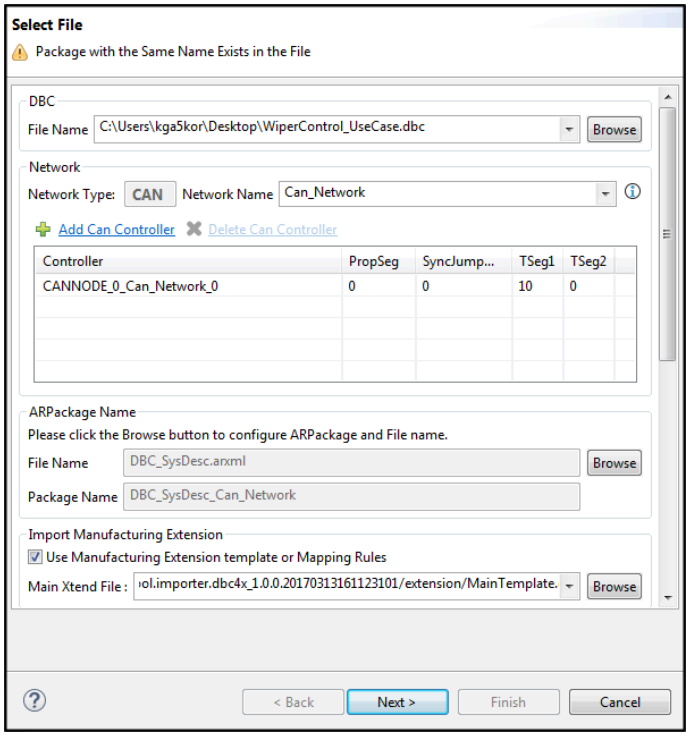

Figure 5.82.DBC Importer - Select File

Step 3: In the “Select ECUs” page, select the “WiperControl” ECU (and deselect the automatically selected “Debug” ECU).

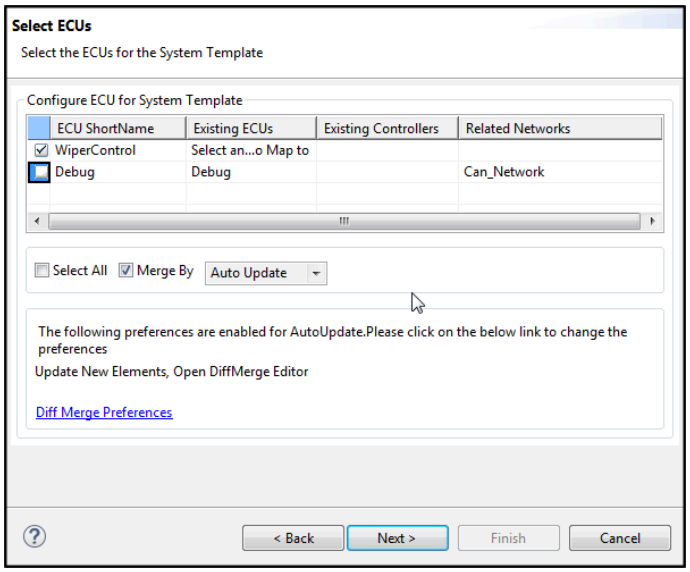

Figure 5.83.DBC Importer - Select ECUs

Step 4: In the “Select Frames” page, select the same “Torque” message to be received by the “WiperControl” ECU, and then click **Finish**. The result will be the

same message transmitted from the “Debug” ECU and received by the “Wiper-control” ECU, in the same network.

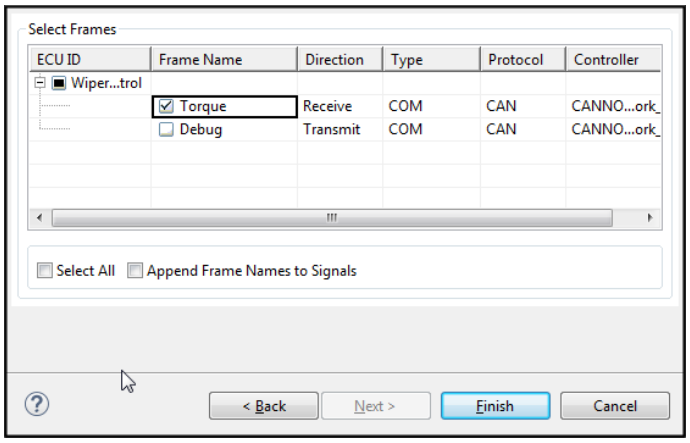

Figure 5.84.DBC Importer - Select Frames

Use Case 3: Same message transmitted from one ECU and received by another ECU in a different CAN network

Repeat the steps in Use Case 2 by selecting the new network (step 2) as shown below. After you’ve clicked **Finish** in the “Select Frame” page, the same “Torque” message is transmitted from the “Debug” ECU and received by the “WiperControl” ECU in a different network.

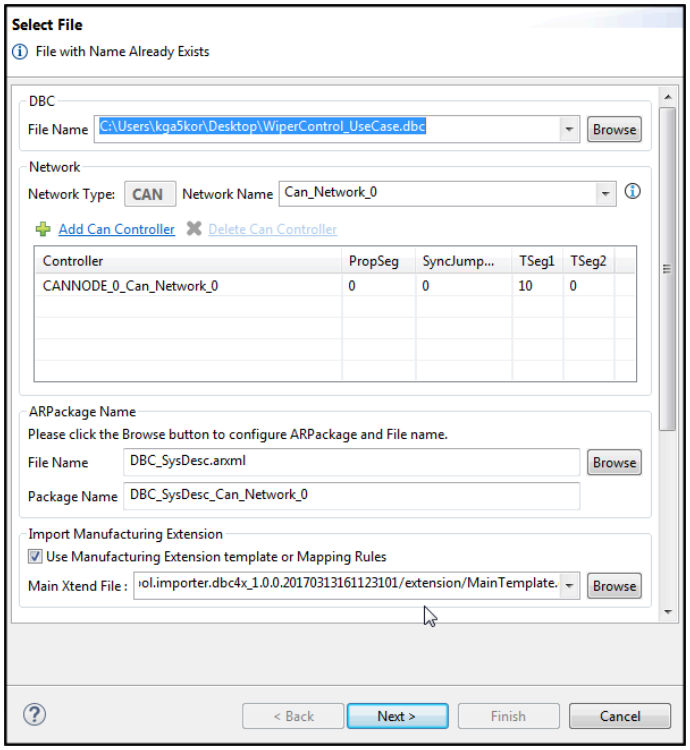

Figure 5.85.DBC Importer - Select File

Use Case 4: Importing an updated DBC file to merge the contents into the same existing ECU

Note: In addition to this use case, it is also possible to merge the updated ECU to the existing ECU with a different short name.

Step 1: In this example, the configuration already has a “Debug_1” ECU in the project.

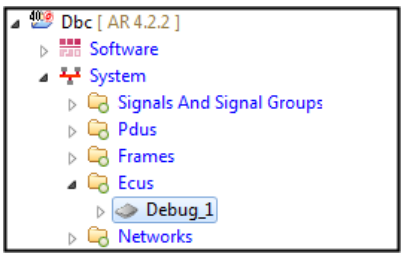

Figure 5.86.Already configured ECU

Step 2: Now we have the updated DBC file with “Debug” as the ECU name, and with the additional message. Import this updated DBC file and select the existing cluster.

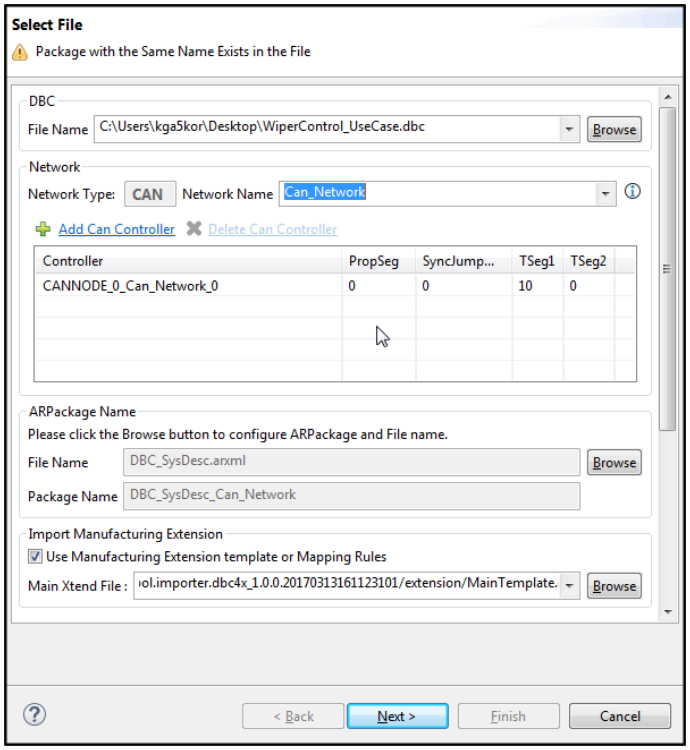

Figure 5.87.DBC Importer - Select File

Step 3: Do the following in the “Select ECUs” page:

- Select the “Debug” ECU
- In the “Existing ECUs” column, select the existing ECU (“Debug1”) to merge into
- Select the existing controller in the “Existing Controllers” column
- Select the “Merge by” type from the drop down, options available are:

Auto Update	Merge only the newly added elements
Overwrite	Replace the existing configuration with updated configuration
Manual Merge	Opens the diffmerge editor to manually merge the contents

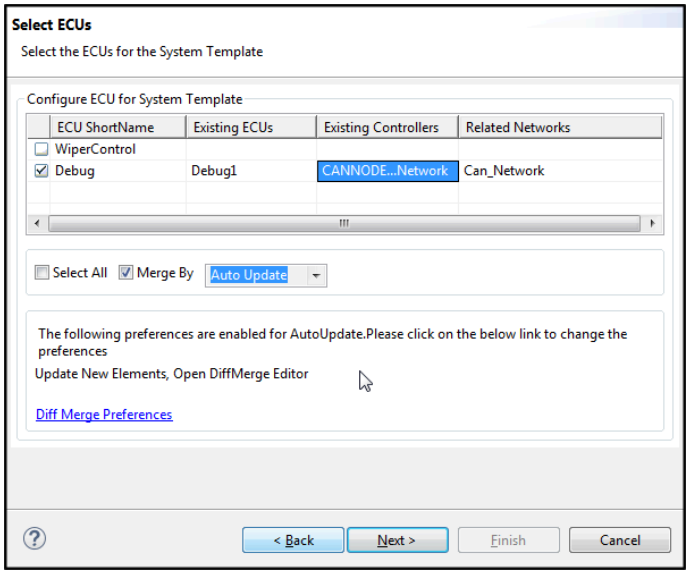

Figure 5.88.DBC Importer - Select ECUs

Step 4: Select the Frames and click **Finish**. The additional contents will be merged into the existing "Debug_1" ECU.

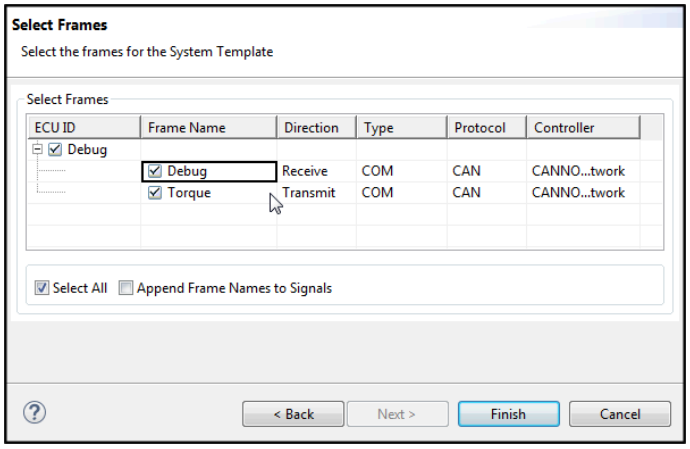

Figure 5.89.DBC Importer - Select Frames

Merge Actions

The merge actions will be enabled based on the element's difference.

- If the element is new, only the **Update** action will be enabled.
- If the element is removed, only the **Delete** action will be enabled.
- If only children of reference changes, only the **Overwrite** action will be enabled.

Update: Update will be enabled only when the parent element already exists in the project and the selected element is new.

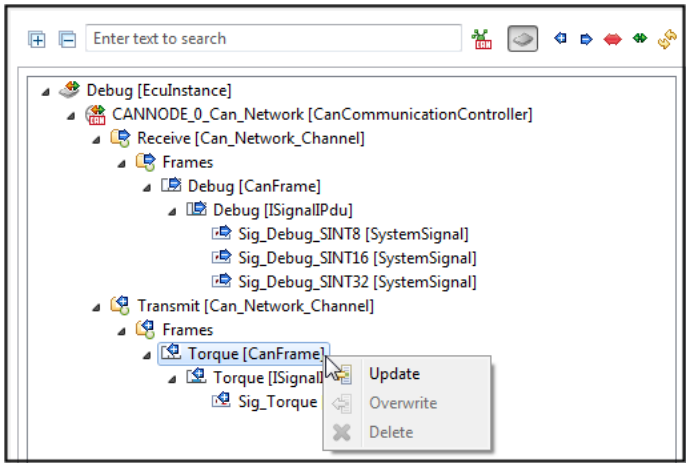

Figure 5.90.Update action

On each merge operation, the following prompt will be shown.

Figure 5.91.Merge prompt

When you close the DiffMerge editor, the following dialog will prompt for the merged elements report summary.

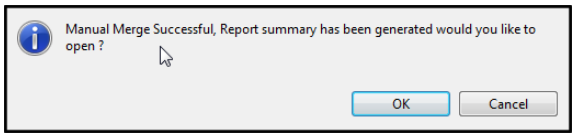

Figure 5.92.Merge Report Summary prompt

Delete: The Delete action will be enabled only if the element is removed in the updated DBC file.

- System elements such as Frames, PDUs and ISignals will be deleted only if they are not referred to in any other element.
- If the above elements are referred in other elements, then only ports from the triggering of the respective ECU will be deleted.

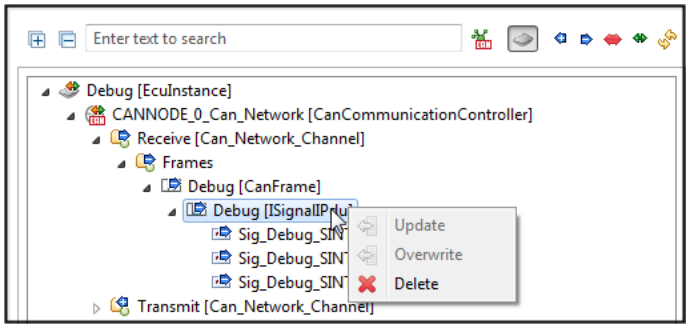

Figure 5.93.Delete action

Overwrite: This completely replaces the existing configuration from the updated file.

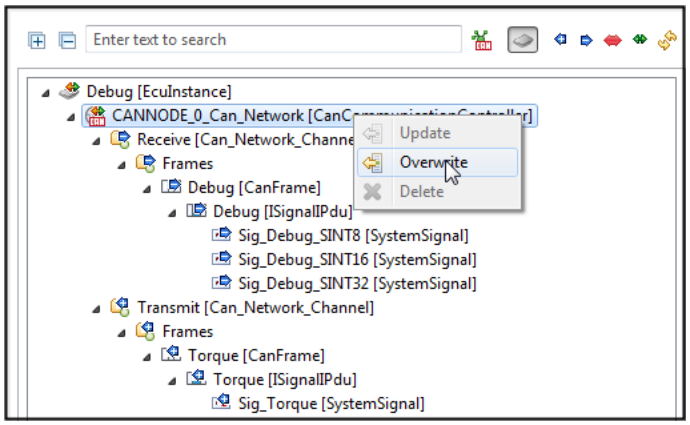

Figure 5.94.Overwrite action

5.5.5.2 DiffMerge / Three Point Merge for ECUExtract

ISOLAR-A's Compare/Merge Editor allows you to compare two similar AUTOSAR elements. The comparison is done based on Short Name.

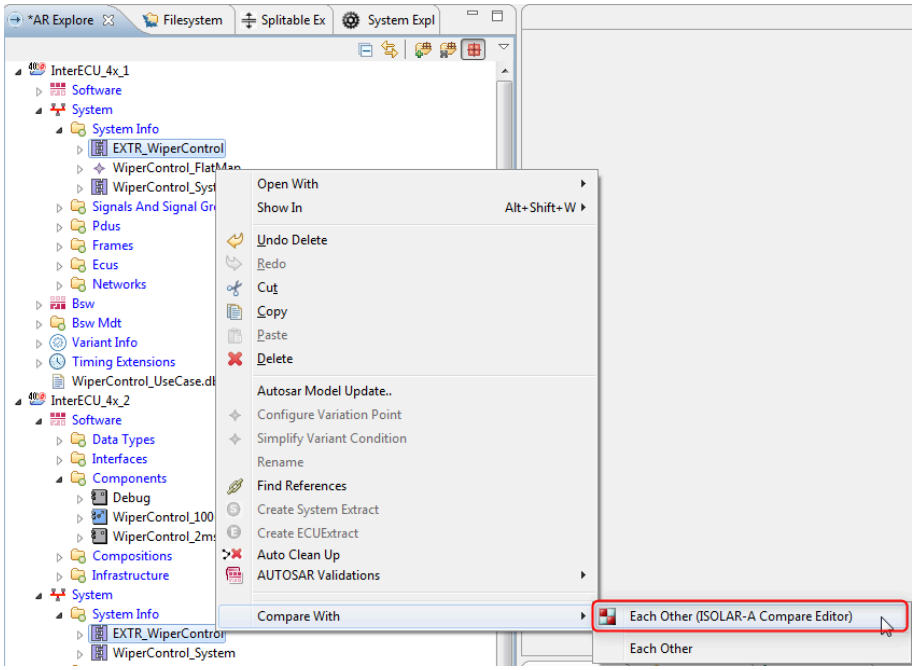

Figure 5.95.Compare/Merge on two ECUExtract elements

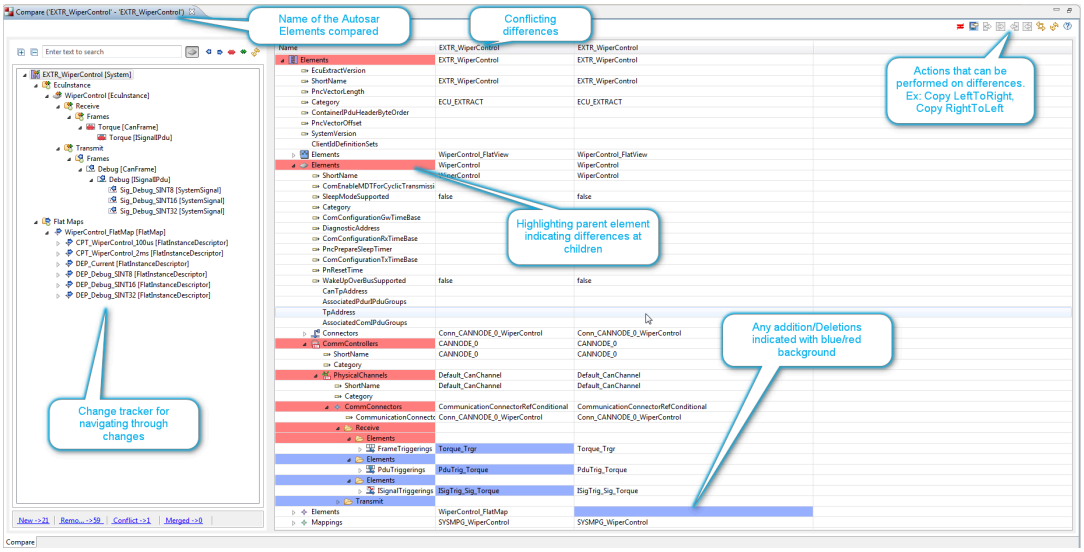

Figure 5.96.Compare/Merge results

5.5.5.3 DiffMerge Report Generation

A summary report will be generated for all the Diff Merge comparisons supported by ISOLAR-A. When you close the Diff Merge Editor, you'll be prompted with the following dialog.

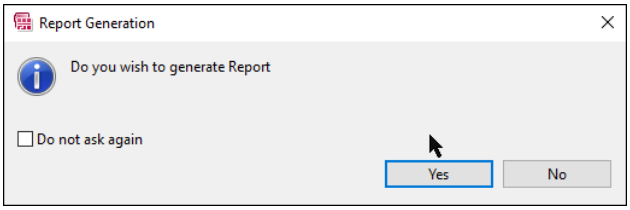

Figure 5.97.Invoke Report Generation

When you click Yes, the report will be generated and you'll then be given the opportunity to open the report.

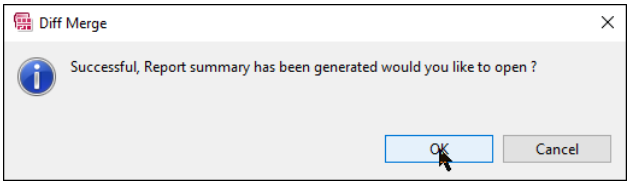

Figure 5.98.Open Generated Report

5.6 Configure System Mapping

5.6.1 Introduction

In this section we look at mapping the System onto ECU Instances and Network Clusters.

5.6.2 Configure a System Mapping

Once the System Description has been configured, the next step is to configure a System Mapping. To do so, first create a System Element with "SYSTEM_EXTRACT" as the category.

1. Create a System Element using the context menu, as shown here.

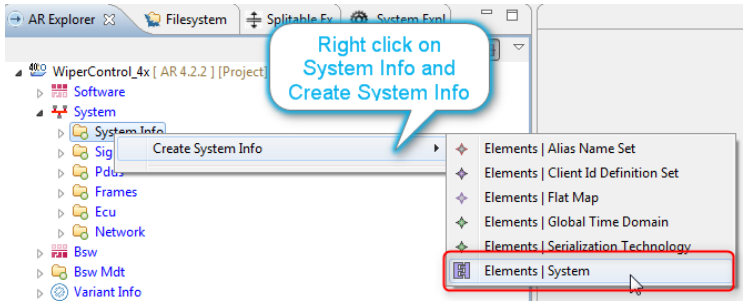

Figure 5.99.Create a System element

2. In the "System" field, change the name of the system to

WiperControl_System, tick the “Create New AR Packages” option, set the “AR Package Path” to *WiperControl_System* and select *07_WiperControl_System.arxml* as the file name.

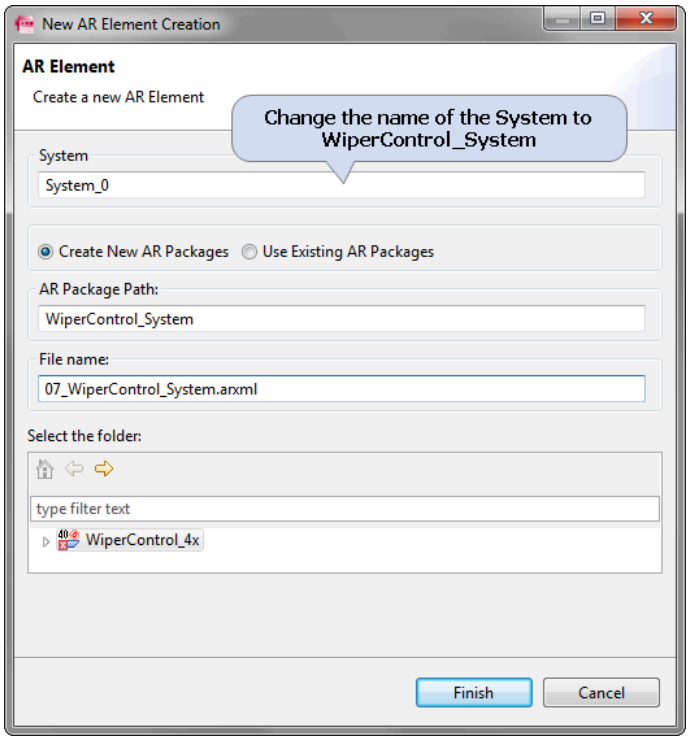

Figure 5.100.Create new AR Package

3. Select the *WiperControl_System* from the AR Explorer and, in the properties view, configure the category as “SYSTEM_EXTRACT” (make sure you use upper case).

5.6.3 Mapping SWCs to ECU Instances

The next important step is to map the Components to ECU Instances.

1. Open the “SwcToEcuMapping” editor on the newly created “WiperControl_System

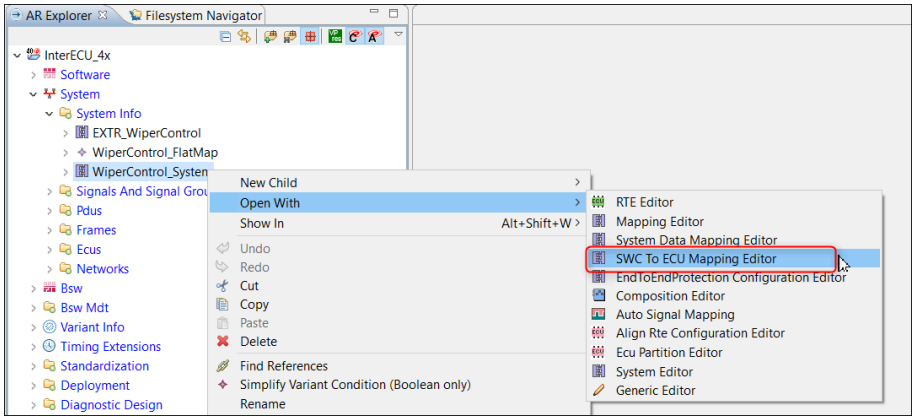

Figure 5.101.Invoking the SwcToEcuMapping Editor

2. Configure the **Top Level Composition** to the "WiperControl_Composition" (that we created in Section 5.4.4 during the Software design) so that the Components are visible in the Editor.

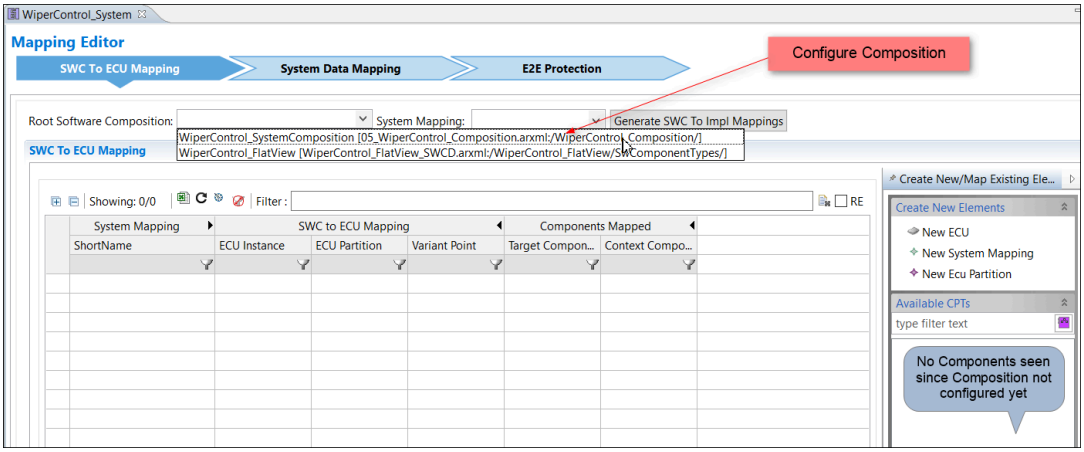

Figure 5.102.SwcToEcuMapping Editor - Configure Top Level Composition

3. Create the SystemMapping below the "WiperControl_System".

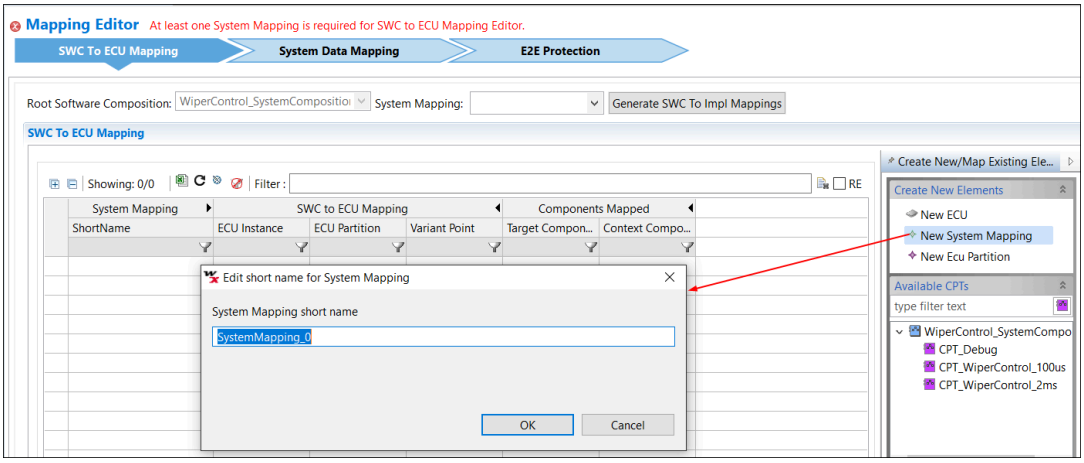

Figure 5.103.SwcToEcuMapping Editor - Create SystemMapping

4. Once the Composition is configured, the available Component Prototypes and ECU Instances are shown, from which the Components can be configured (“mapped”) to the required ECU.

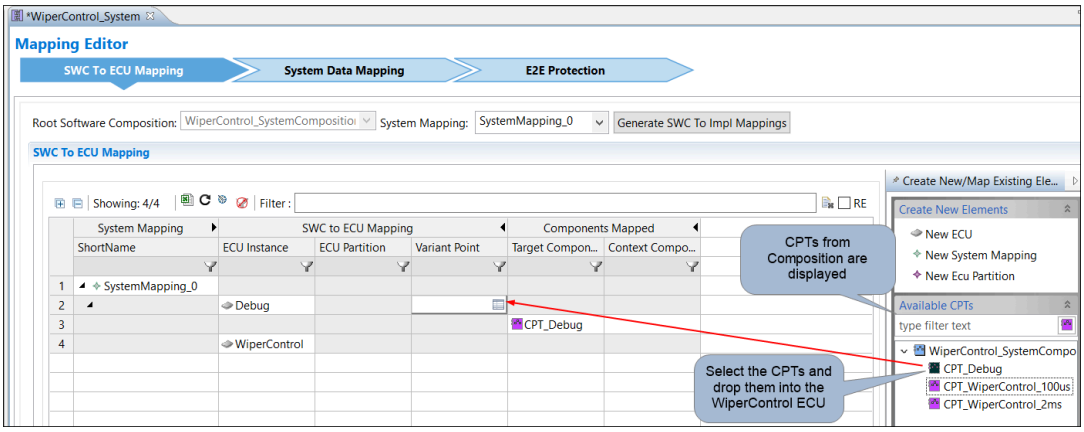

Figure 5.104.SwcToEcuMapping Editor - Map Components to ECUs

5. The “CPT_WiperControl_2ms” and “CPT_WiperControl_100us” components are now mapped to the “WiperControl” ECU Instance.

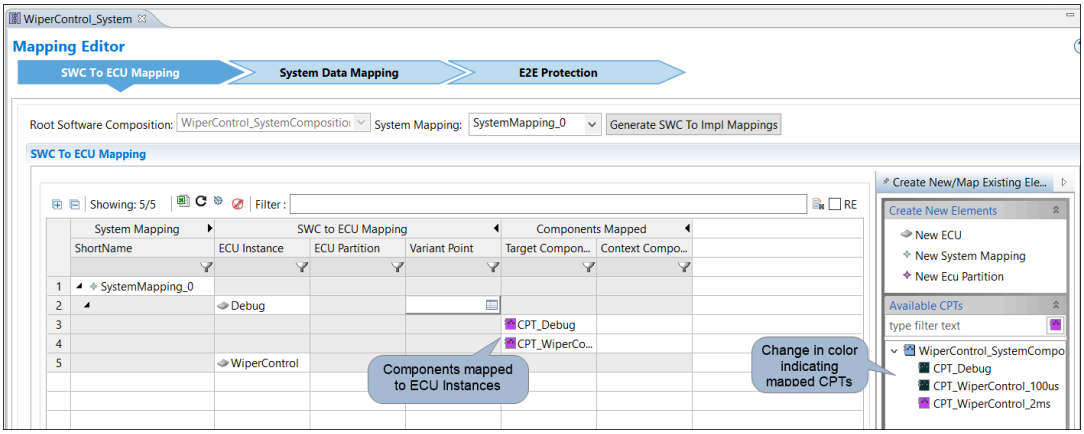

Figure 5.105.SwcToEcuMapping Editor - Component mapping completed

Note: Creating the SwcToEcuMapping can also be achieved through the command line mode. For more information on command line mode, see the ISOLAR-A Help Reference Manual.

5.6.4 Mapping System Signals to ECU-internal DataElements

After the Components are mapped to the ECU, the next step is to map the System Signals to the DataElements (of ports of components). Here we are mapping system-level artifacts to the Virtual Function Bus (VFB) level artifacts, which allows communication between ECUs via the ComStack.

The following steps are required to map the Data Element to the System Signals.

1. Open the “System Data Mapping Editor”.

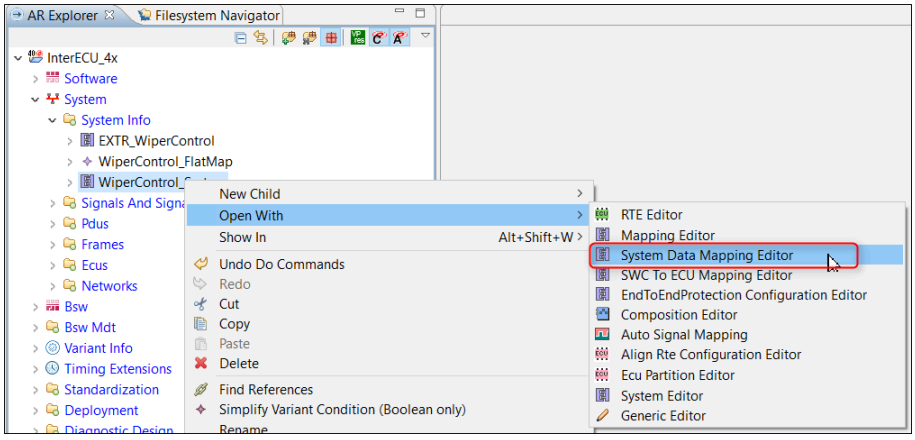

Figure 5.106.Invoking the System Data Mapping Editor

2. This will open the editor with all the Components and Data Elements under the components for mapping to System Signal.

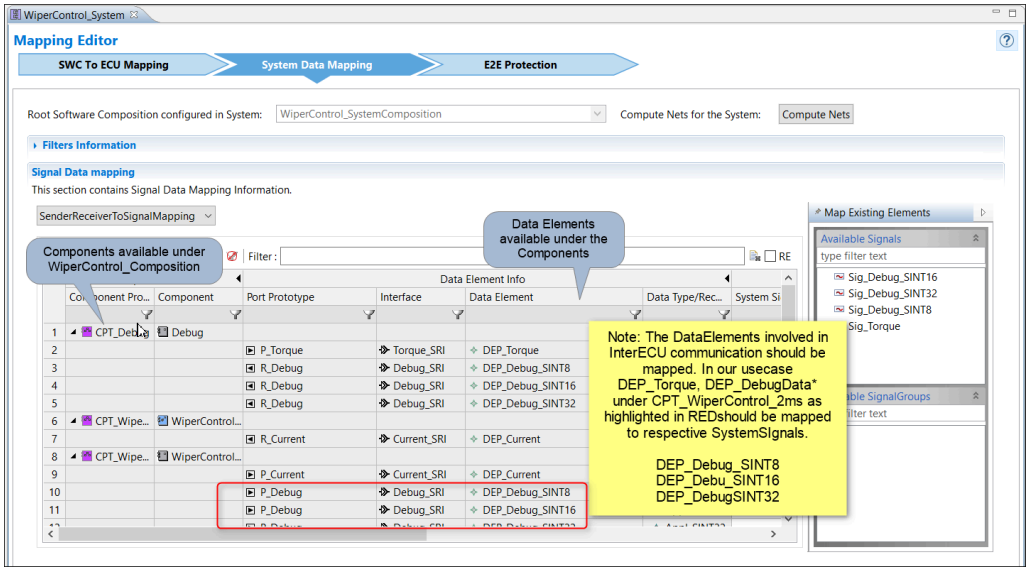

Figure 5.107.The System Data Mapping Editor

3. Double Click on the highlighted cell for mapping DataElement to the Signal, and then select the Signal.

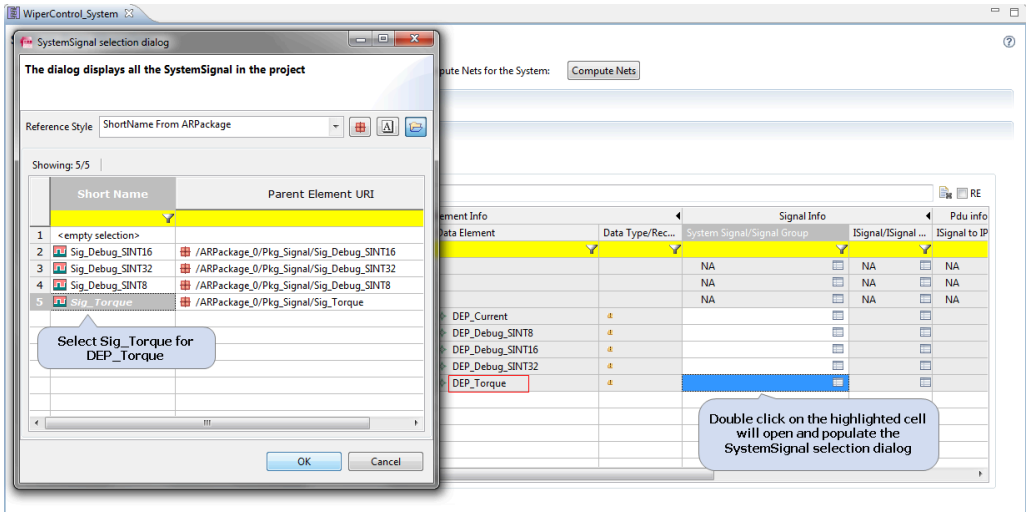

Figure 5.108.System Data Mapping Editor - System Signal dialog

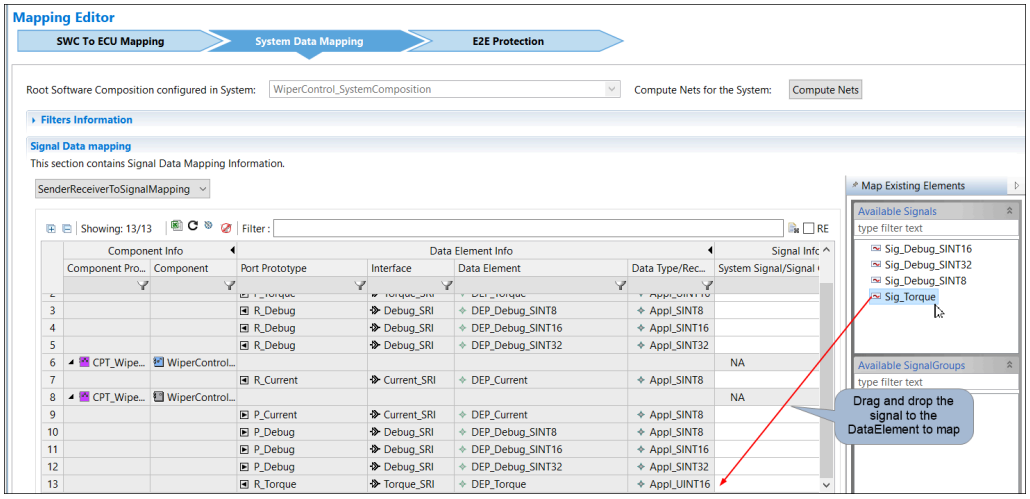

Figure 5.109.System Data Mapping Editor - Drag and drop to map

4. Similarly, configure other Data Element Prototypes (“DEP”) of the Component Prototype “WiperControl_2ms” to complete the mapping.

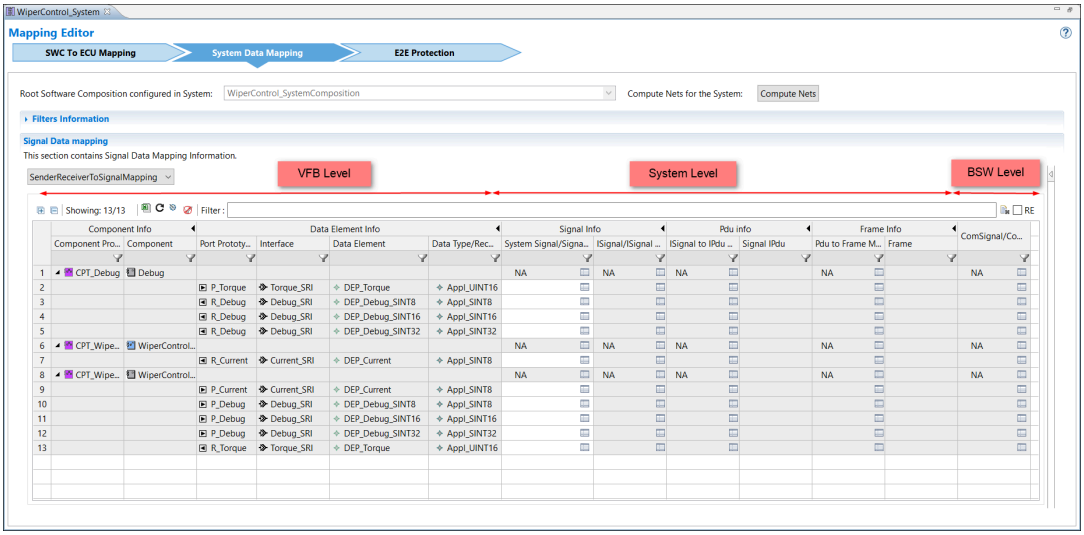

Figure 5.110.System Data Mapping Editor - Frame configuration completed

Note: No signal mapping is necessary for DEP_current since this is used only in intra ECU communication.

5.6.5 Creating a System Extract

After mapping SWCs to ECU Instances, and mapping System Signals, the next step (which is optional) is to generate a System Extract. This will extract, from the configured system, information related to one or more ECUs from the SYSTEM_DESCRIPTION or SYSTEM_EXTRACT.

1. Right-click on the **WiperControl_System** [SYSTEM_EXTRACT] and select the “Create System Extract” option.

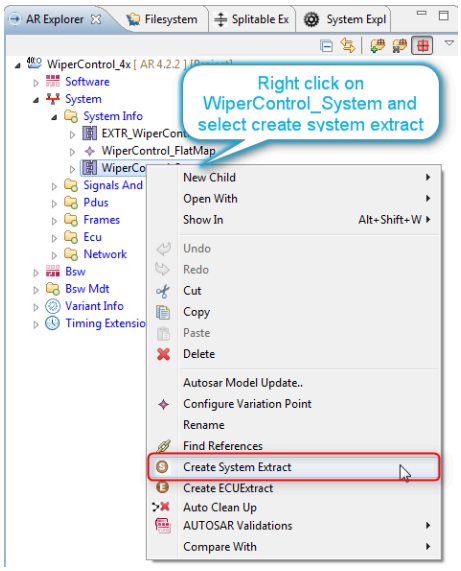

Figure 5.111.Invoking System Extract

2. Select the **WiperControl ECU** Instance, and enter “ExtractProject” in the Project Name field. When you click **Finish** a new project will be created containing all the AUTOSAR elements related to WiperControl ECU.

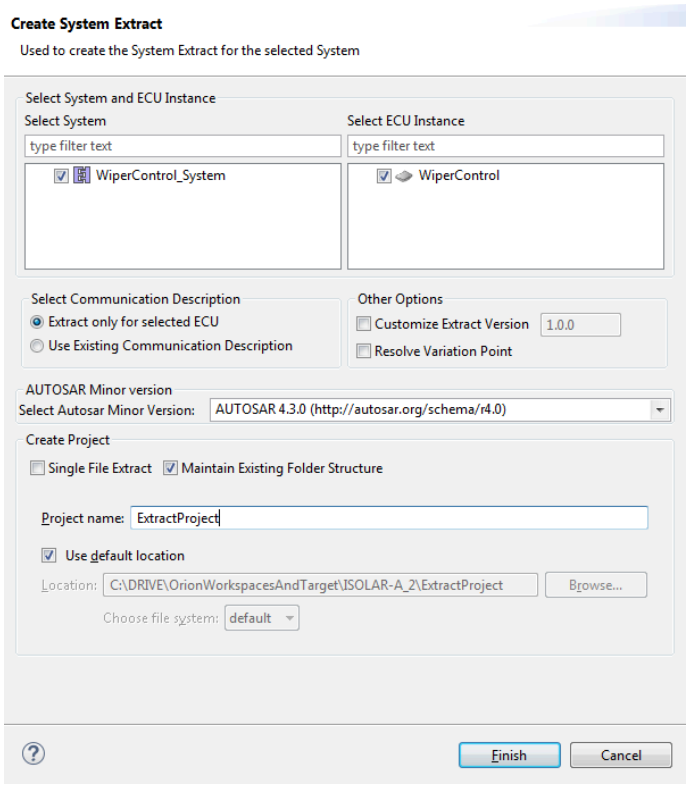

Figure 5.112.Create System Extract

5.6.6 Creating an EcuExtract

With the Software and System Design phases now complete, we can generate an ECU_EXTRACT, which will extract the ECU information from the SYSTEM_EXTRACT and also create the Flat view.

1. Right-click on the **WiperControl** ECU and select the “Create ECUExtract” option.

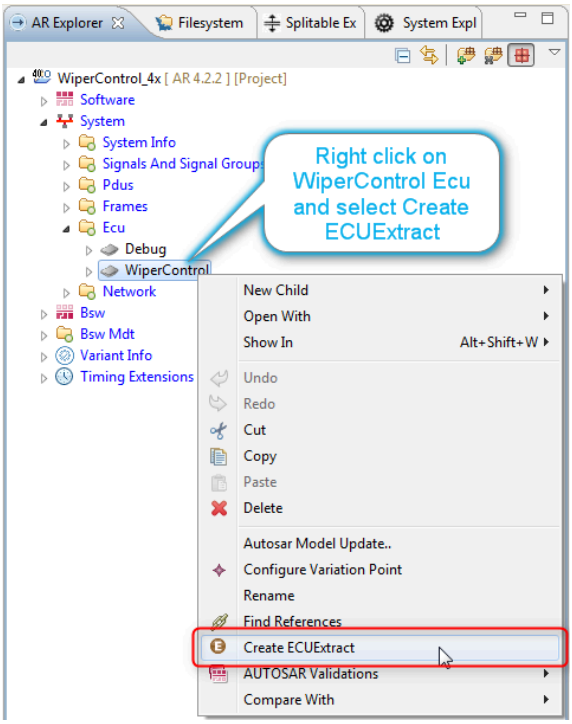

Figure 5.113.Invoking ECUExtract

2. This opens the “ECUExtract” wizard as shown below. None of the fields or options are mandatory, so you can go ahead and click **Finish**.

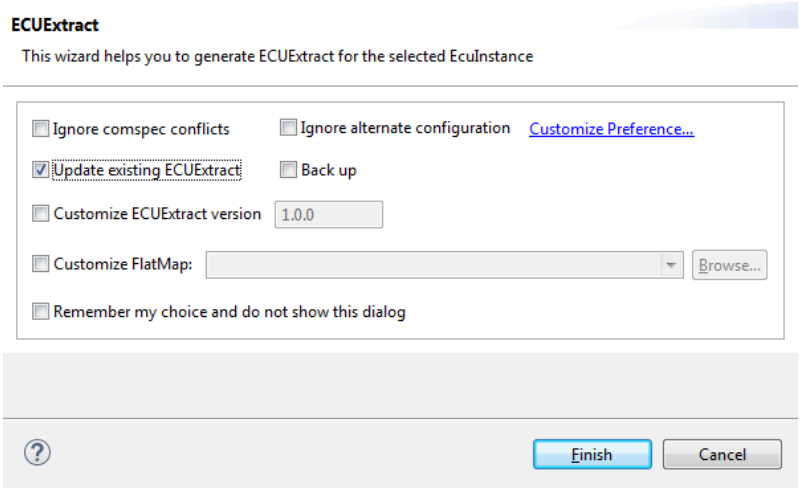

Figure 5.114.Generate ECUExtract

3. The ECUExtract will be generated and its associated files and AUTOSAR Elements will be created (see step 4). To view the files created, right-click on the project and select **Show In** > **AUTOSAR Explorer**.

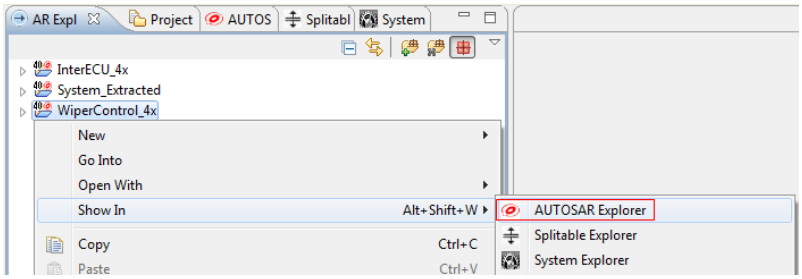

Figure 5.115.Invoking AUTOSAR Explorer

4. In the AR Explorer view, you can see the newly created files and AUTOSAR elements (i.e. the FlatMap, FlatView and EcuExtract) from the ECUExtract generation.

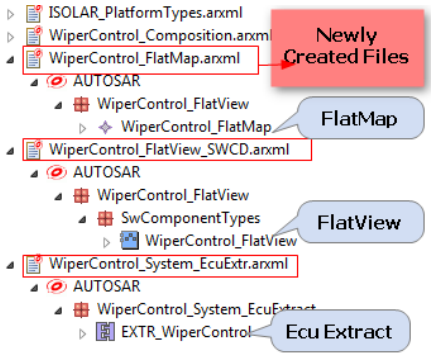

Figure 5.116.AR Explorer

Note: You can also create an ECUExtract in the command line mode. For more information on command line mode please see ISOLAR-A Help Reference Manual.

5.6.7 Creating OsTask using RunnableToTaskMapping Editor

The next step is to create an OsTask, which we can do using the "RunnableToTaskMapping" editor.

1. You first need to copy the *08_COMSTACK_CFG.arxml* file from the InterECU_4x project. This file contains the EcucModuleConfigurationValues.
2. You then need to configure the ECU EXTRACT reference to the generated ECU EXTRACT, as shown here.

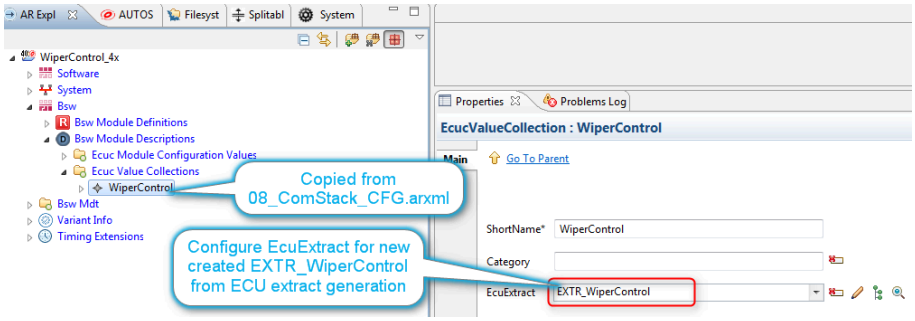

Figure 5.117.Configure EcucValueCollection

3. Then open the RTE Editor on the **WiperControl** EcucValueCollection.

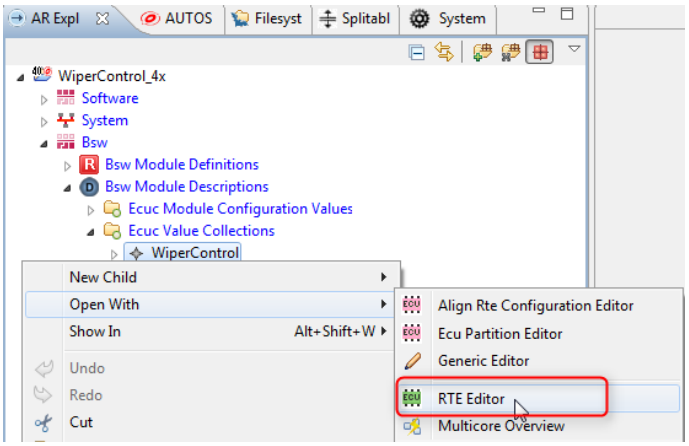

Figure 5.118.Invoking the RTE Editor

4. You can ignore the message at the top of the RTE Editor. Note that the two Runnables from the components that are mapped to our “WiperControl” ECU, which are contained in the “EXTR_WiperControl ECUExtract”, are displayed in the Unmapped Entries table on the right.

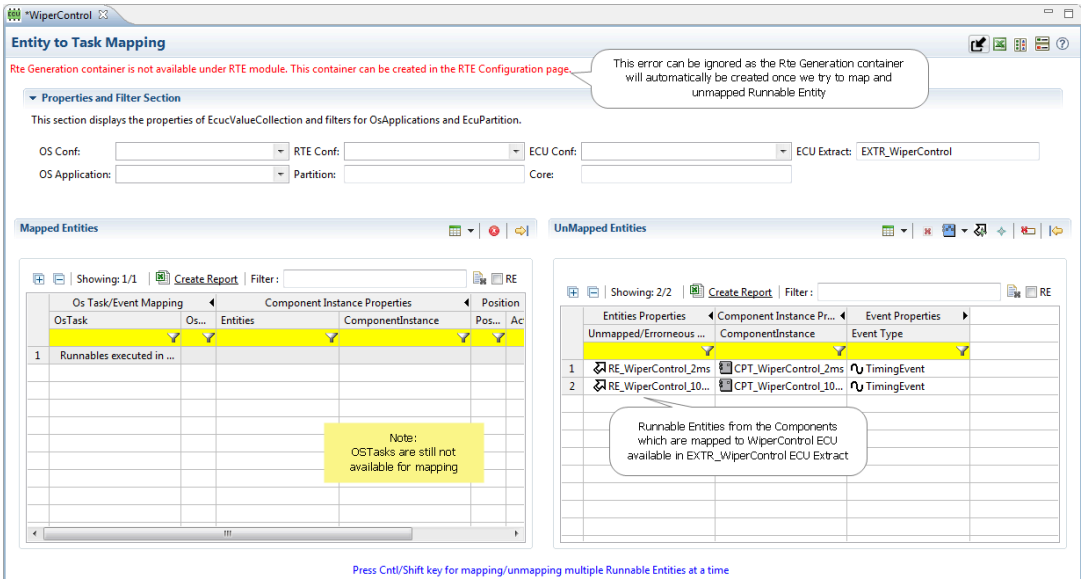

Figure 5.119.RTE Editor - Entity to Task Mapping

5. Right-click in the Mapped Entities table on the left, select “Create OSTask”, then configure the new task as shown here.

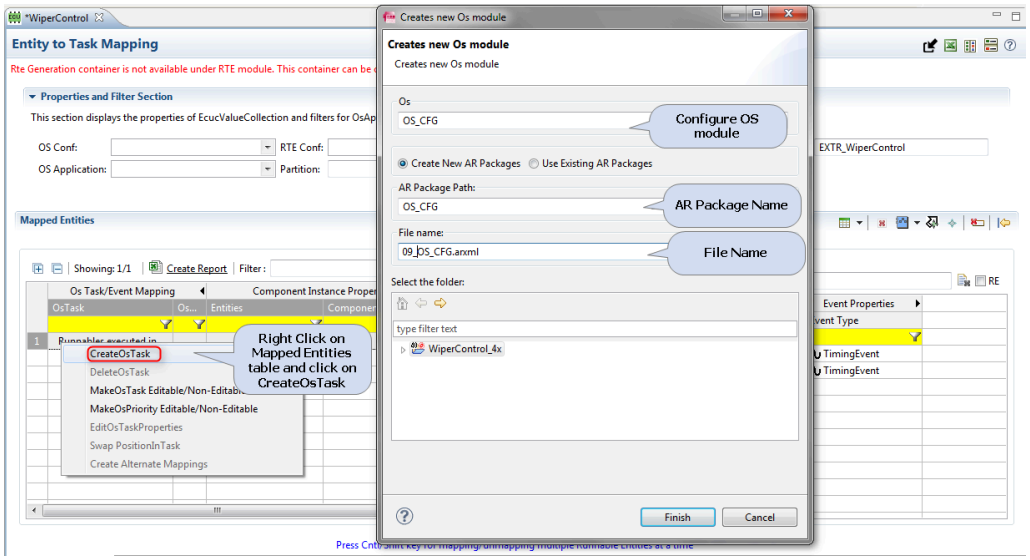

Figure 5.120.RTE Editor - Create new OS Task

6. The new OsTask will be created, but you will not be able to edit it until you enable the editing with the “Make OSTask Editable” option, as shown here. You can then rename the task to “OsTask_2ms”.

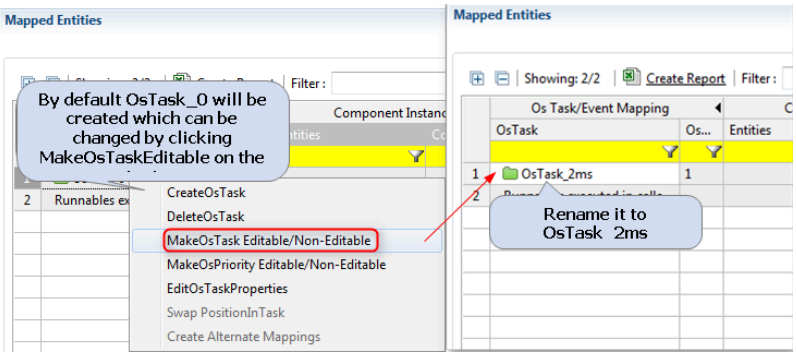

Figure 5.121.RTE Editor - Enable editing of the OS Task

7. Repeat the above step to create "OsTask_100us", and the RTE Editor will display the two OsTasks in the Mapped Entities table.

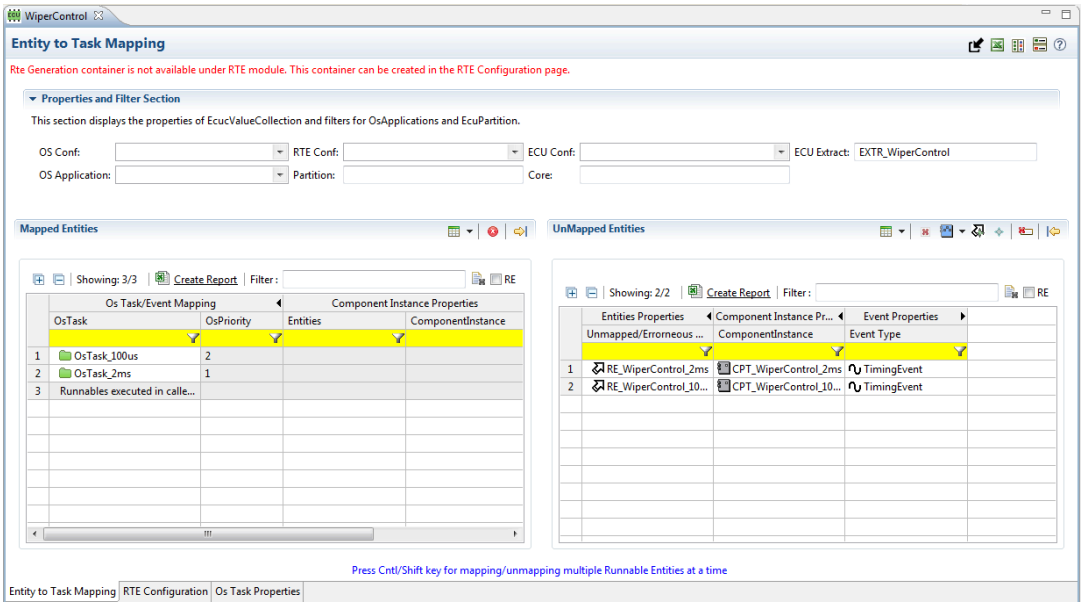

Figure 5.122.RTE Editor with two OS Tasks created

Note: Creating RunnableToTaskMapping can also be achieved in the command line mode. For more information, see the ISOLAR-A Help Reference Manual

5.6.8 Mapping of SWC Runnable Entities on the Operating System

The final step in the configuration of our example project is the configuration of the RTE Module, which involves the mapping of Runnable Entities to available OS Tasks (OS Raster). This requires ECU Configuration Values (with BSW module information generated with the ECUExtract).

1. In the RTE Editor, drag and drop **RE_WiperControl_100us** on the right, to the **OsTask_100us** OsTask on the left, to create the mapping.

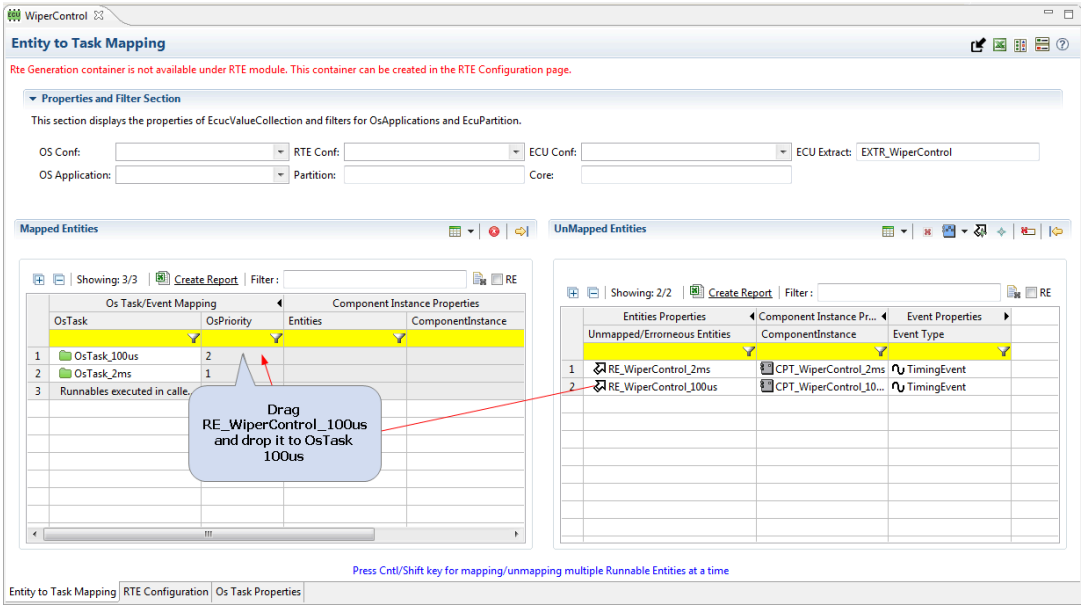

Figure 5.123.RTE Editor - Map Runnable to Task

2. As no RTE Module currently exists, we must create one, as shown here.

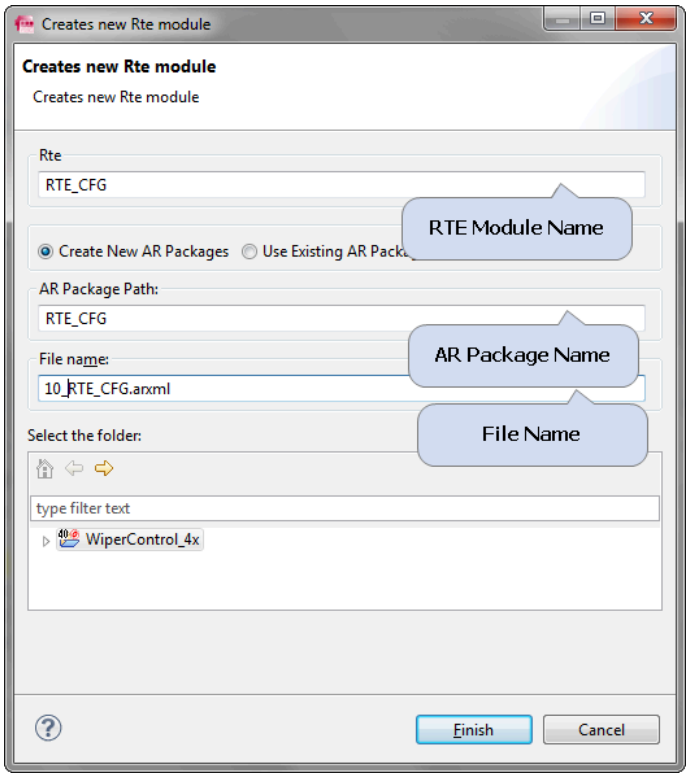

Figure 5.124.RTE Editor - Create new RTE Module

3. The RunnableEntities will be mapped to the appropriate OsTask.

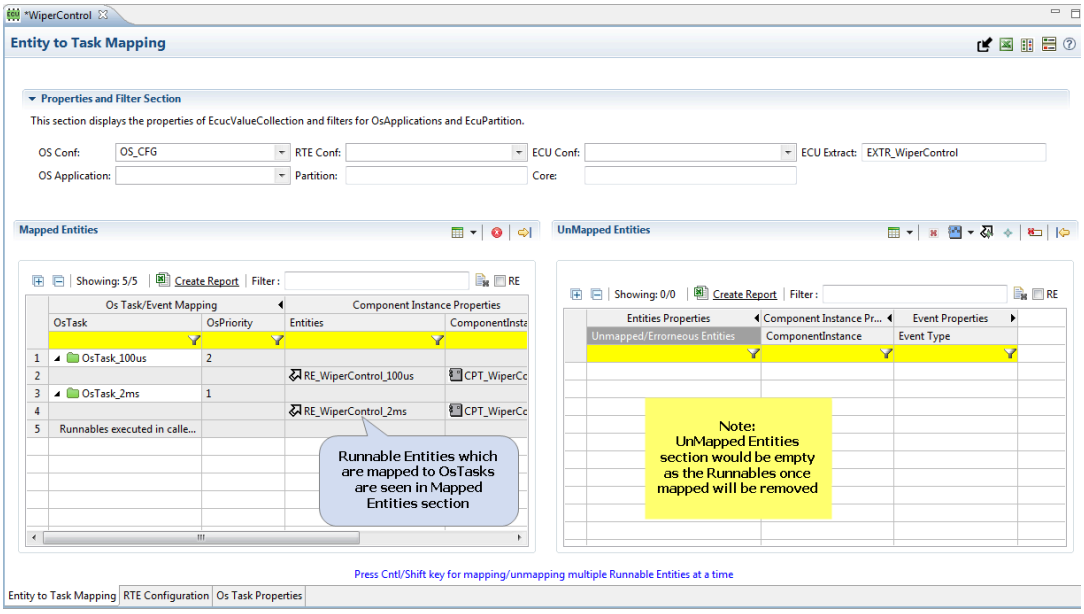

Figure 5.125.RTE Editor - Mapped entities

4. When you repeat the above steps for other Runnables, there will be no need to create the RTE Module as it already exists.

5.7 CP Flex Configuration

5.7.1 Creating the CP Software Cluster

The CP Software Cluster can be created in the CP Flex Mapping Editor.

1. To open the CP Flex Mapping Editor, right click on System and select **Open With** > **CP Flex Mapping Editor**.

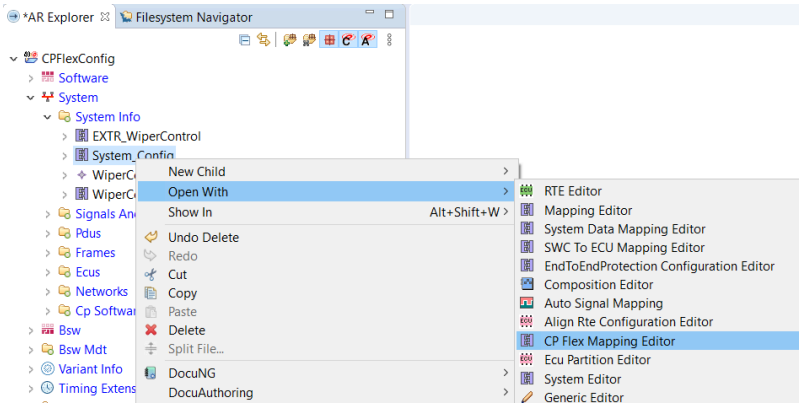

Figure 5.126.Invoking the CP Flex Mapping Editor

2. Do the following in the CP Flex Mapping Page (see below):

- In the drop downs at the top of the editor, select the **System Mapping** and **ECU** to create the CP Software Cluster for the specified ECU.
- Drag the “New CP Software Cluster” item from the palette on the right and drop it into the table to create the new CP Software Cluster.
- Select the “Component Instance” from the palette and drop it on the required Cluster to create the Software Cluster to Component mapping.

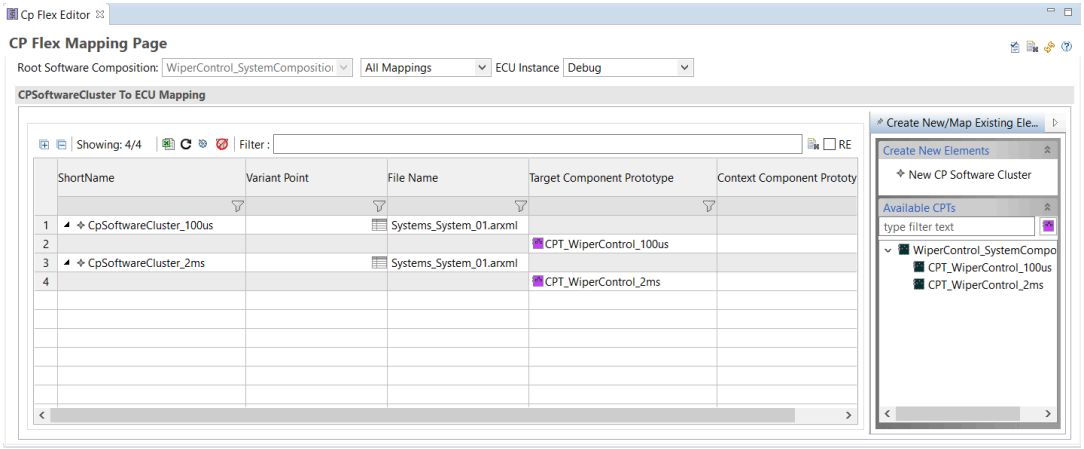

Figure 5.127.Creating the CP Software Cluster

5.7.2 Creating the Cluster Extract

After creating Clusters on a System Description level, you need to extract information for each specific Cluster into a cluster Software Description.

1. Right click on System and select **Create Cluster Extract**.

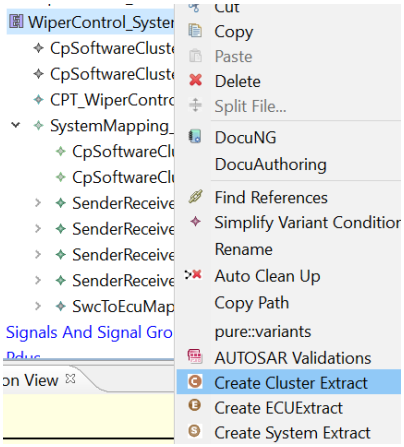

Figure 5.128.Invoking Create Cluster Extract

2. In the “Cluster Extract” dialog, the System name, file name and Package Path are auto filled and can be altered as required. Select the CpSoftwareCluster that needs to be extracted, then click **Finish** to generate the extract.

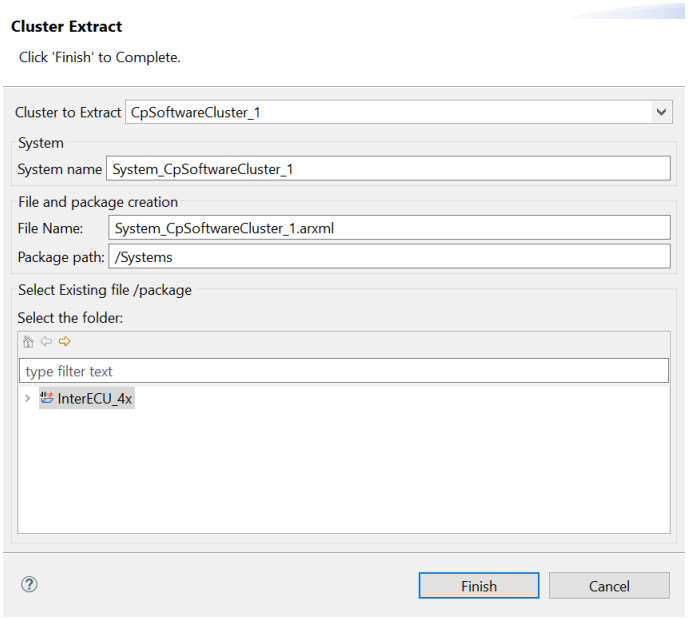

Figure 5.129.Cluster Extract dialog

5.7.3 Creating ECUExtract for a Cluster

After creating the Cluster extracts for each CpSoftwareCluster, the final step is to generate an ECUExtract for each cluster extract.

1. Right-click on the ClusterExtract System (category: SW_CLUSTER_SYS-TEM_DESCRIPTION) and select the **Create ECUExtract** option.

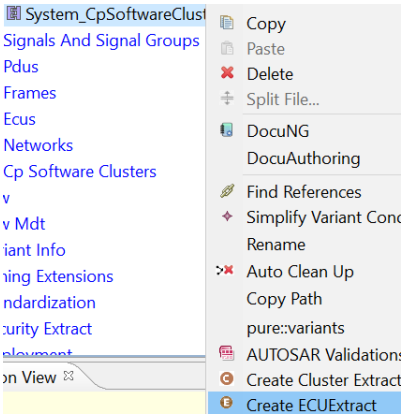

Figure 5.130.Invoking Create ECUExtract

2. This opens the “ECUExtract” wizard as shown below. None of the fields or options are mandatory, so just click Finish, and this will flatten only the cluster specific information.

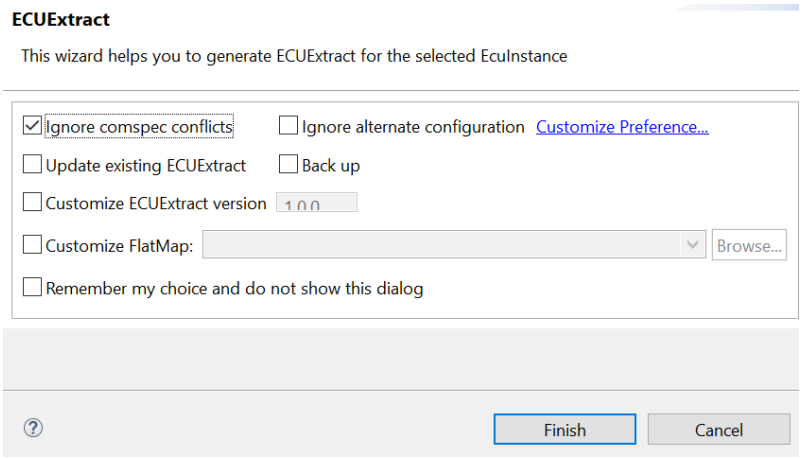

Figure 5.131.ECUExtract dialog

5.8 Validation

5.8.1 Introduction

Validation is a set of AUTOSAR defined constraints to help ensure the correctness of the configuration that has been created.

When activating the Validation feature, AUTOSAR Software and System related artifacts will be validated.

5.8.2 Validation in ISOLAR-A

ISOLAR-A provides validation support at the levels of System template and SWC template.

1. To validate the complete configuration within a project, right-click on the project and select **AUTOSAR Validations** › **Run Profile based Validations**, then click **OK** in the Run Validations dialog.

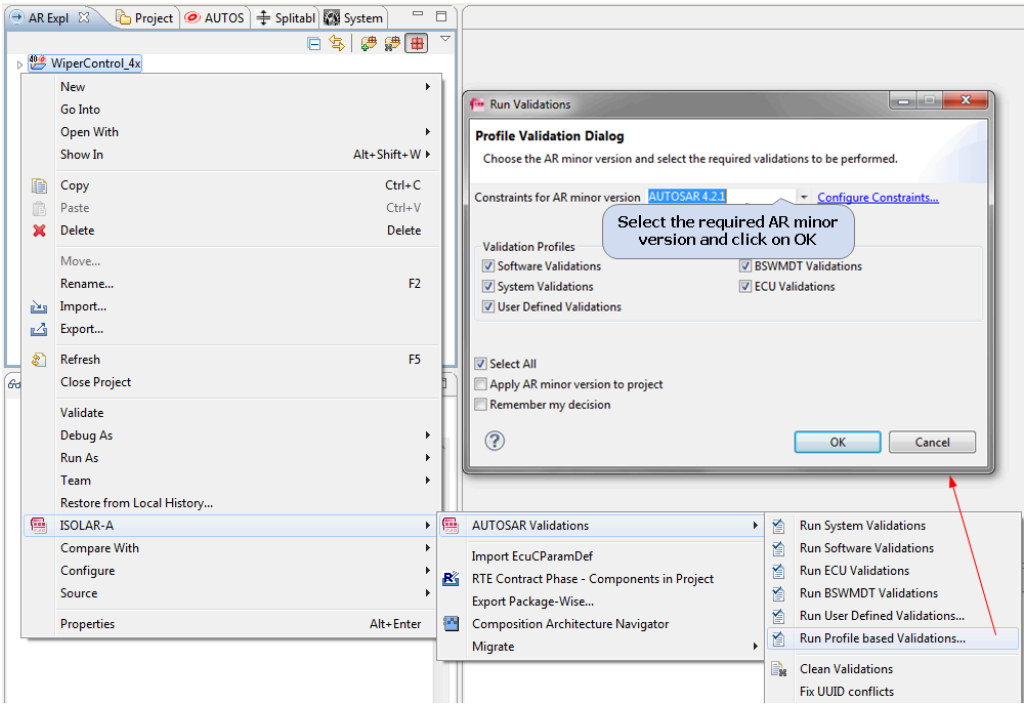

Figure 5.132.Invoking Validation

2. The validated information will be available in the Problems Log. Double click on any error in the log to open it in the Generic Editor.

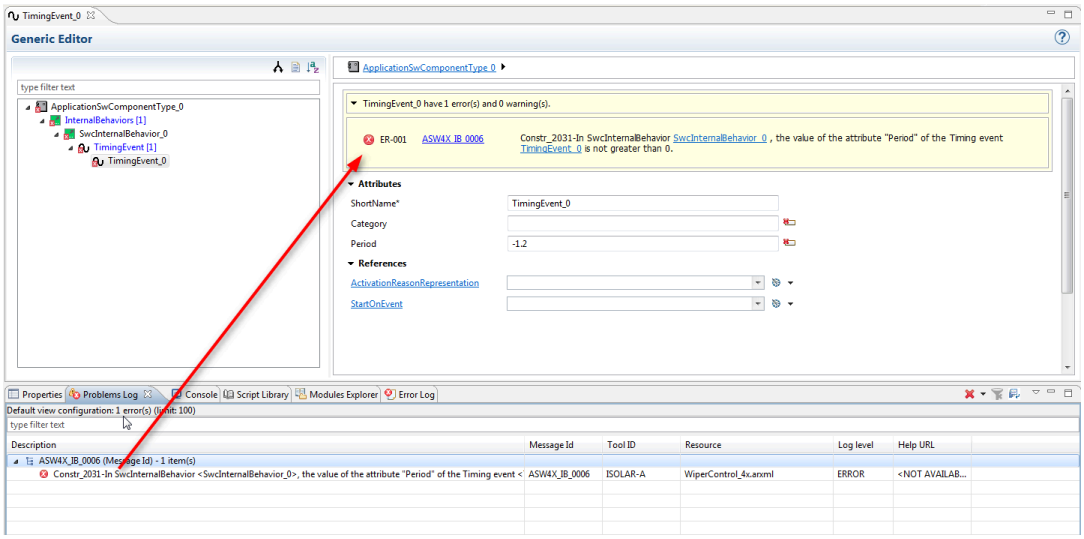

Figure 5.133.Validation errors in the Problems Log

Validations can also be executed in the command line mode. For more information on command line mode please see headless validation in ISOLAR-A Help Reference Manual.

5.8.3 Quick Fix

In addition to providing error information in the Generic Editor, the respective attribute or reference which is the subject of the error is highlighted, and when you click an error marker, a drop down listing displays Quick Fix options.

Step 1: As shown earlier, on running the AUTOSAR validations, all the errors will be logged in the Problems Log.

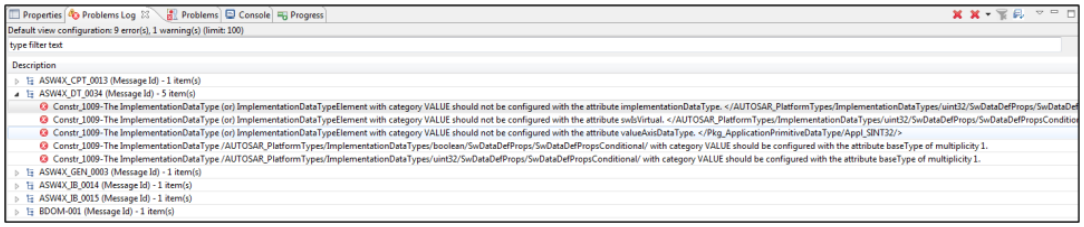

Figure 5.134.Quick Fix - Problems Log

Step 2: Double clicking an error message in the Problems Log opens it in the Generic Editor, as shown below

- When you click on an Error Marker, quick fixes relating to the Validation ID are displayed and you can select the fix option you wish to run.
- As an example, a Non-Autosar Complaint Shortname for a Composition can be Auto-Corrected to an Autosar Complaint name by selecting the required option.

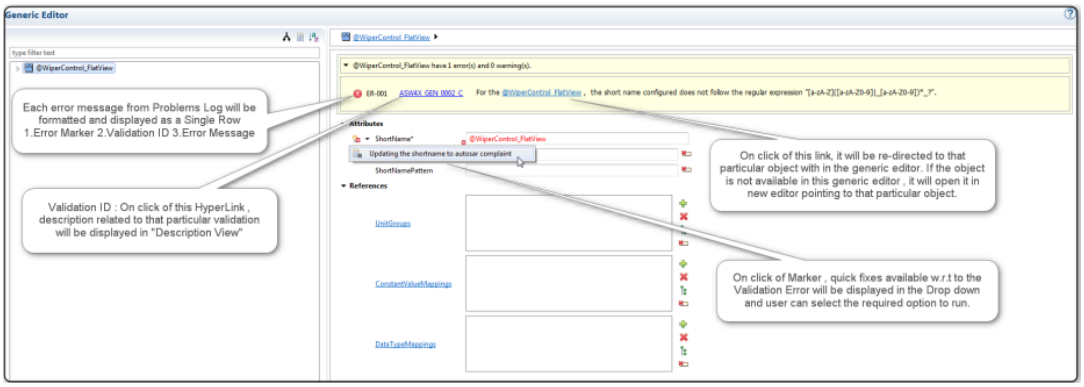

Figure 5.135.Quick Fix - Problems Log (Select Fix option)

5.8.3.1 Invoking Quick Fix

Various invocation options are available for Quick Fix:

Attribute Level: On opening the Generic Editor, an Attribute with a configuration Error will be marked with an Error Icon which, when clicked, will display a drop down menu of fix options.

Error Info Bar: The error section at the top of the Generic Editor, which is used to display information about the Validation, contains the Error Icon that can be clicked to quick fix the error (without having to navigate to the error itself).

5.8.3.2 Types of Quick Fix

The following two types of Quick Fix are available

- Auto Fix
- Semi-Auto Fix

AUTO-FIX

Validations involving an error in an attribute (Numerical/Literal/String) can be fixed immediately (auto-fixed) without requiring you to make a choice.

Example: In an ApplicationDataType, if SwImplPolicy is FIXED then SwCalibrationSuccess should be configured to NOT-ACCESSIBLE. As shown below, the problem can be resolved automatically by clicking on the Quick Fix suggestion.

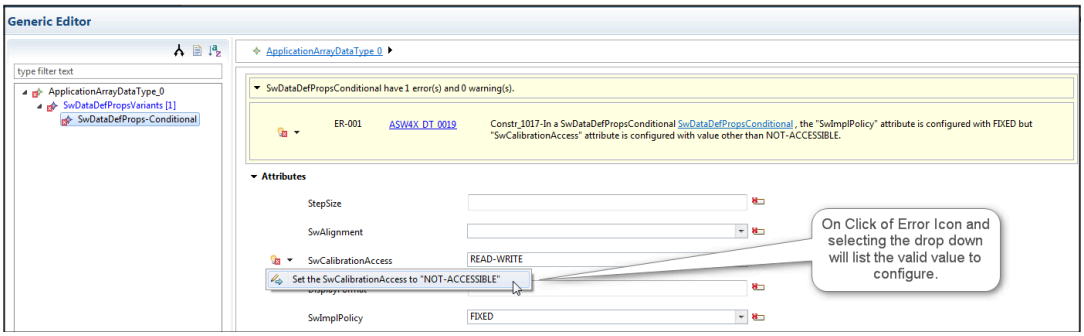

Figure 5.136.Auto-Fix

Once the error is fixed, the notification will be replaced by a green tick.

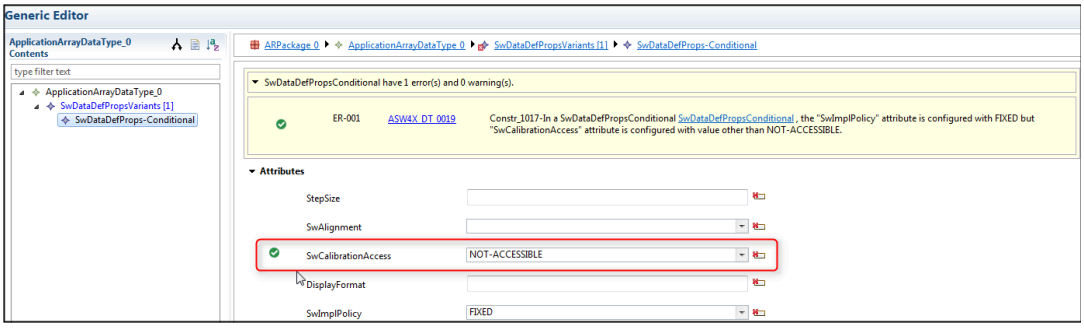

Figure 5.137.Error is fixed

SEMI-AUTO FIX

Reference errors can only be resolved by you choosing the appropriate corrective action from a list of available options.

Example 1 (Reference Dialog)

In this example there's an error in the attribute with respect to the configured reference. This can be fixed by manually selecting the valid value from the list displayed in the dialog.

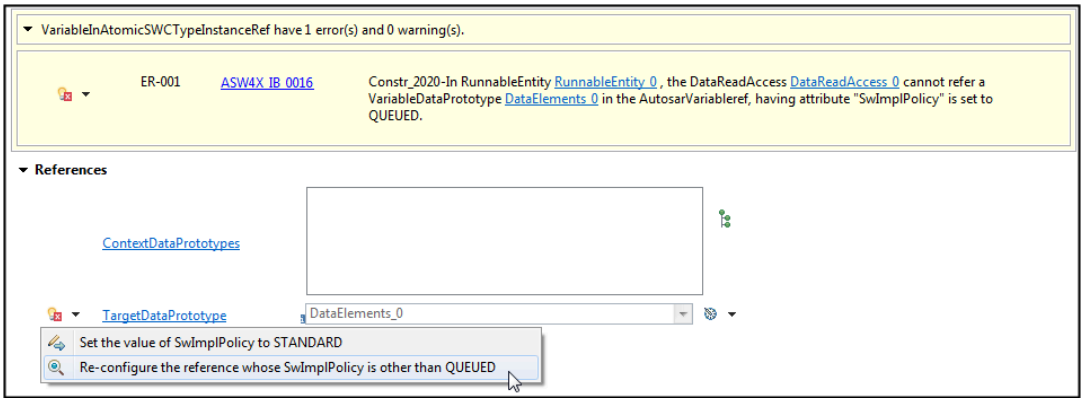

Figure 5.138.Semi-Auto Fix

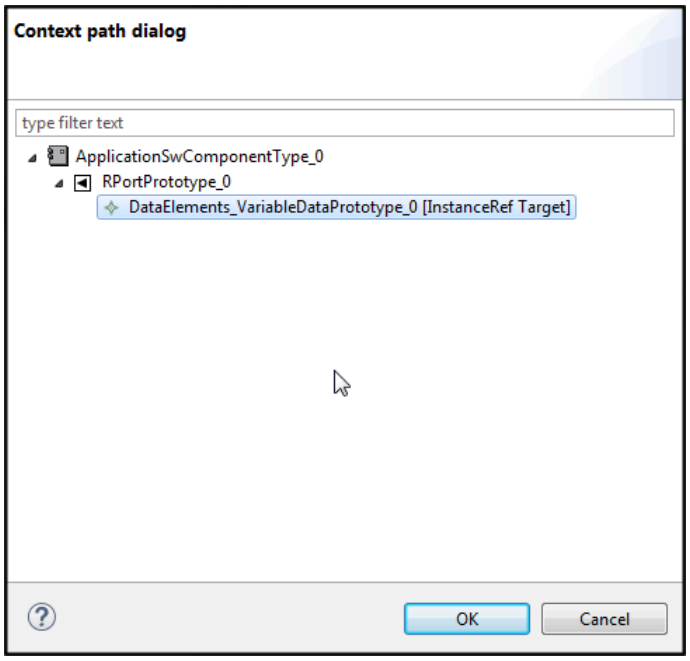

Figure 5.139.Manually configure the element

Example 2 (Bulk Fix Dialog)

The following images show a “bulk fix” being applied to multiple elements with the same error.

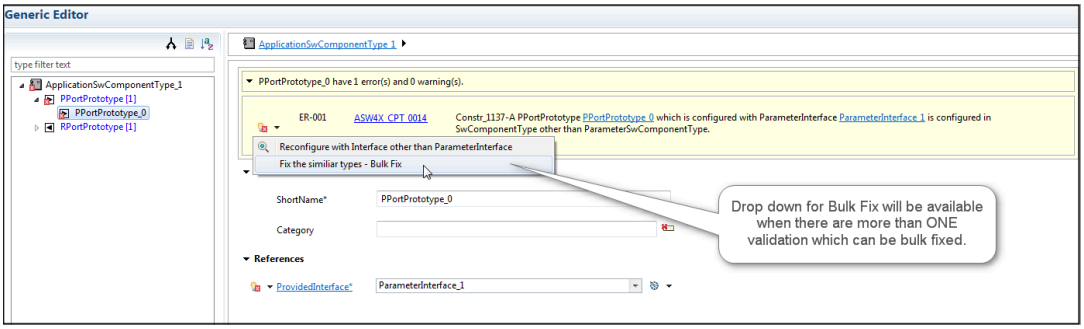

Figure 5.140.Bulk Fix

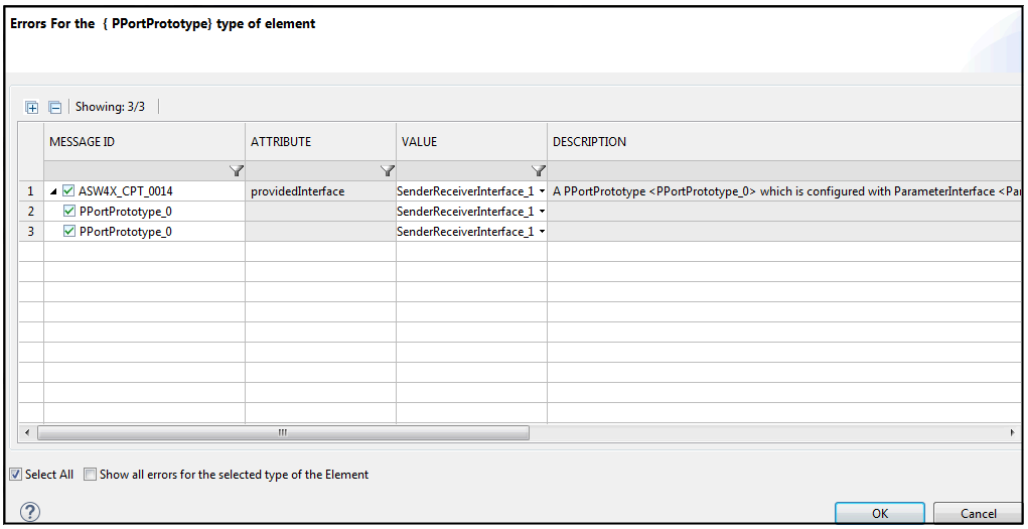

Figure 5.141.Bulk Fix

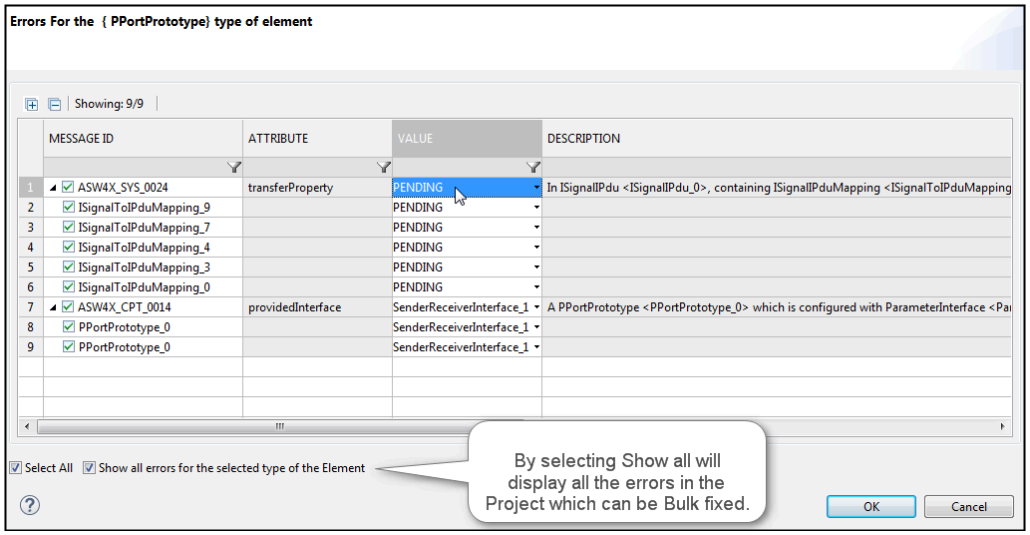

Figure 5.142.Bulk Fix

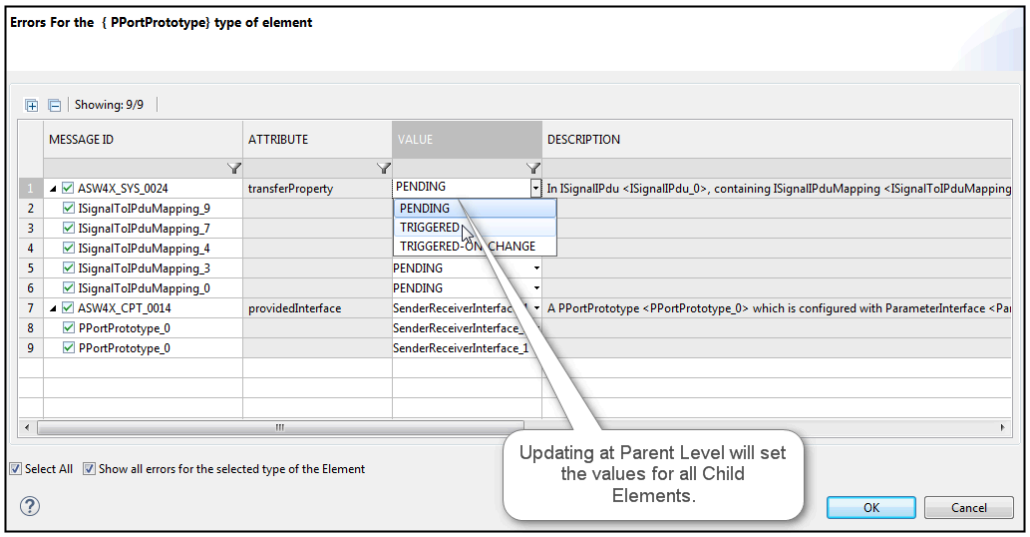

Figure 5.143.Bulk Fix

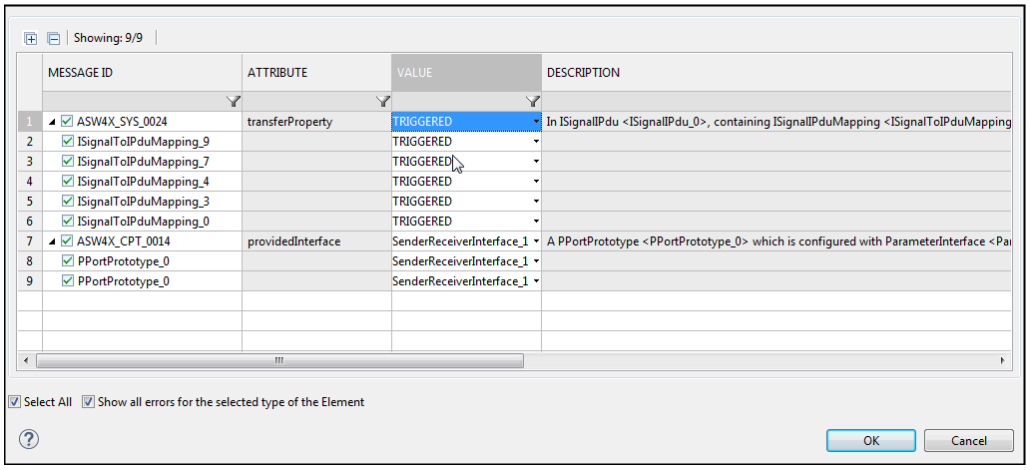

Figure 5.144.Bulk Fix

Example 3 (Retain References Dialog)

This next example shows validation where more than one child reference is required, and where you can provide the required input to add the additional /missing child references.

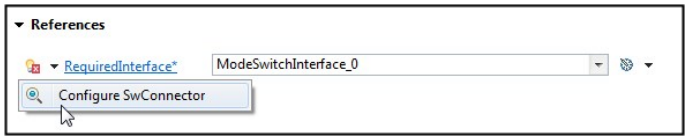

Figure 5.145.Retain References

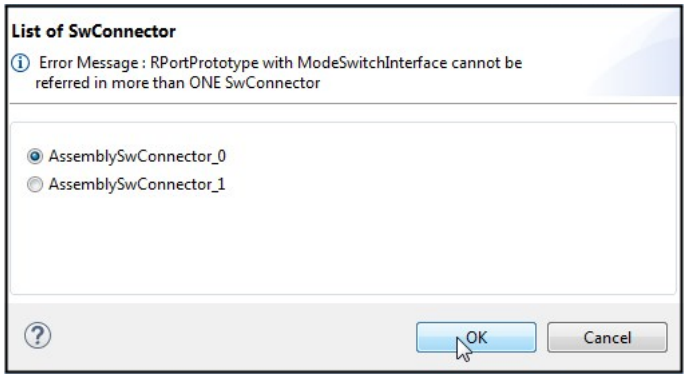

Figure 5.146.Retain References

Example 4 (ToolTip)

In this final example we see a validation involving the attributes which require you to input values manually (within a certain limit/range), with a tool tip guiding you to enter a valid value.

Figure 5.147.ToolTip

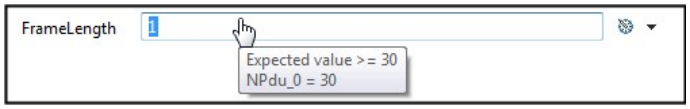

Figure 5.148.ToolTip

5.9 Code Generation

5.9.1 Introduction

ISOLAR-A provides C-code generation support for SWCs, and for the RTE via the ETAS RTA-RTE generator.

5.9.2 Generation of SWC C-code skeletons

An SWC C-code skeleton can be generated using the ISOLAR-A Component Code-Frame wizard.

1. Right-click on the relevant software component, then select **Generate** > **Component Code-Frame**.

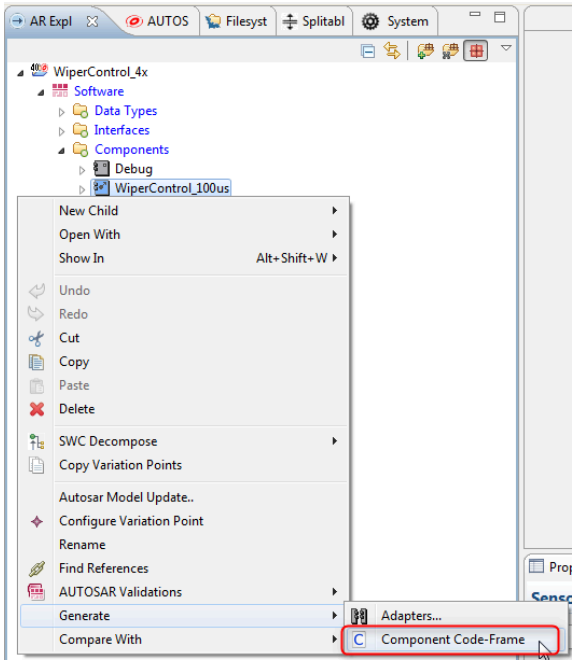

Figure 5.149.Invoking Component Code Frame

2. In the first page of the “Component Code-Frame” wizard, the Component name will be displayed and options for single or multiple C file are available:

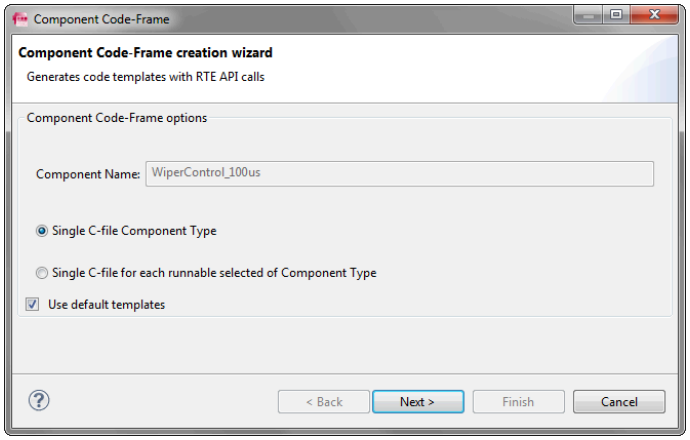

Figure 5.150.Component Code Frame wizard (page 1)

3. In the second page, the Component file name and source folder can be modified.

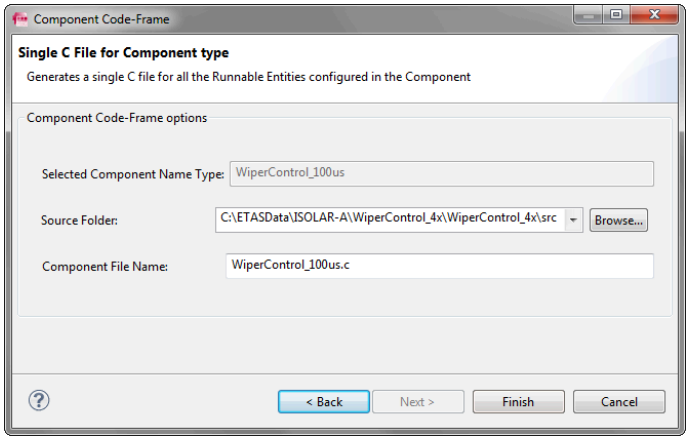

Figure 5.151.Component Code Frame wizard (page 2)

4. When you click **Finish**, the C file will be generated in the specified path.

5.9.3 Generation of the RTE C-code

RTE code can be generated by invoking the RTE generation wizard.

1. Select the “WiperControl_4x” project and the “R” Button in tool bar will be enabled.

Figure 5.152.Invoking the RTE Code Generation wizard

2. When you click “R” button, the “RTE Code Generation” wizard will be launched.

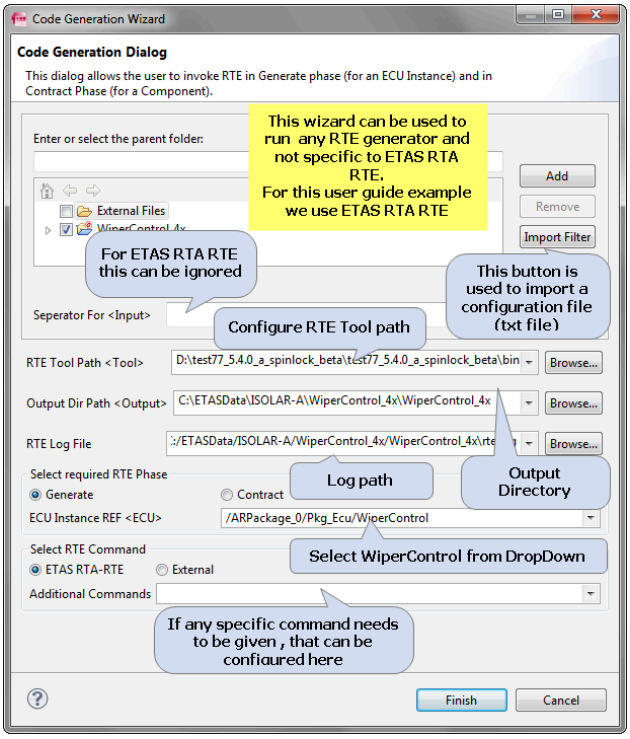

Figure 5.153.RTE Code Generation wizard

Note: Any RTE generation tool can be used for generating the RTE Code, including the ETAS RTA-RTE. Please note, however, that no RTE software is included in the ISOLAR-A delivery.

5.10 Generic Editor

5.10.1 Introduction

The Generic editor provides the basic editing functionalities for all kind of Autosar Elements (excluding BSW elements – See BSW Editor below), and it includes the following functionality:

- Editing AUTOSAR Templates (Software, System, ECU Resources)
- Undo/redo, Cut/Copy/Paste
- Double-clicking on an AR template (except where an element has a specific editor) will open in the Generic Editor.
- Double-clicking in the Problem Log will also default open in the Generic editor.

The Generic Editor can be opened in the Context Menu of the AR Explorer for an Autosar project.

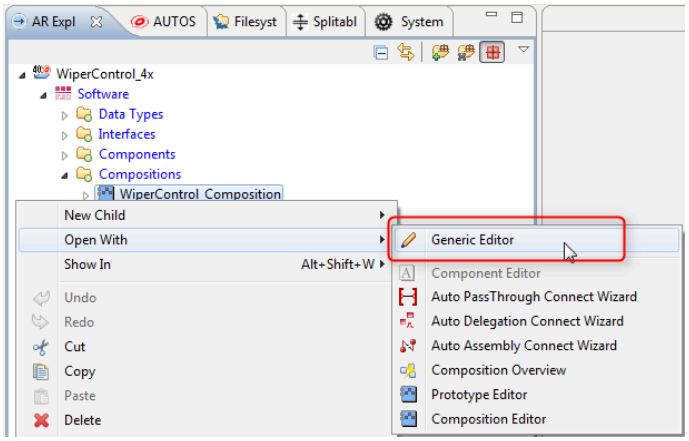

Figure 5.154.Invoking the Generic Editor

5.10.2 BSW Generic Editor

The BSW Generic Editor provides the basic editing functionalities for the BSW containers Elements, and it includes the following functionality:

- Editing of RTE, OS and COM Module elements
- Support for Undo/redo, Cut/Copy/Paste

The BSW Generic Editor can be opened in the Context Menu in AR Explorer for an Autosar project.

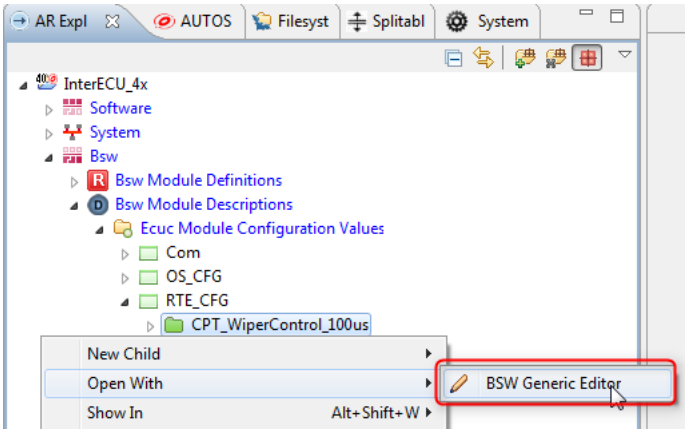

Figure 5.155.Invoking the BSW Generic Editor

5.11 System Editor

5.11.1 Introduction

The System Editor is designed exclusively to handle system related data. Its features include editing, adding, deleting and mapping system elements, and it

can be opened by double-clicking on a System Element in the AR Explorer.

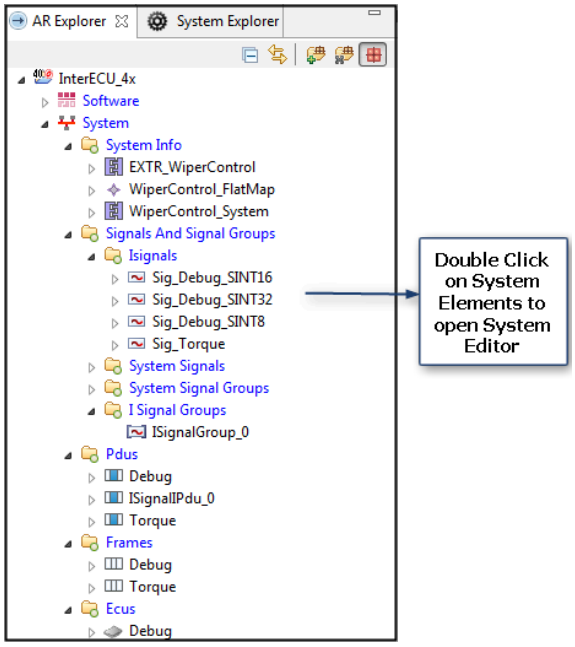

Figure 5.156.Invoking the System Editor (on System Element)

Or you can click **Open with** › **System Editor** on Project/System Elements in the AR Explorer.

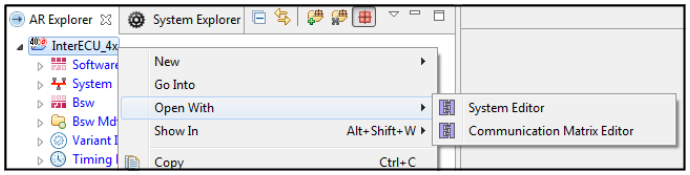

Figure 5.157.Invoking the System Editor (via Open With from AR Explorer)

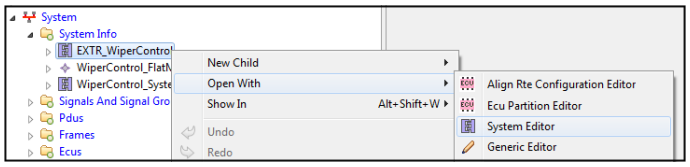

Figure 5.158.Invoking the System Editor (via Open With from System Explorer)

5.11.2 The System Editor UI

The System Editor is divided into three primary sections, as shown in the example below:

- System Elements Navigator

- Editor
- Create/Map Existing Elements

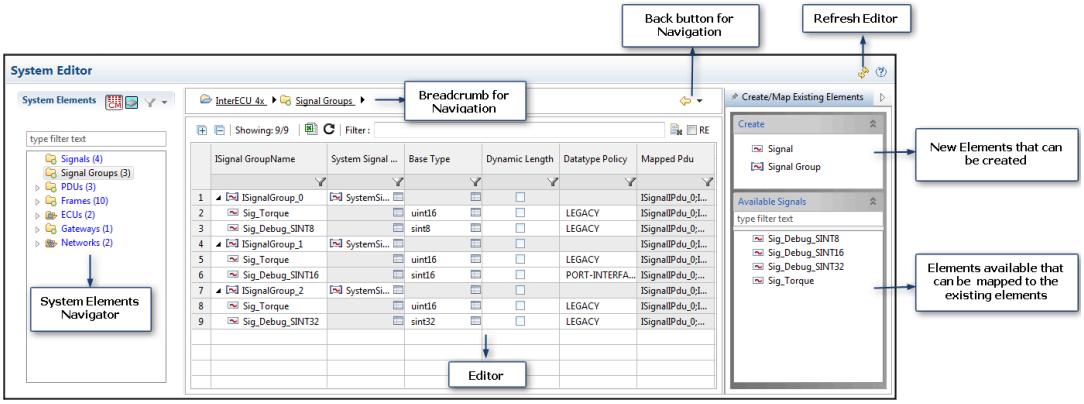

Figure 5.159.System Editor overview

5.11.2.1 System Elements Navigator

The System Elements Navigator is used to navigate to the corresponding editors.

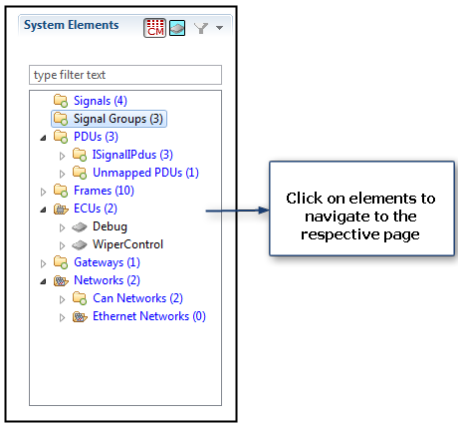

Figure 5.160.System Elements Navigator

5.11.2.2 Editor

The contents of the Editor will change based on the selection in the System Navigator. When the Editor is opened on a container, all the elements are shown with the relevant data in tabular form.

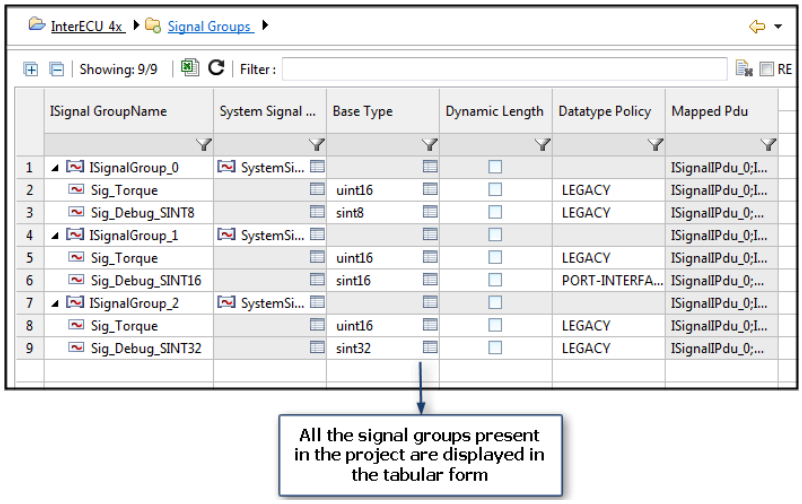

Figure 5.161.Editor - Tabular view

If the Editor is opened with leaf element, the element properties are shown in the editor.

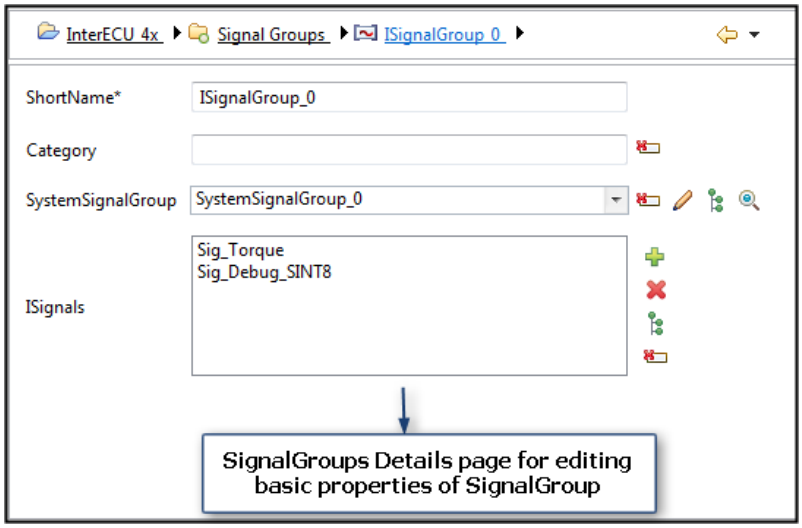

Figure 5.162.Editor - Details view

5.11.2.3 Create/Map Existing Elements

The “Create/Map Existing Elements” section shows new elements that can be created, and the existing elements that are available for mapping.

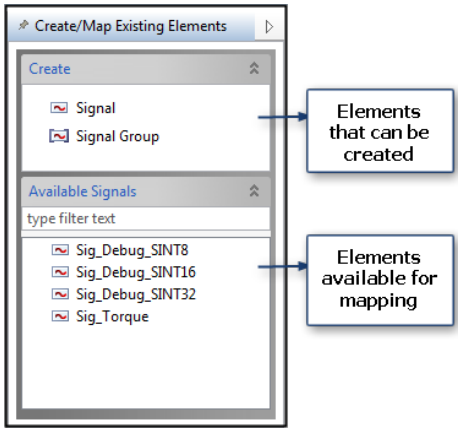

Figure 5.163.Create/Map existing Elements

Example

The Signal Group tables page allows the creation of new SignalGroups and Signals. All the available signals in the project are listed, and they can be mapped to the existing SignalGroup.

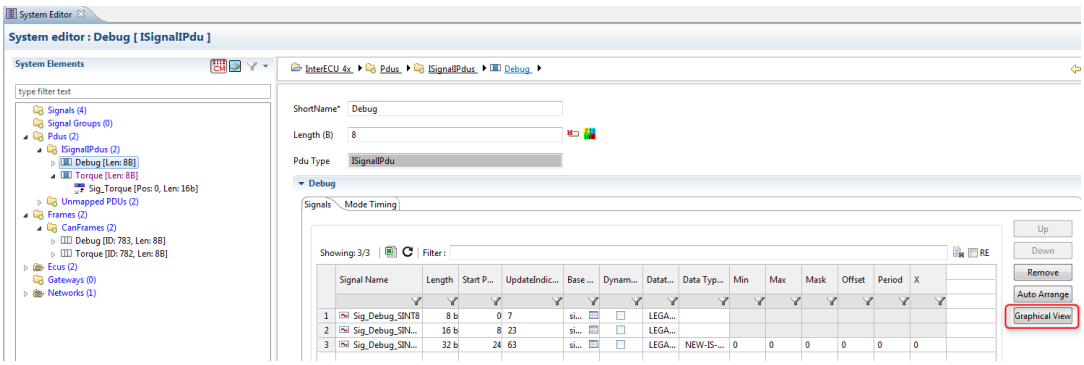

Figure 5.164.System Editor (Signal Group tables page)

5.11.3 Graphical View

The System Editor includes a Graphical View, which can be accessed by clicking on any of the ISignalIPdus and then clicking **Graphical View**.

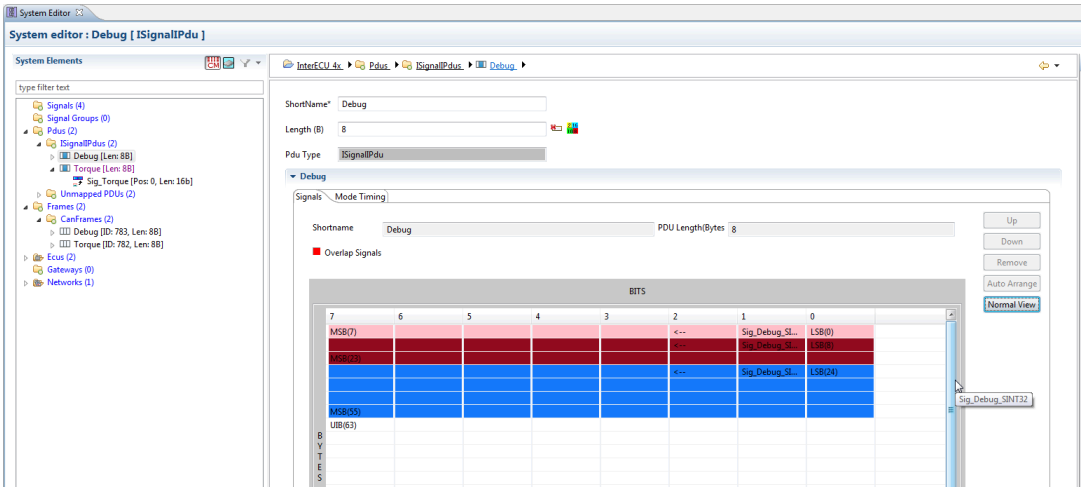

Figure 5.165.System Editor (Graphical view)

5.11.4 Properties View

The System Editor allows you to view the element properties in the same page.

- When you right-click on a Row header or any cell, it will display the property details in the right-hand palette section.
- Clicking “Show Advanced Props” shows the table sections of the selected element.
- Click on the palette header to switch between Palette and Property view.

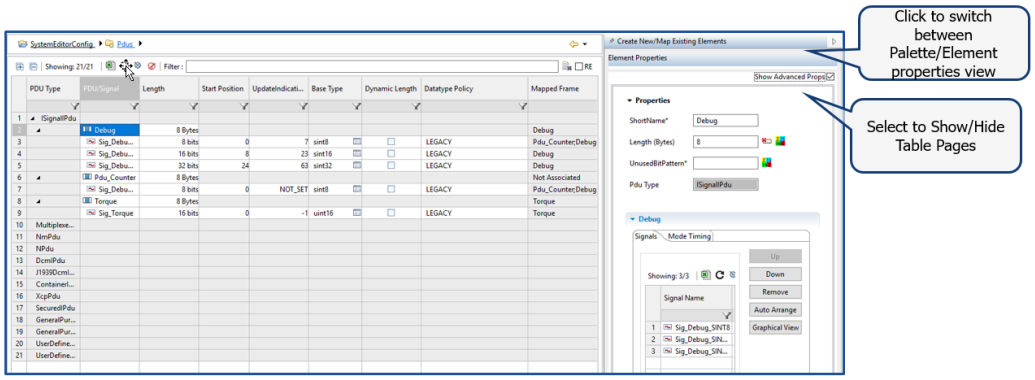

Figure 5.166.System Editor (Properties view)

5.11.5 Ethernet Auto Generation Wizard

The “Ethernet Auto Generation” wizard allows you to generate Ethernet configurations from Network communications, as well as from Swc communications.

- As shown below, the wizard can be invoked from within the System Editor via the “Ethernet Auto Generation” option.

- It can be invoked either as “Generate From Network Communication” or “Generate From Swc Communication”.

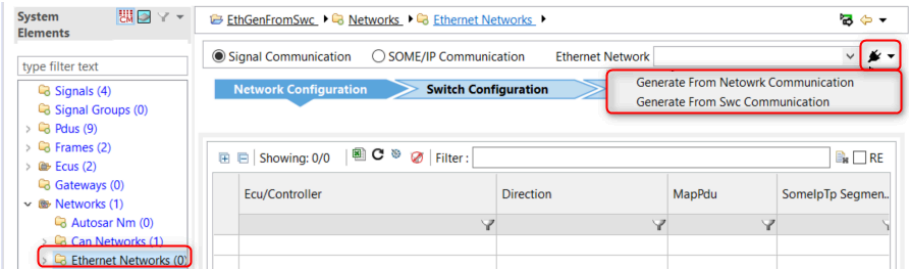

Figure 5.167.Ethernet Auto Generation

5.12 EndToEndProtection Configuration Editor

5.12.1 Introduction

EndToEndProtection (E2E) is used to protect safety related data, and the EndToEndProtection Editor helps you to quickly create and configure your EndToEndProtection. The editor supports:

- **Intra-ECU Configuration:** Configuring EndToEndProtection at the DataElement level
- **Inter-ECU Configuration:** Configuring EndToEndProtection at SignalGroups and Pdu levels
- **EndToEndProtection OverView:** Creating and configuring EndToEndProtections

5.12.2 Invocation

Open the editor by right-clicking the System to be edited and selecting **Open With** > **EndToEndProtection Configuration Editor**.

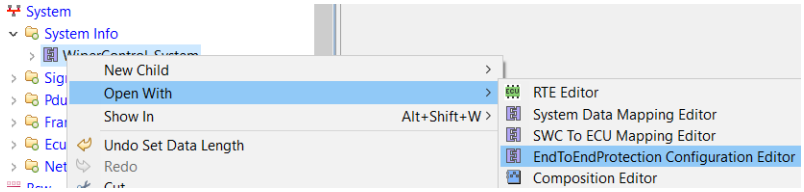

Figure 5.168.Invoking the EndToEndProtection Configuration Editor

5.12.3 Use cases

In this section we describe several use cases for the EndToEndProtection editor.

5.12.3.1 Use Case 1: Inter ECU

All the Pdus along with the relevant SignalGroup are shown in the table. If they are already configured with EndToEndProtection, those details are also shown. To create new EndToEndProtection, or to use the existing EndToEndProtection configuration, drag and drop the elements from the palette to the desired SignalGroup/Pdu.

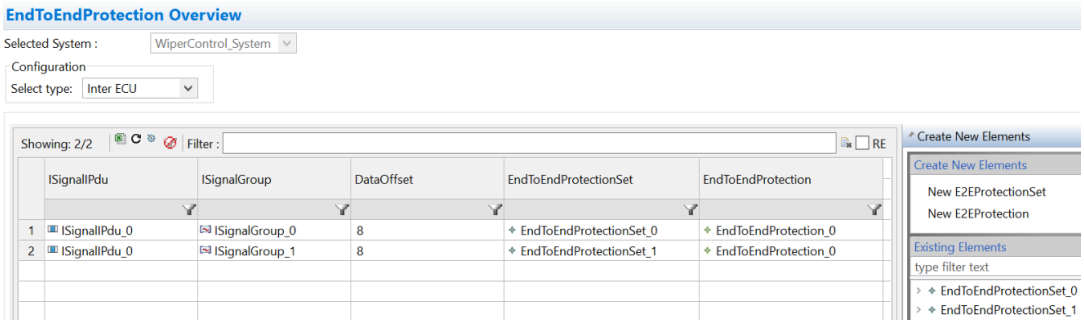

Figure 5.169.EndToEndProtection Overview (Inter ECU view)

5.12.3.2 Use Case 2: Intra ECU

All eligible DataElements (Complex DataElements) along with their context is displayed in the table. If a DataElement is already configured with EndToEndConfiguration, those details are also shown. To create a new EndToEndProtection, or to use the existing EndToEndProtection configuration, drag and drop the elements from the palette to the desired DataElement.

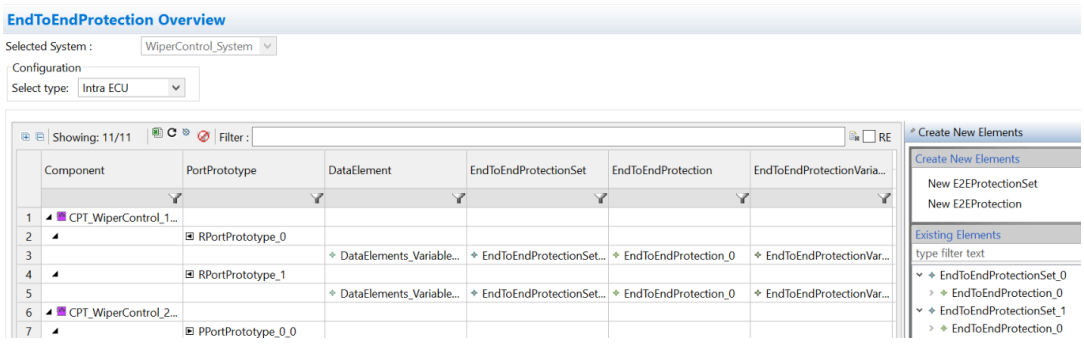

Figure 5.170.EndToEndProtection Overview (Intra ECU view)

If an existing EndToEndProtectionVariablePrototype with an existing sender is dragged and dropped on a PPortPrototype DataElement, the sender reference is overwritten.

5.12.3.3 Use Case 3: E2E Overview

All the existing EndToEndProtection configurations can be viewed in the EndToEndProtection Overview.

Attribute and shortnames can be updated from here, and new EndToEndProtectionSet and EndToEndProtection can be created as in other views.

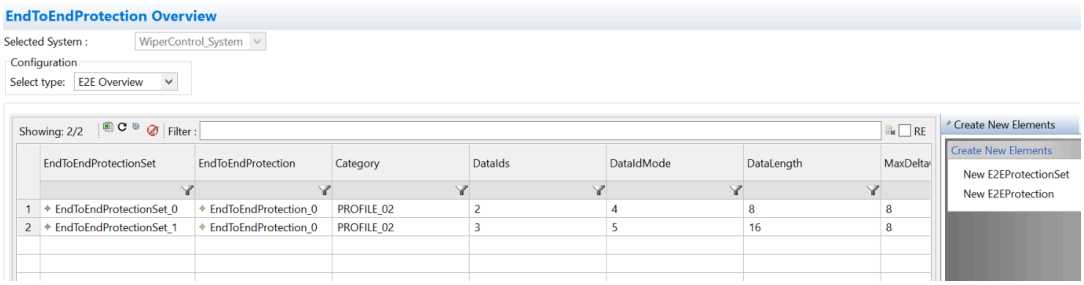

Figure 5.171.EndToEndProtection Overview

5.12.4 E2E Preferences

The following preferences can be set for the EndToEndProtection Configuration Editor.

- **File name:** The file under which all the newly created configuration is generated.
- **Package path:** The Package path that needs to be followed for creating new EndToEndProtection configuration elements.

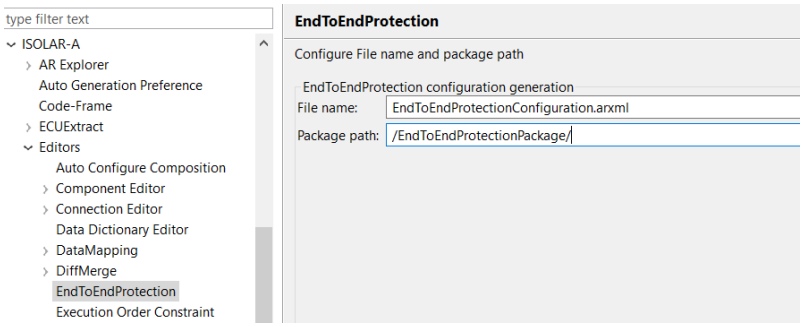

Figure 5.172.EndToEndProtection Preferences

5.13 Execution Order Constraint Editor

5.13.1 Introduction

The Execution Order Constraint (EOC) Editor allows you to edit support of the EOC functionality in ISOLAR-A. The editor provides the following features:

- Creating a requirement or guarantee timing constraint (an EOC element).
- Creating new EOCExecutableEntityRef objects inside an EOC element.
- Deleting EOC and/or EOCExecutableEntityRef elements of a Timing Extension element.

- Renaming EOC elements.
- Re-ordering EOCExecutableEntityRef objects inside an EOC element and computing the position in EOC values.
- Editing Timing Extension properties (e.g. the name or context).
- Computing the context of a Timing Extension element.
- Obtaining and displaying all the executable entities that are available from the current context.
- Inspecting the target SW component instance used as context, and the SW component type that defines the corresponding executable entity.
- Displaying available events (e.g. Timing Events of the available executable entities from current context).
- Drag and Drop Support.

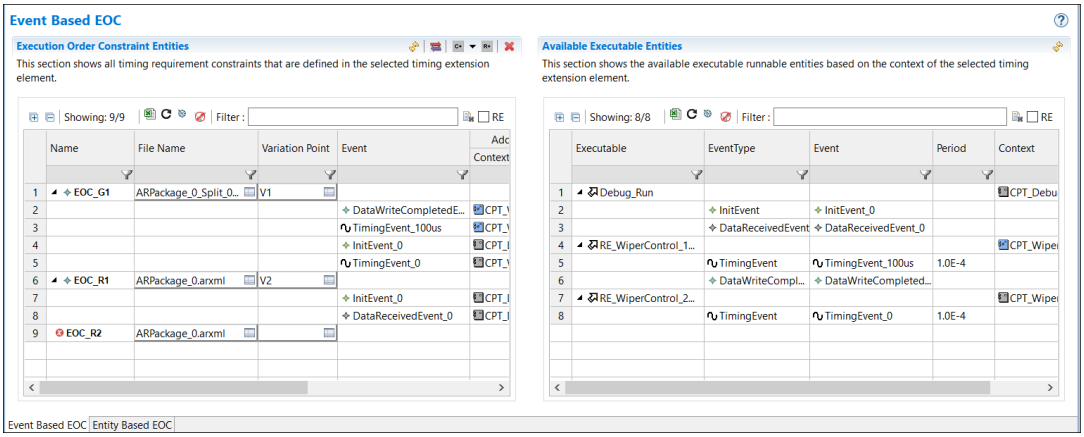

Figure 5.173.EOC Editor

5.13.2 Invocation

The Execution Order Constraint Editor is provided for VFB, System, SWC, ECU and BSW Timing Extension elements.

Right-click on the element, then select **Open With** › **Execution Order Constraint Editor**.

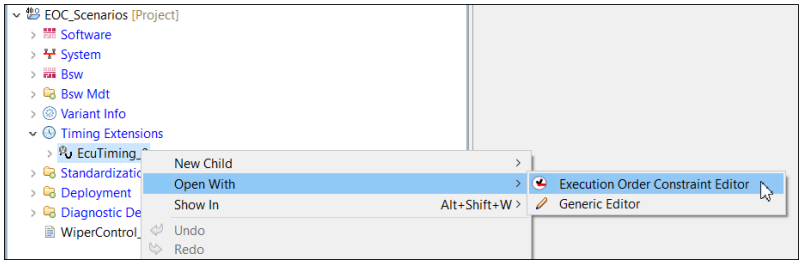

Figure 5.174.Invoking the EOC Editor

5.13.3 Editing Features

The following images illustrate the various editing features of the EOC Editor.

1. Creating a requirement or guarantee timing constraint (i.e., an EOC element).

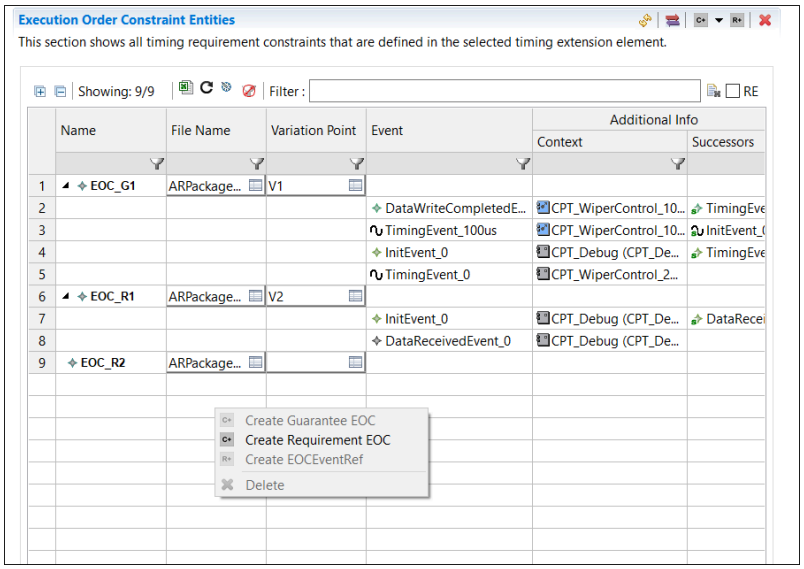

Figure 5.175.Create Requirement EOC

2. Creating new EOCEventRef objects inside an EOC element.

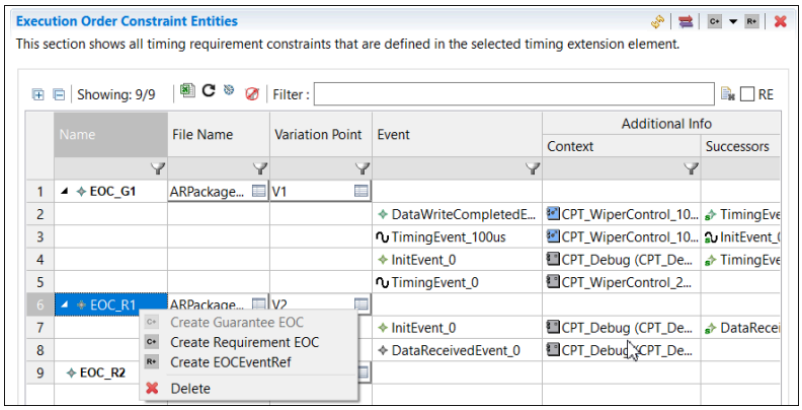

Figure 5.176.Create EOCEventRef

3. Deleting EOC and/or EOCEventRef elements of a Timing Extension element.

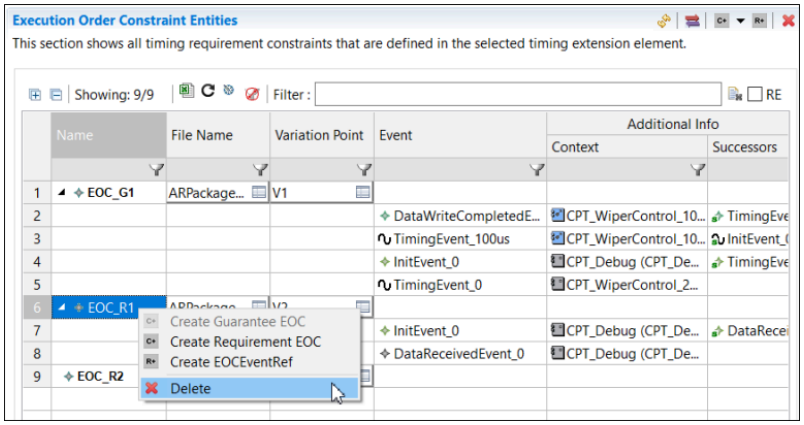

Figure 5.177.Delete EOC/EOCEventRef

4. Renaming EOC elements.

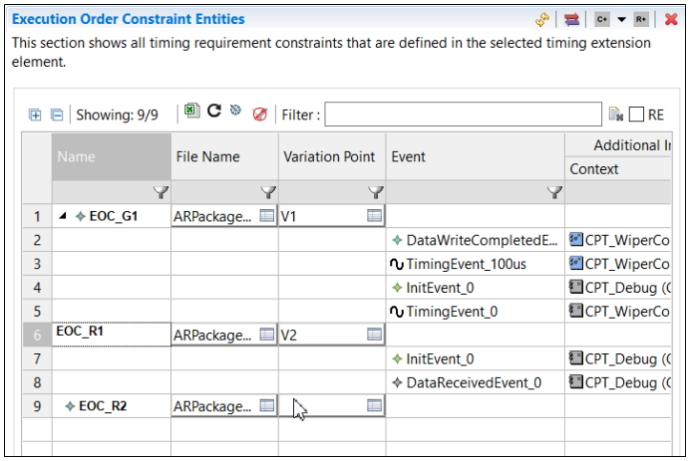

Figure 5.178.Rename an EOC Element

5. Obtaining and displaying all the executable entities that are available from the current context.

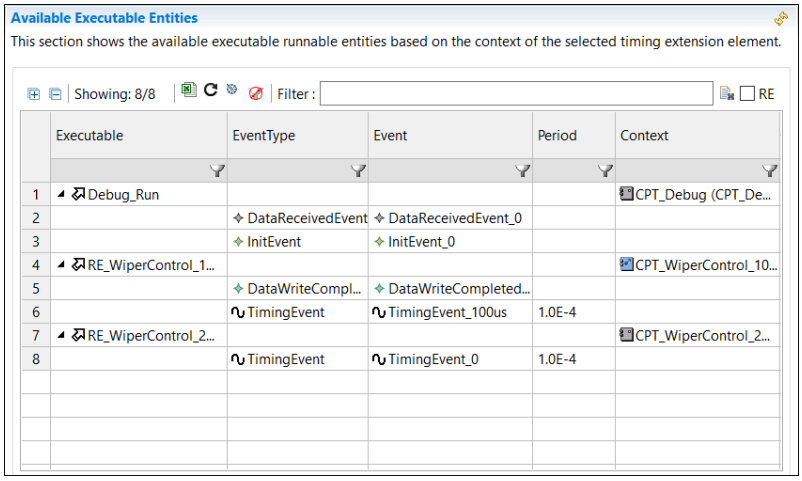

Figure 5.179.Display all executable Entities

6. Selecting executable entities and setting the referenced executable for an EOCExecutableEntityRef object (from one of the available executable entities from the context). EOCExectubleEntityRef can be grouped under EOCExecutableEntityRefGroup.

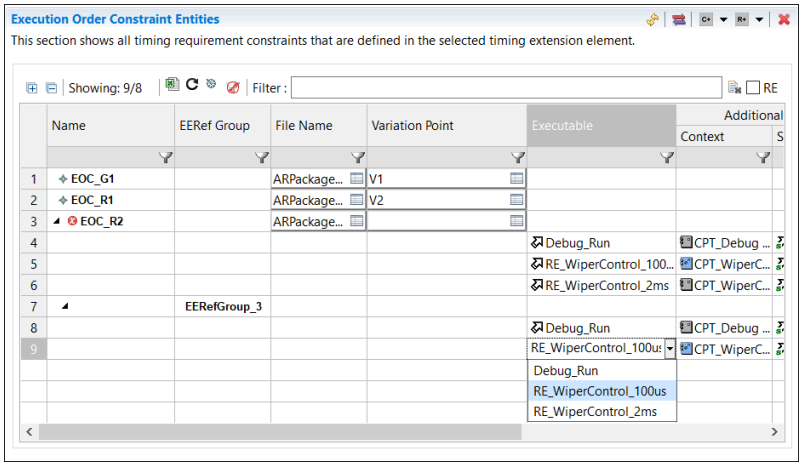

Figure 5.180.Select executable Entity

7. Inspecting the target SW component instance used as context, and the SW component type that defines the corresponding executable entity.

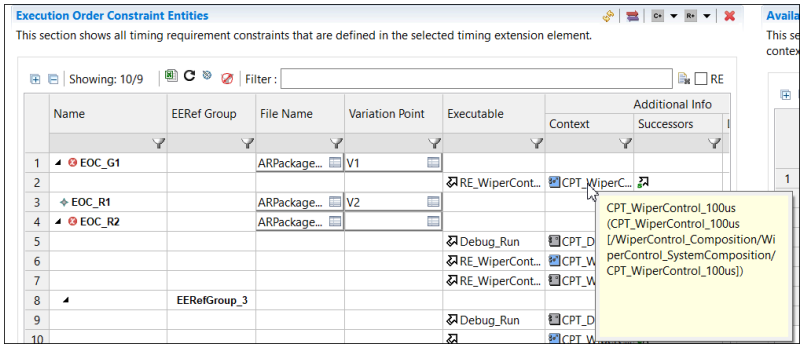

Figure 5.181.Inspect target SWC

8. Displaying available events (e.g. timing events of the available executable entities from current context).

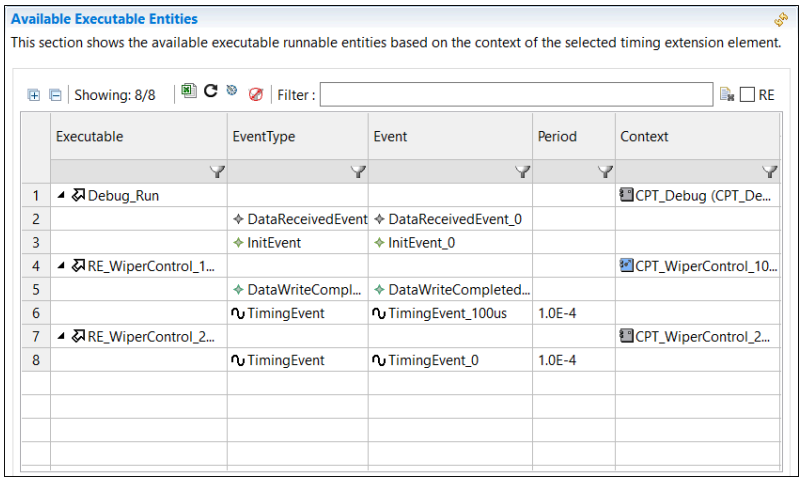

Figure 5.182.Display Timing Events

5.13.4 Drag and Drop Support

The EOC Editor supports the following types of drag and drop:

Re-Ordering EOCExecutableEntityRef elements

- Drag and drop one or more EOCExecutableEntityRef elements inside the same EOC object (for re-ordering their positions in EOC).

Moving EOCExecutableEntityRef elements

- Drag and drop one or more EOCExecutableEntityRef elements from an EOC object to another EOC (cut-and-paste behavior).

Creating new EOCExecutableEntityRef elements

- Drag and drop one or more executable entities (from right to left) to create

new EOCExecutableEntityRef elements inside the target EOC.Validation.

5.14 Java Action

5.14.1 Introduction

The Java Action feature helps you to execute Java based files for AUTOSAR ASW use cases.

5.14.2 Template Creation

A Java Action file with an appropriate header template can be created using the option provided in New File Creation.

1. Go to **File** > **New** > **Other...** > **ISOLAR-A** > **Java Action File**

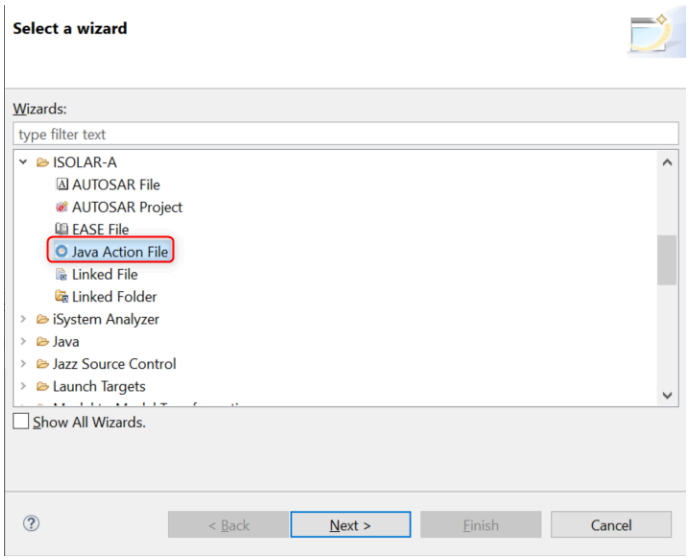

Figure 5.183.Invoking the Java Action File wizard

2. Select the project and provide a file name. (Note: Java Action files should be always created under the *src-java* folder within the project).

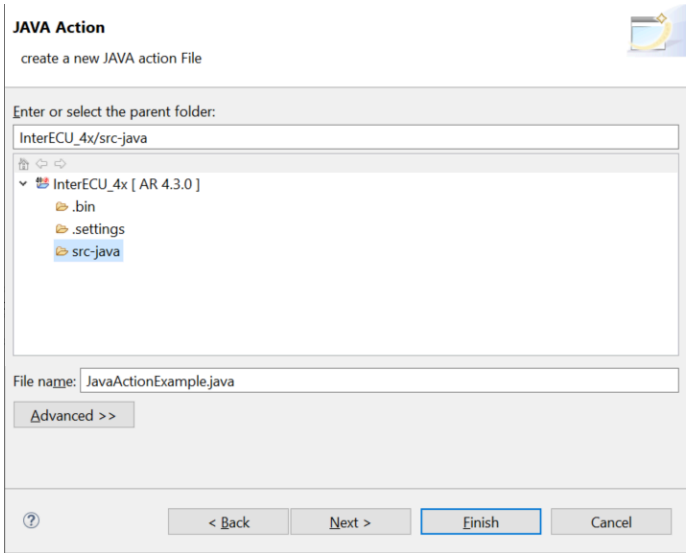

Figure 5.184.Create the Java Action file

3. Provide the template header inputs, the “Name” and “Input Element Type” fields are customizable. The name that you provide here will be used as context menu name, which will be enabled on the provided Input Element Type. When you click **Finish**, the Java file will be generated and the Java Action will be enabled.

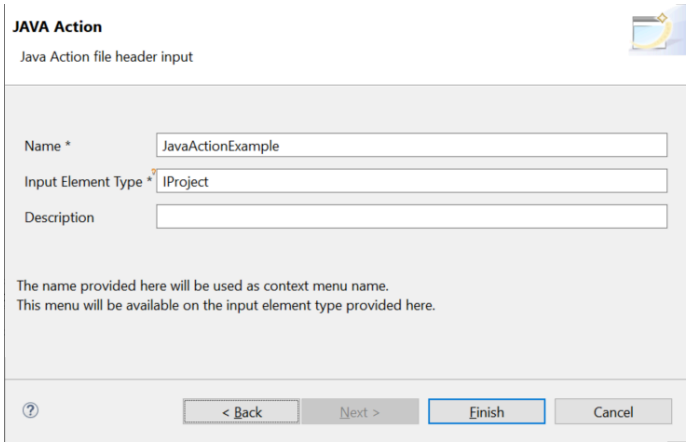

Figure 5.185.Java Action file header input

5.14.3 Enabling Java Action

There are 2 ways of enabling Java Action:

- Java Action can be enabled while creating a Java Action template file.
- Or, if a project already has Java Action files, right-click on the project and select **Enable Java Action**.

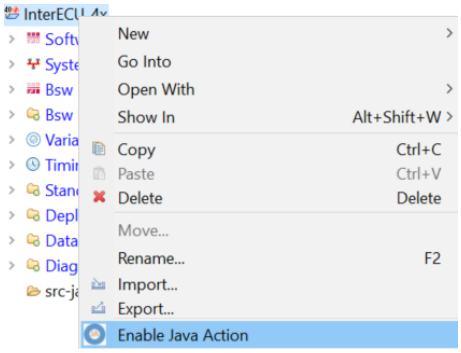

Figure 5.186.Enable Java Action

5.14.4 Executing a Java Action

After enabling Java Action, from the context menu on the element (specified as input type), you can execute the Java Action file.

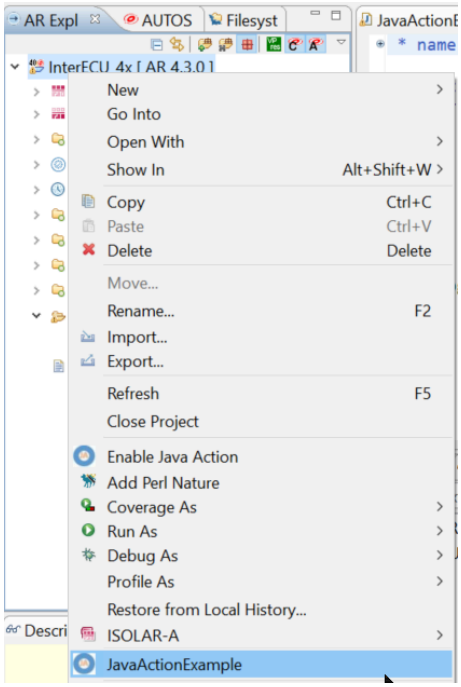

Figure 5.187.Execute Java Action

The Java Action can also be invoked from the Script Library. Just right-click on the intended script and select **Run Script**.

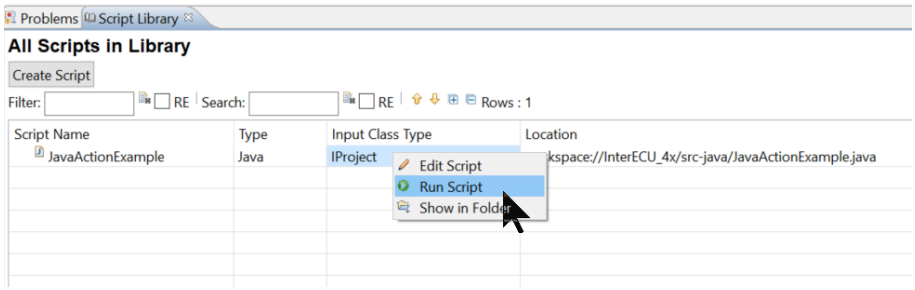

Figure 5.188.Run Script

In the “Script Execution” dialog, select the intended element for which the Java Action should be executed.

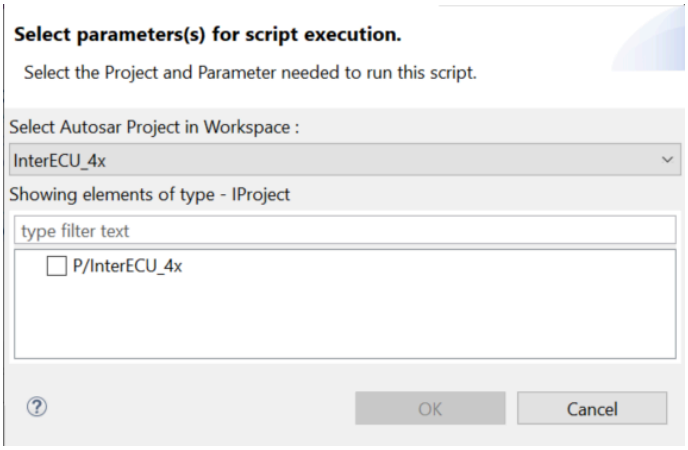

Figure 5.189.Script Execution (select parameters)

5.14.5 Java Modules Explorer

The Java Modules Explorer displays all the available library APIs that can be used for development. Any of the APIs can be dragged and dropped onto the Java file opened in the editor.

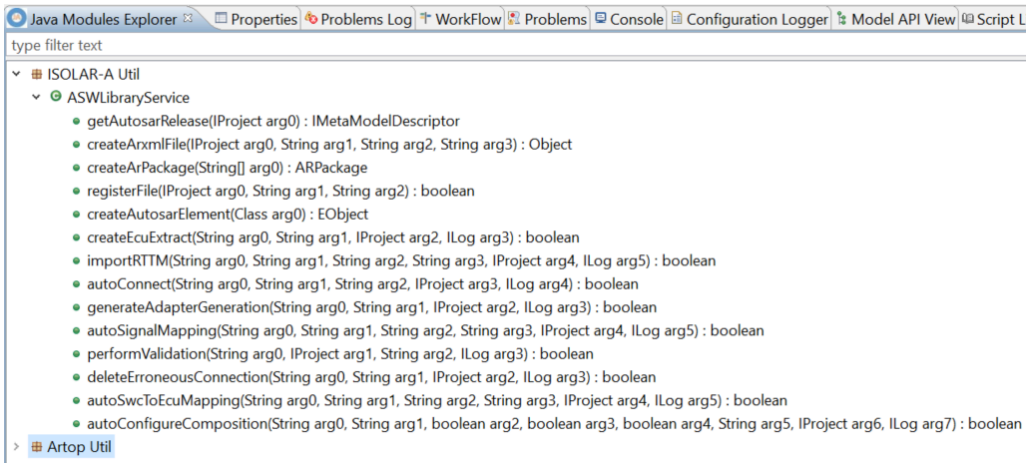

Figure 5.190.Java Modules Explorer

User-defined Java APIs can also be displayed in the Modules Explorer if the Class and APIs adhere to a predefined format. Class should be annotated with:

```
@WrapToJavaAction(getNode = “Custom API”, considerAllMethods = true, getInvokingSignature = “new ClassName()” )
```

APIs that should be considered should also be annotated as follows.

getNode	Used to group under a category in Java Modules Explorer
considerAllMethods	If true, then all the APIs in the class will be displayed in the view, otherwise only APIs annotated with *@WrapToJavaAction* will be considered
getInvokingSignature	This signature will be used when the API is dragged and dropped on the editor. In the example below, on dragging and dropping *getMyName()*, *new MyUtilities().getMyName()* will be pasted in the editor

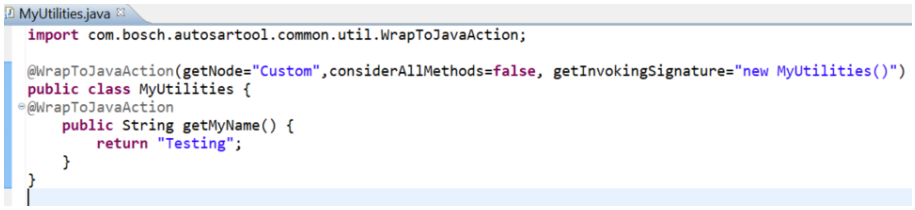

Figure 5.191.Annotate user-defined APIs

5.15 User-Defined Java Validation

5.15.1 Introduction

This feature assists in the writing of user-defined validation in Java for AUTOSAR ASW use cases, and is run as part of AUTOSAR Validation.

5.15.2 Creating a new Java Validation File

The following steps describe how to create a new user-defined Java Validation file with a default validation method.

1. Go to **File** > **New** > **Other...** > **ISOLAR-A** > **Java Action File** (or **File** > **New** > **Java Action File**).

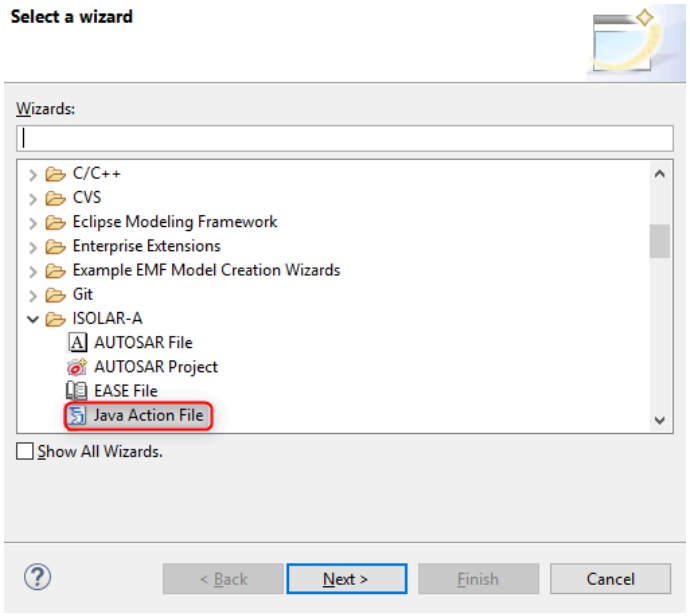

Figure 5.192.Invoking the Java Action File wizard

2. Select the Project and provide a File Name.

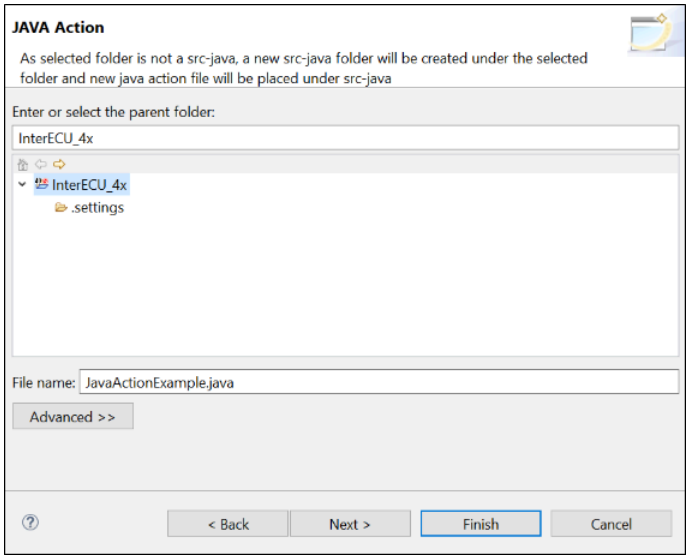

Figure 5.193.Select project amd provide file name

Note: If the *src-java* folder is not present it will be automatically created under the selected container. If the parent folder is selected, and if *src-java* is already present inside it, the new Java Validation file will be created under that existing *src-java* folder.

3. Provide the **Name** and the **Validation Input Type**. The Name provided here will be used as the Validation Id, which will be enabled on the provided input element type.

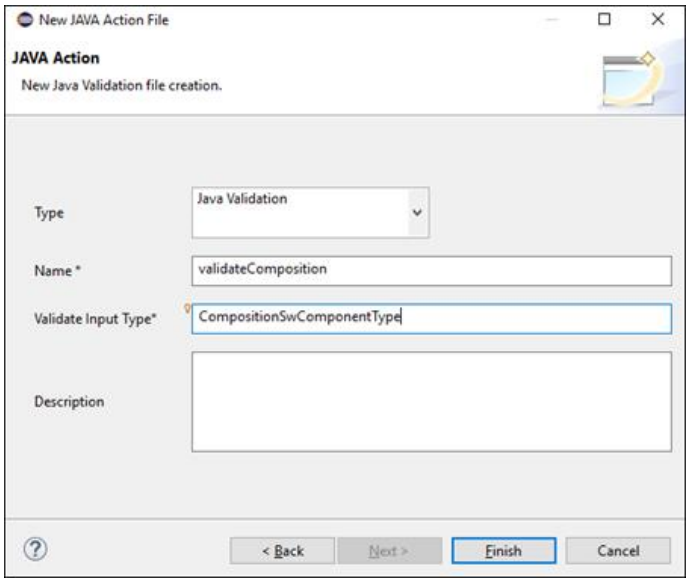

Figure 5.194.Provide name and validation input type

4. When you click **Finish**, the Java file will be generated with the following default template. (Note: Multiple APIs can be added in a single validation class).

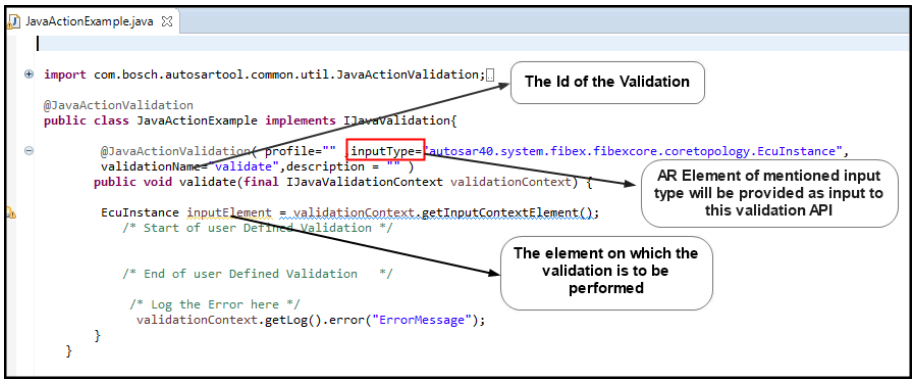

Figure 5.195.Java File created

5.15.3 Executing User-Defined Java Validation

User-defined Java Validation can be executed as part of ISOLAR-A Autosar Validation. All the Java Validation files within the project will be considered while executing the User-defined Validations.

In order to execute Java Validation, you must first have enabled Java Action, (see Section 5.14.3). Editing the model is not recommended in user defined validations.

There are several ways to execute User-defined Java Validations.

Method 1

Go to **Project** › **ISOLAR-A** › **AUTOSAR validations** › **Run User Defined Validations...**

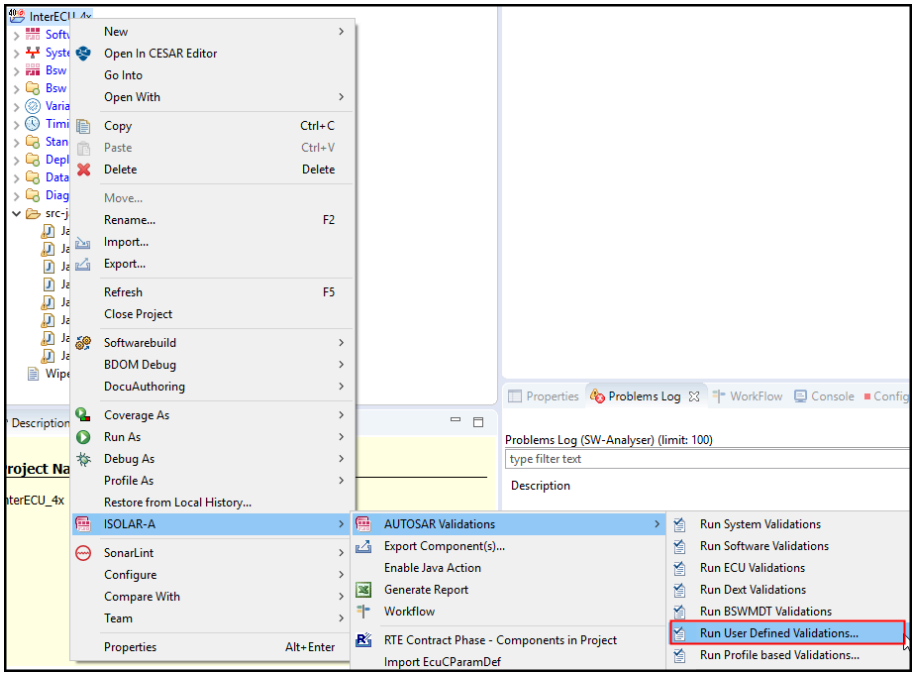

Figure 5.196.Invocation via “Run User Defined Validation”

Method 2

As above, but select Run Profile Based Validations… and then tick the “User Defined Validations” option in the Profile Validation dialog.

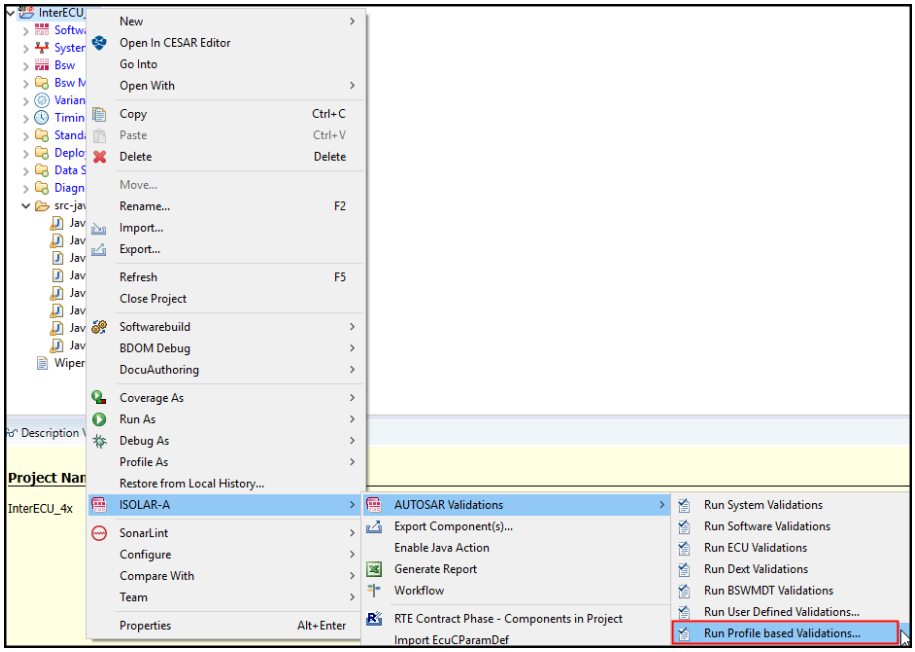

Figure 5.197.Invocation via “Run Profile Based Validations”

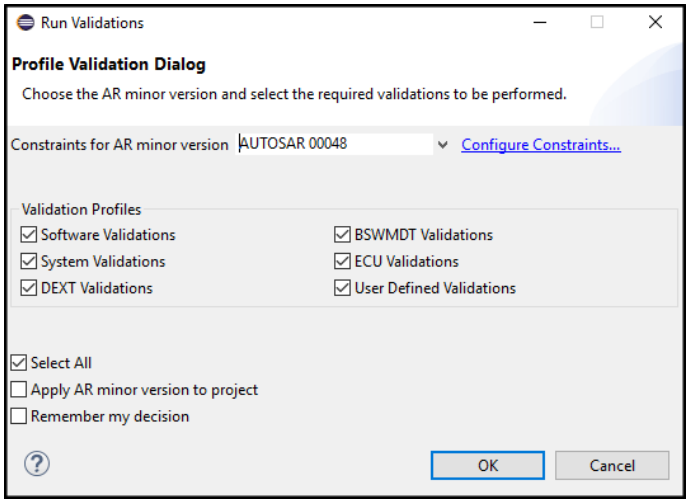

Figure 5.198.Tick the "User Defined Validations" option

Method 3

User-defined Java Validation can also be performed on a specific AR Element. This will run all the User-defined Validation (along with Autosar Validation) which is available for the selected AR Element type.

Just right-click on the AR Element, then select **AUTOSAR validations** › **Run Validations**.

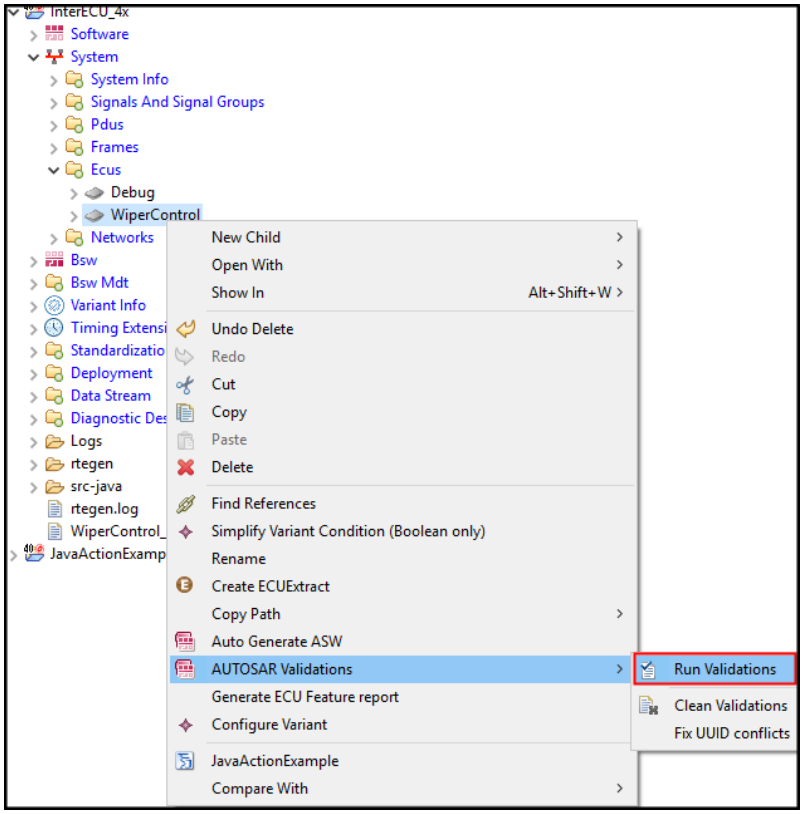

Figure 5.199.Invocation via "Run Validation" as part of AUTOSAR Validation

5.15.4 Validation Errors

Validation errors are logged in the Problems Log with validationName (from Annotation) as the Validation ID.

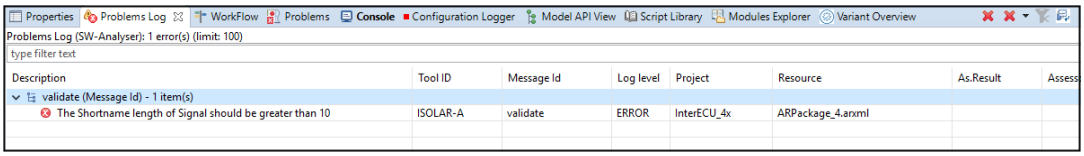

Figure 5.200.Validation Errors in the Problems Log

Double click on an entry in the log to open the Generic Editor for the AR Element on which the error is logged.

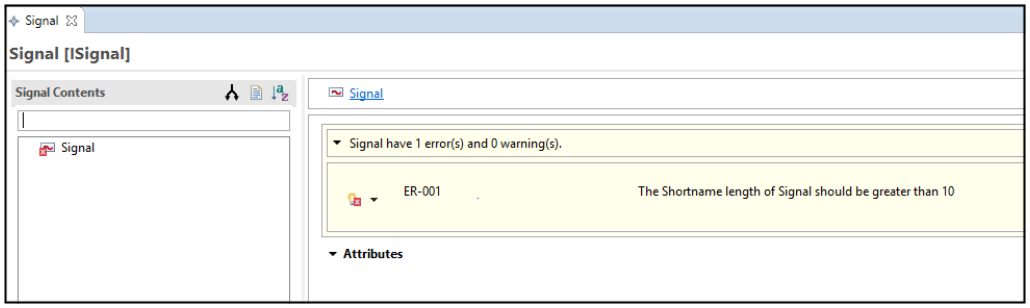

Figure 5.201.Validation Error details

5.16 Transforming Autosar Model Update Scripts to Java Action

5.16.1 Introduction

The Autosar Model Update feature uses ".ext" and Java files. The functionality available in a Java file can be invoked from a Java Action. This section explains how to reuse/invoke these Java files from a Java Action.

5.16.2 Integrating AutosarModelUpdate files to Java Action

The ".ext" files are executable scripts for Autosar Model Update. Only functionalities written in Java files of AutosarModelUpdate can be invoked with Java action.

> **NOTE**
>
> The project on which the Java Action is to be executed must be an Autosar project.

1. To create a new Java Action file, go to **File** > **New** > **Other...** > **ISOLAR-A** > **Java Action File**.

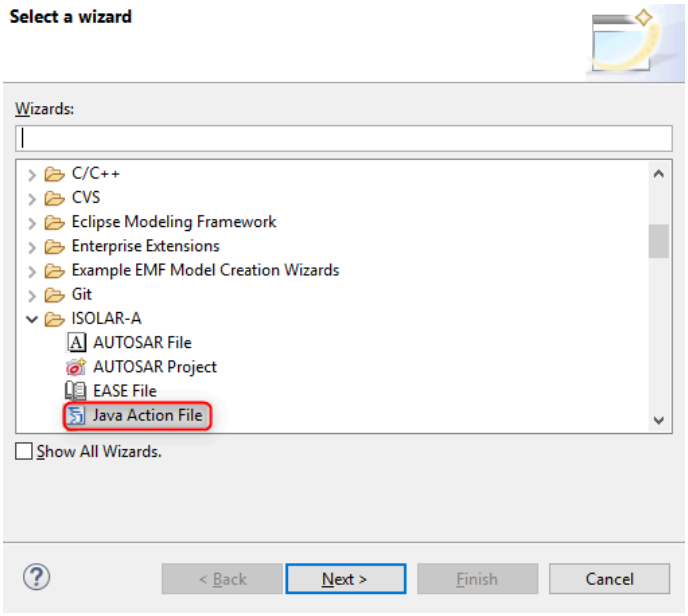

Figure 5.202.Invoking the Java Action File wizard

2. In the “New JAVA Action File” dialog, the Input Element Type should be same as on which the Autosar Model Update script is executed.

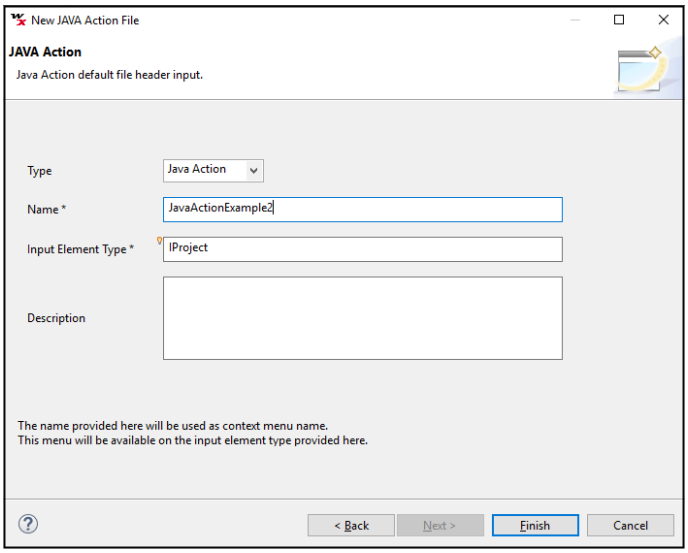

Figure 5.203.Create Java Action File

3. All the Java files should be maintained in the same folder location as the new Java Action file.

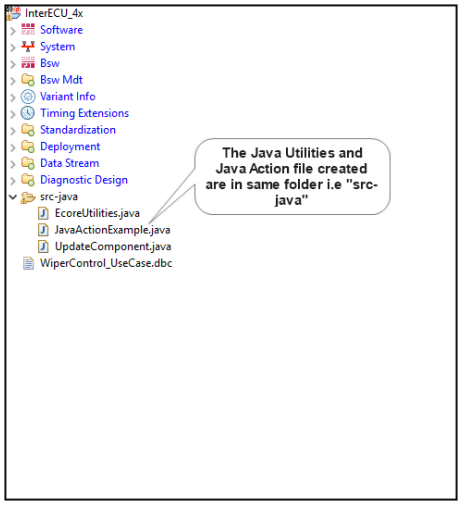

Figure 5.204.Caption

4. Call the APIs from the Model Update script in the newly created Java Action file.
5. Invoke the Java Action on the Input Element in AR Explorer.

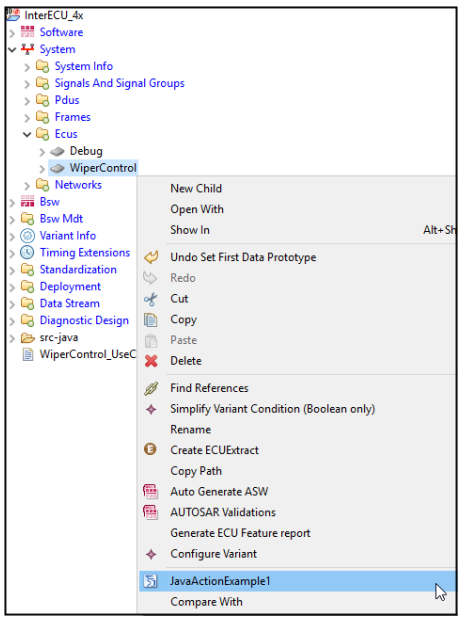

Figure 5.205.Invocation via the AR Explorer menu

5.17 Project Specific Preferences

5.17.1 Introduction

Project Specific Preferences allow you to set preferences and enable rules for a

specific project (i.e. instead of the workspace preference), and then save those configurations in the project's settings folder.

5.17.2 Features that support Project Specific Settings

Project Specific Settings is supported in the following features:

- Adapter Generation
- ASW Auto Generation
- Code-Frame
- Data Dictionary Editor
- ECU Extract
- Auto Configure Composition
- Component Editor
- Connection Editor
- Data Mapping
- Diff Merge
- Importer (ARXML, DBC, Fibex, Odx)

5.17.3 Invocation

Right-click on a Project and select Properties.

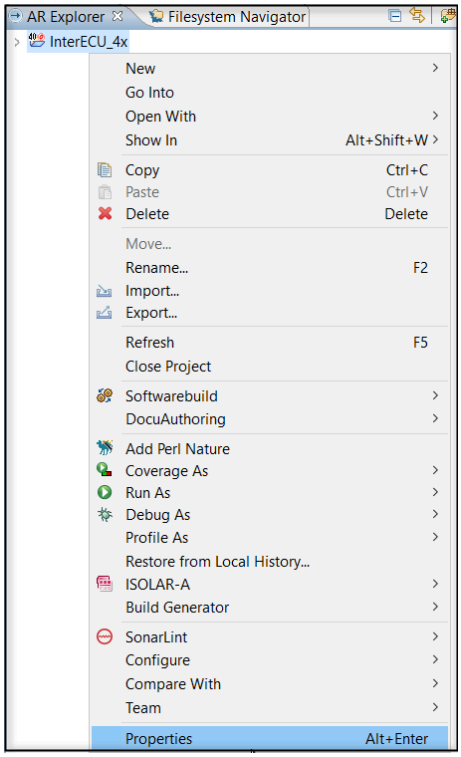

Figure 5.206.Properties

Click on “Configure Project Specific Settings”, then select the project from the “Project Specific Configuration” dialog and click **OK**.

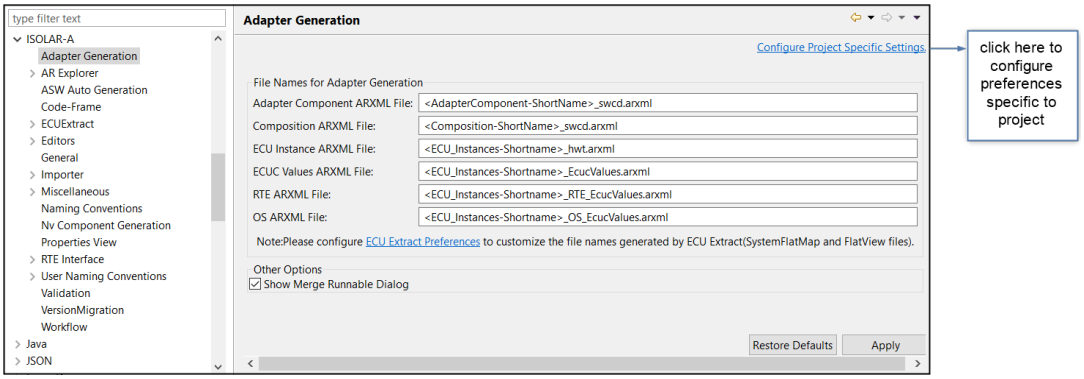

Figure 5.207.Configure Project Specific Settings

5.17.4 Enabling Project Specific Preferences

Settings can be enabled/disabled via the **Enable project specific settings** option.

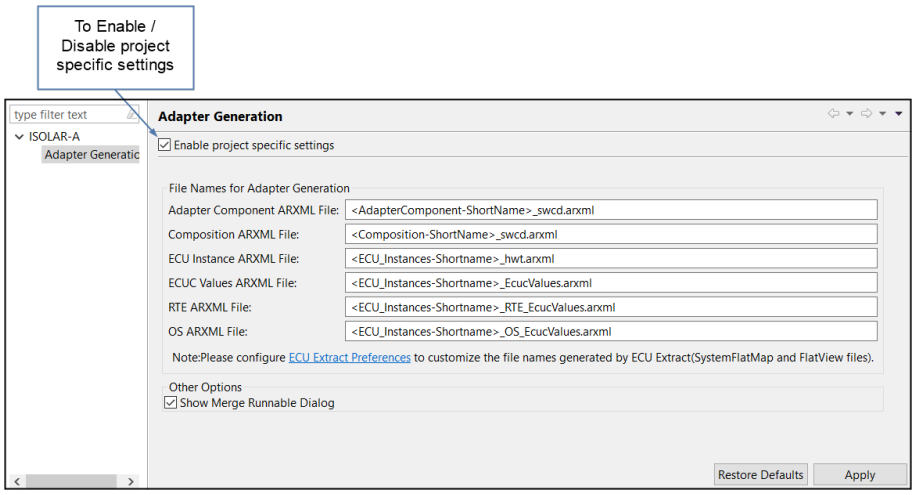

Figure 5.208.Enable Project Specific Settings

5.17.5 Project Specific Settings Path

Once Project Specific Settings has been enabled, you'll see the *.settings* folder when viewing the project in AR Explorer.

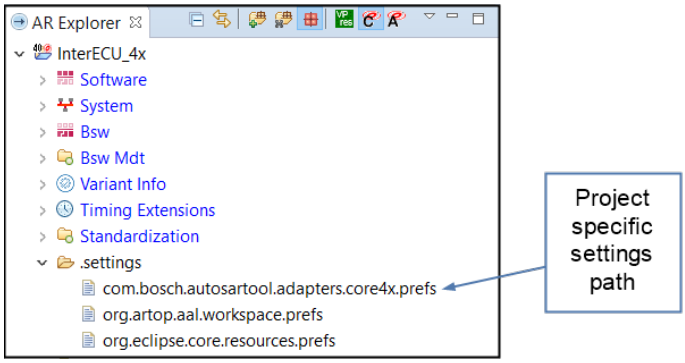

Figure 5.209.The .settings folder

5.17.6 Examples

The following two examples illustrate the use of Project Specific Settings within some of the above listed features.

5.17.7 Example 1: ASW Auto Generation

In this first example, Project Specific Settings has been enabled, and we've set a Root Composition Name of *‹System-Shortname›_TLC-PSNode*.

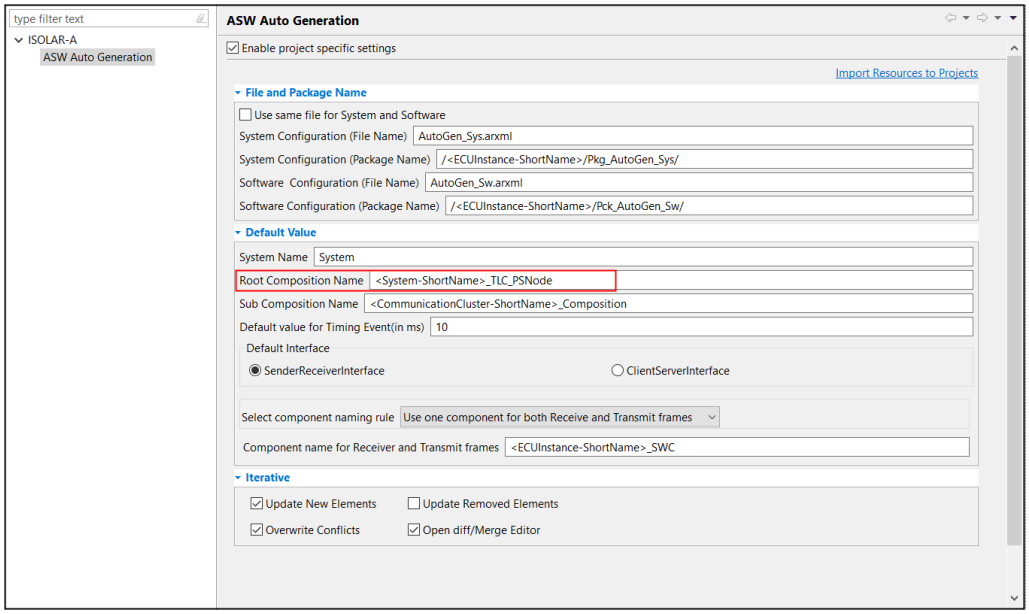

Figure 5.210.Example 1: Set a Root Composition name

The ASW Auto Generation in this project has used our preference setting instead of the workspace preference.

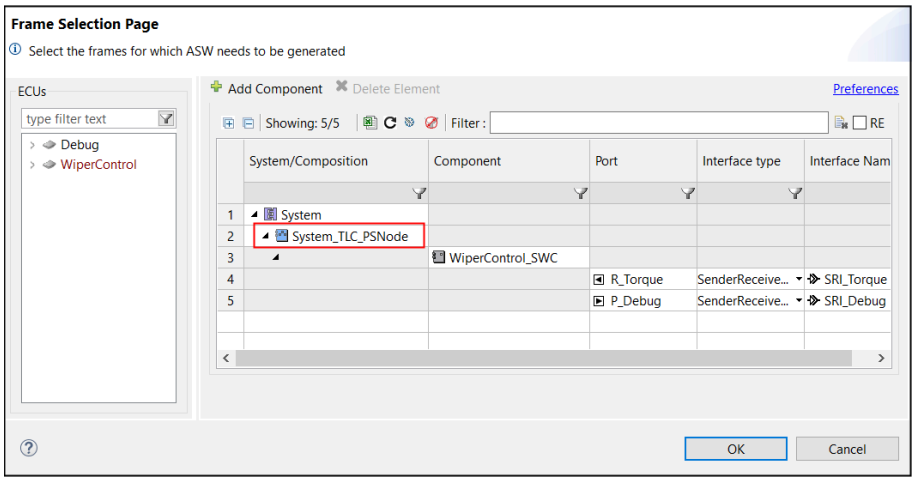

Figure 5.211.Example 1: Root Composition name has been used

5.17.8 Example 2: Auto Configure Composition

In this next example we've enabled the "Perform ECU Extract" option for the Auto Configure Composition feature.

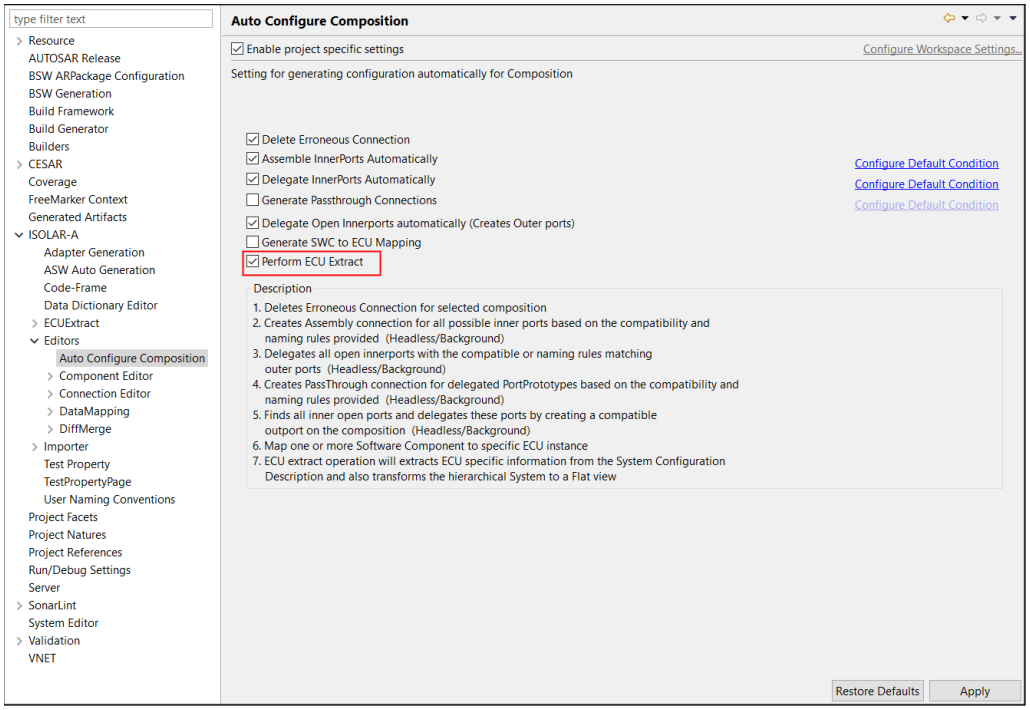

Figure 5.212.Example 2: Enable “Perform ECU Extract”

The Auto Configure Composition dialog will have the settings enabled for Generate ECU Extract.

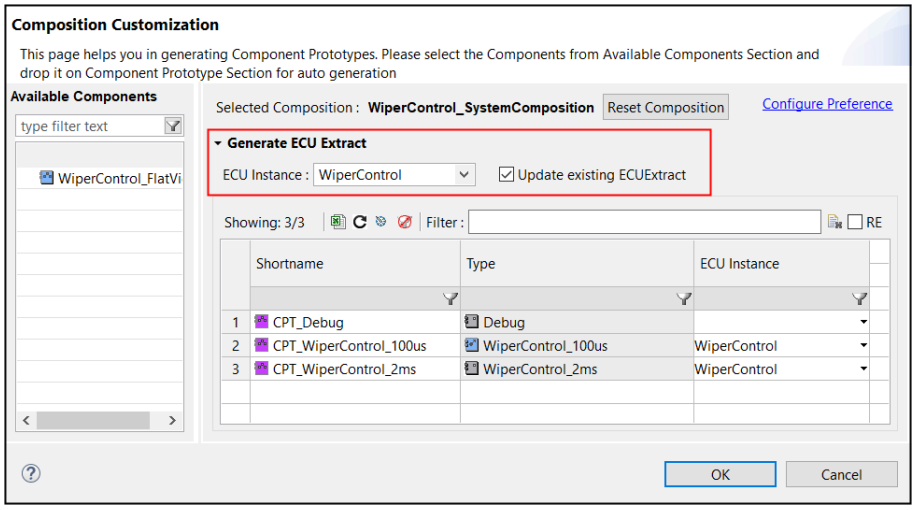

Figure 5.213.Example 2: The “Perform ECU Extract” is auto-enabled.

6 Guided Configuration

6.1 Introduction

Guided Configuration (also known as Guided Workflow) provides you with a configuration path based on the workflow of a specific use case.

Once you've selected the use case, all the required workflow steps are displayed. If you execute them in the suggested order it will help you to complete the configuration more quickly.

Each of the steps listed in the workflow might display a dialog and ask you to provide input, or a step might open relevant editors/wizards through which the specific step can be completed. Some steps might open a sub-module with additional steps.

You can optionally create your own Guided Workflows, and they can be exported and imported if required.

6.2 Invocation

Guided Workflow can be invoked in one of two ways.

1. Go to **Views** and select **Workflow**.

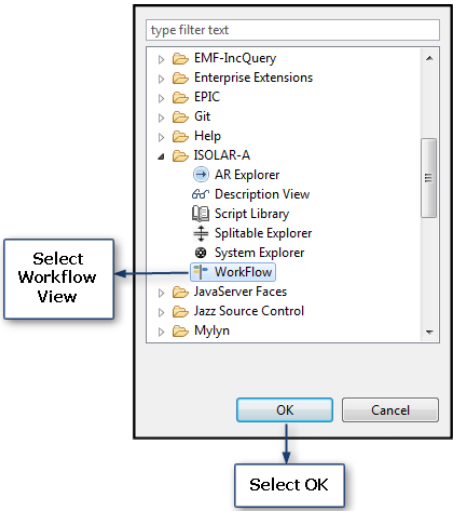

Figure 6.1.Invoking Workflow from Views

2. Or, you can right-click on the project you want to configure, select **ISOLAR-A**, and then select **Workflow**.

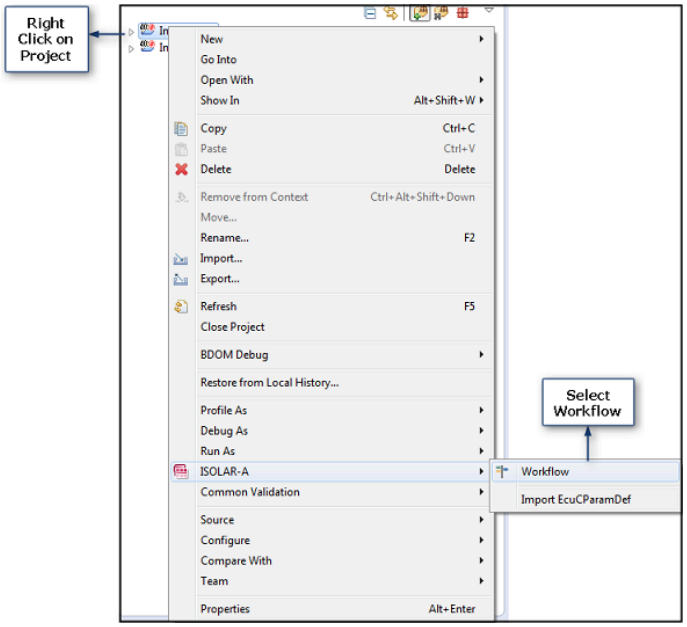

Figure 6.2.Invoking Workflow via Project selection

6.3 Starting the Workflow

There are two ways to start the Workflow.

Method 1

You can click the **Start** icon at the top of the Workflow.

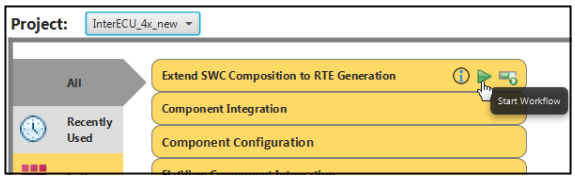

Figure 6.3.Starting Workflow with the Start button

If there are no workflows available for the use case, a new one will be created.

If a workflow already exists for the use case, the following message will appear, giving you the option to either load the existing workflow (Yes), or start a new one (No).

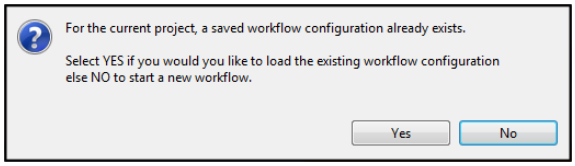

Figure 6.4.Workflow already exists

Method 2

If you invoke workflow from the use case progress bar, the workflow is started for an already existing workflow.

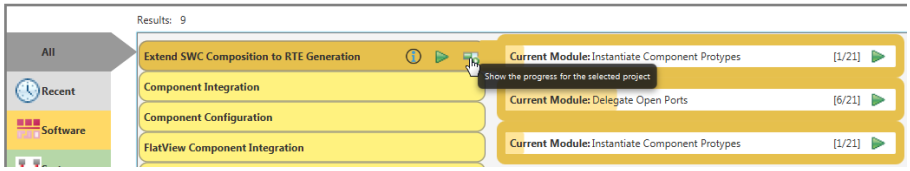

Figure 6.5.Starting Workflow from the Progress bar

7 AUTOSAR Adaptive Support

7.1 Introduction

ISOLAR-A provides basic support of Adaptive Elements. Two actions are provided in the AR Explorer:

- **CP:** This action will show only the Classic Platform elements (enabled by default)
- **AP:** This action will show only the Adaptive Platform elements

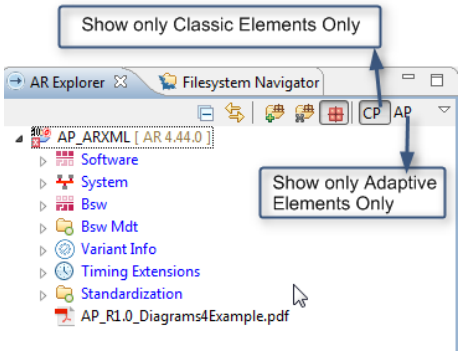

Figure 7.1.CP and AP Actions

The Classic Platform (CP) view allows you to see just the AUTOSAR Classic elements.

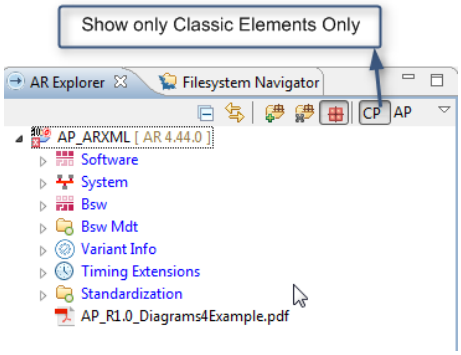

Figure 7.2.Show only Classic elements

The Adaptive Platform (AP) view allows you to see only the AUTOSAR Adaptive platform elements.

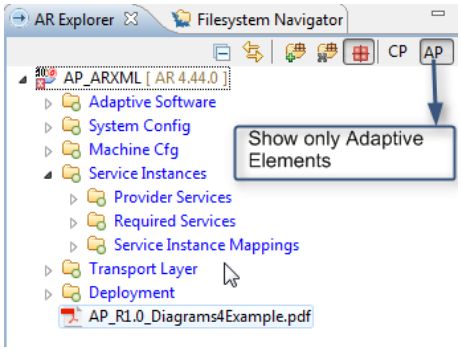

Figure 7.3.Show Adaptive elements

Double click on any element or right click and select **Open With** to open it in the Generic Editor.

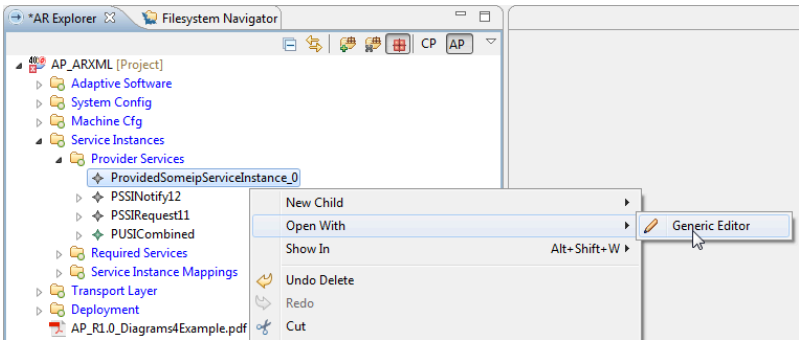

Figure 7.4.Open an element in the Generic Editor

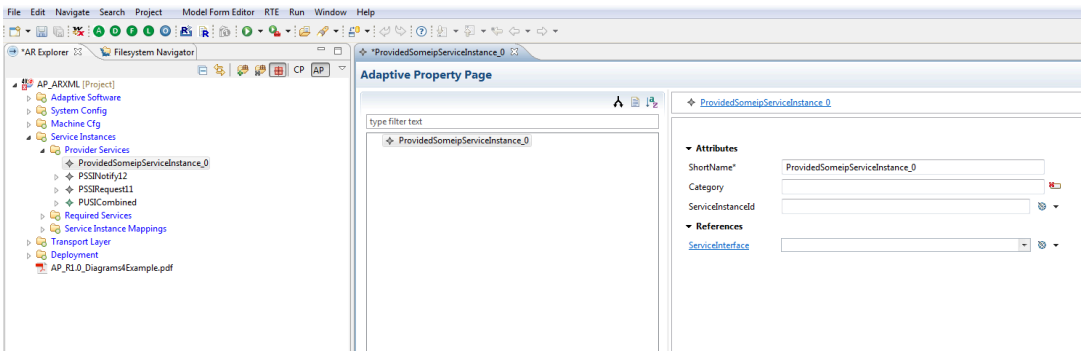

Figure 7.5.Viewing an element in the Generic Editor

8 Configuration Logger

8.1 Introduction

The Configuration Logger logs all the configuration changes that have been made in the workspace, together with details of the UI actions that led to each change. The following types of changes are logged:

- Addition/Deletion of elements
- Modification of an element's properties
- Addition/Deletion of files
- Addition/Deletion of a project

8.2 Invocation

The Configuration Logger should be enabled by default, but if it is not visible, it can be accessed via **Window** > **Show View** > **Other...** and then selecting **ISOLAR-A** > **Configuration Logger**.

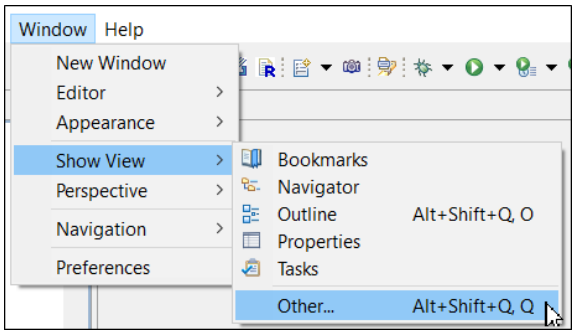

Figure 8.1.Show View > Other

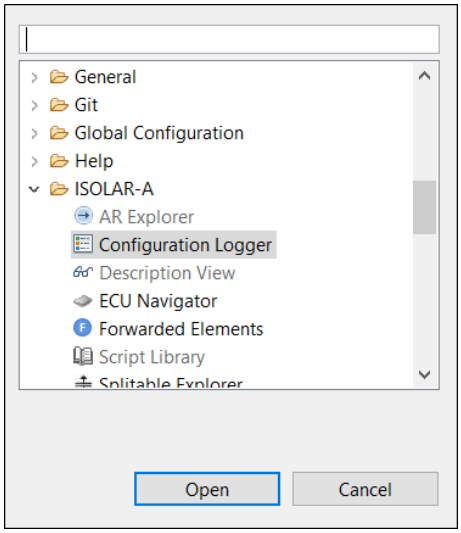

Figure 8.2.Configuration Logger

8.3 Configuration Logger UI

The Configuration Logger shows all the changes that have taken place in the workspace.

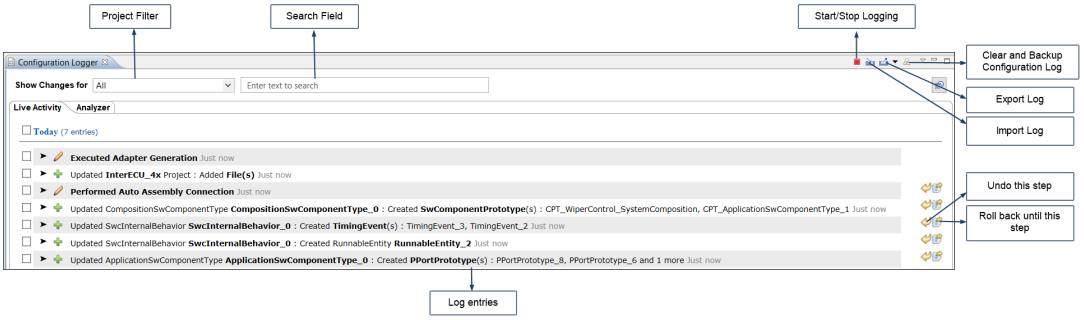

Figure 8.3.Configuration Logger overview

Here we can see the UI Actions that led to the configuration change.

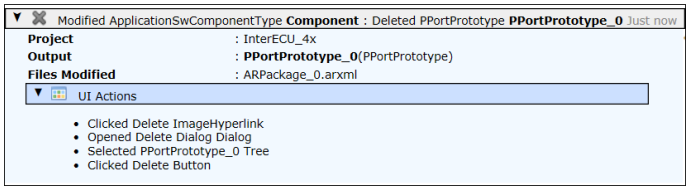

Figure 8.4.UI actions that lead to the configuration change

And here we can see the feature and preference settings for certain bulk operations.

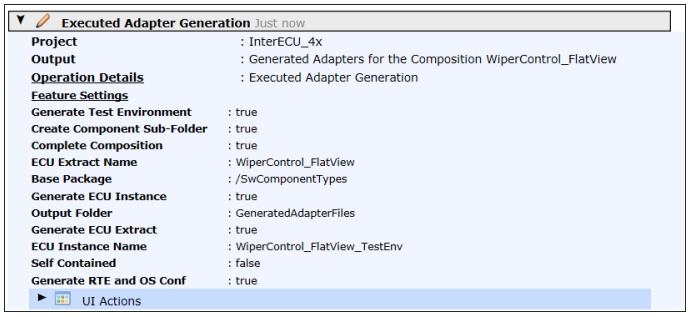

Figure 8.5.Feature and preference settings for bulk actions

The entries related to the workflow will be indicated with the workflow icon.

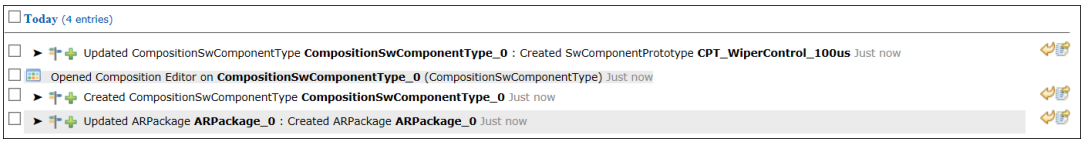

Figure 8.6.Icon indicates entries related to the workflow

8.4 Configuration Logger Options

Project Filter: By default, changes related to all projects are shown, but you can use the Project Filter to see only the changes for a specific project.

Search Field: Allows you to search the log entries.

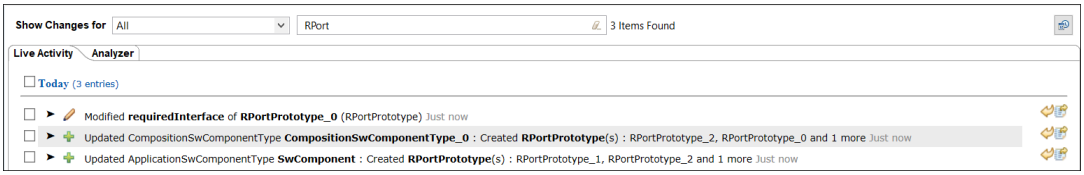

Figure 8.7.Search log entries

Start/Stop Logging: Allows you to Start/Stop the logging.

Import Log: Use this option to import the log and view the imported log file contents.

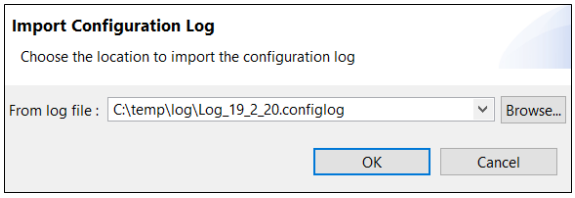

Figure 8.8.Import Configuration Log

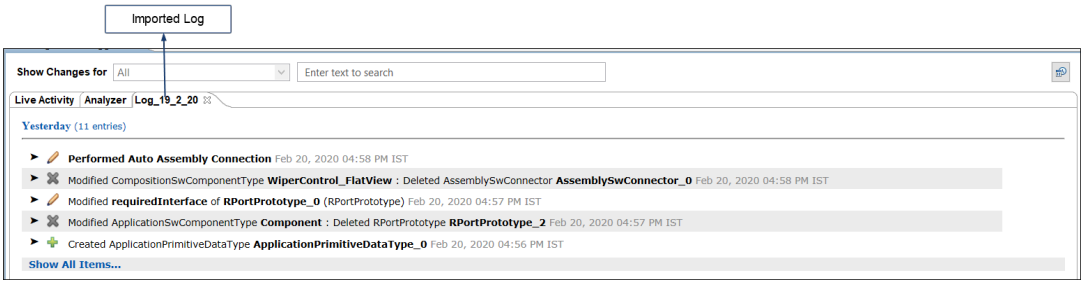

Figure 8.9.View the imported Log file contents

Export Log: Use this option to export the log to a specified location.

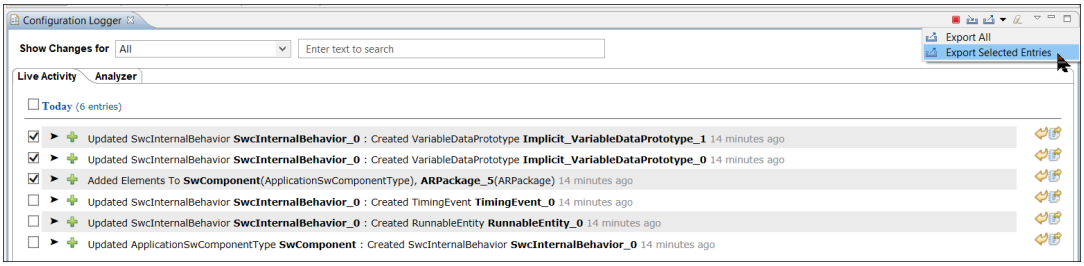

Figure 8.10.Export log options

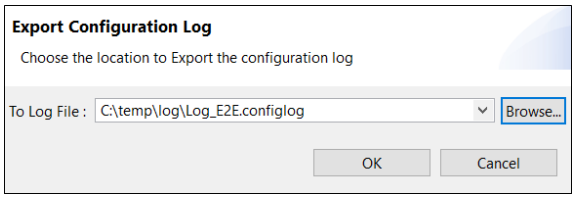

Figure 8.11.Export Configuration log

Clear and Backup Configuration Log: Allows you to clear the log and optionally take a backup of the configuration log file.

Undo: The undo icon is displayed next to any action that can be undone.

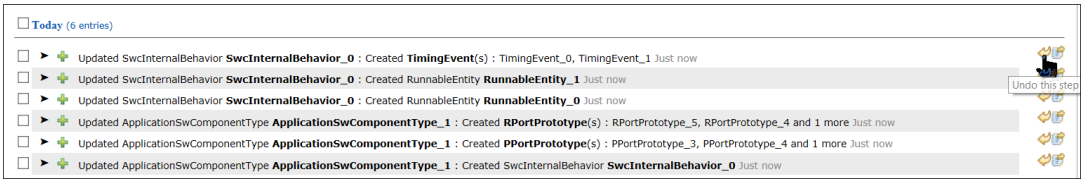

Figure 8.12.Icon to indicate an action can be undone

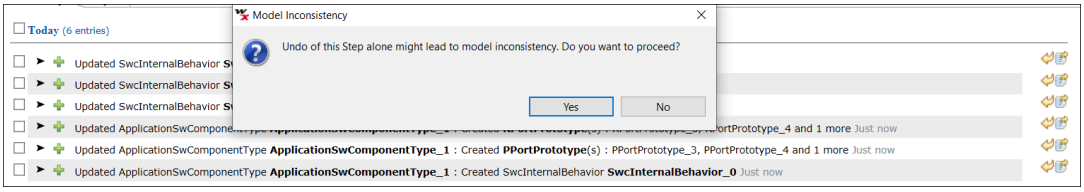

Figure 8.13.Undo warning

When the changes related to the selected action have been undone, the undo button is then disabled.

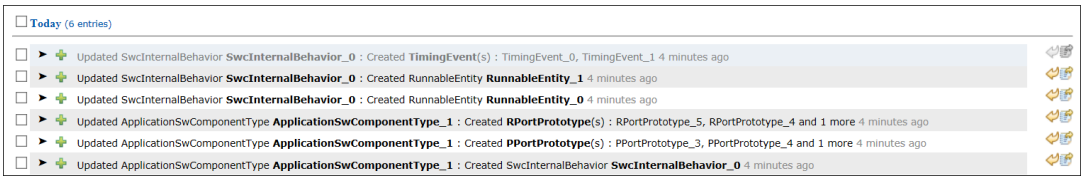

Figure 8.14.Undo completed (button now disabled)

Roll back: The roll back icon is displayed next to any action that can be rolled back (note that not all actions can be rolled back). All the eligible actions up to the selected action are rolled back, and once the operation is completed, the roll back icon that was displayed next to each rolled back action will be disabled.

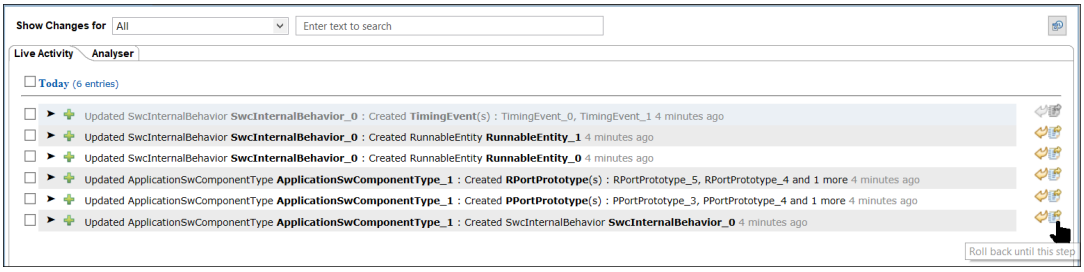

Figure 8.15."Roll back until this step"

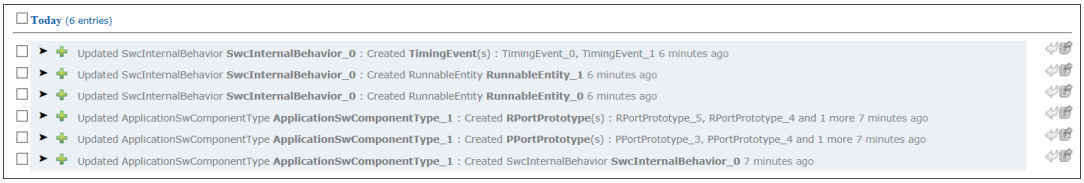

Figure 8.16.Roll back completed

8.5 Analyzer

The Analyzer tab assists with the analysis of the log by providing a logical grouping of the log entries based on changes to the packageable elements. All the entries related to a particular packageable element will be visible under the same group.

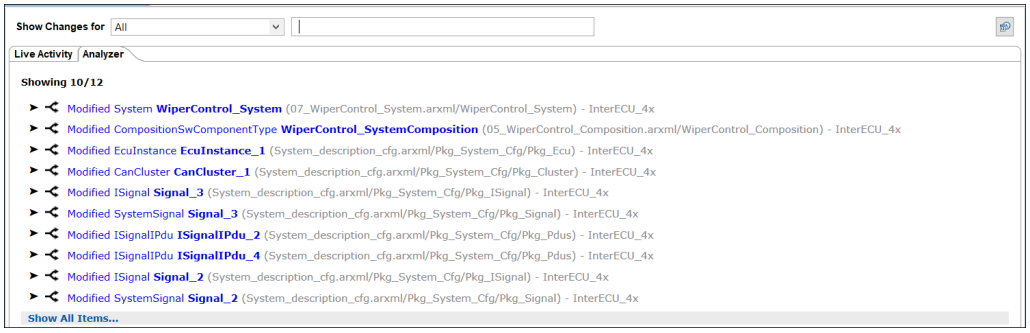

Figure 8.17.Analyzer tab

The Roll back feature in the Analyzer rolls back the changes that are spanned across the groups, so a dialog indicating all the changes that would be rolled back as a result of roll back on the selected step would be shown.

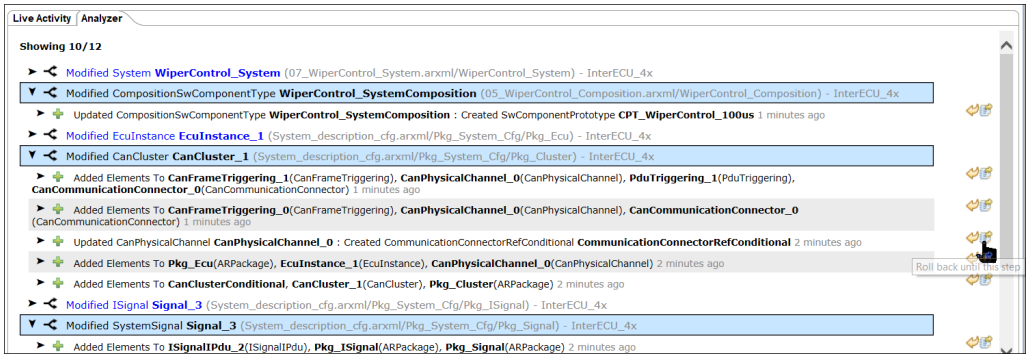

Figure 8.18.Roll back in the Analyzer view

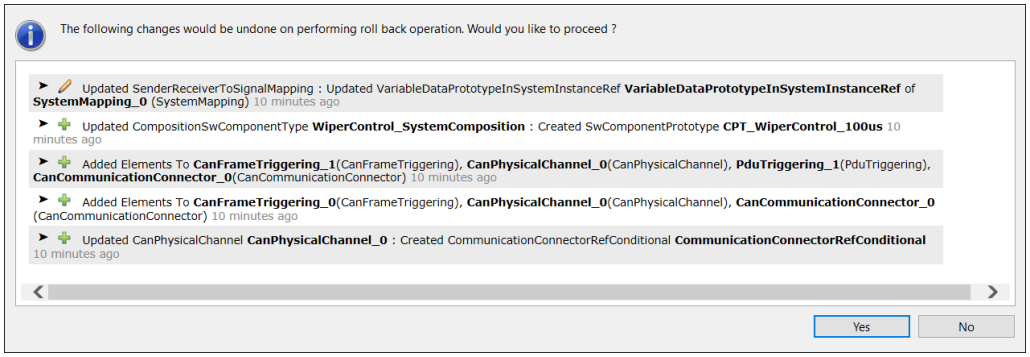

Figure 8.19.Changes that will be rolled back

After performing the roll back, the entries are disabled.

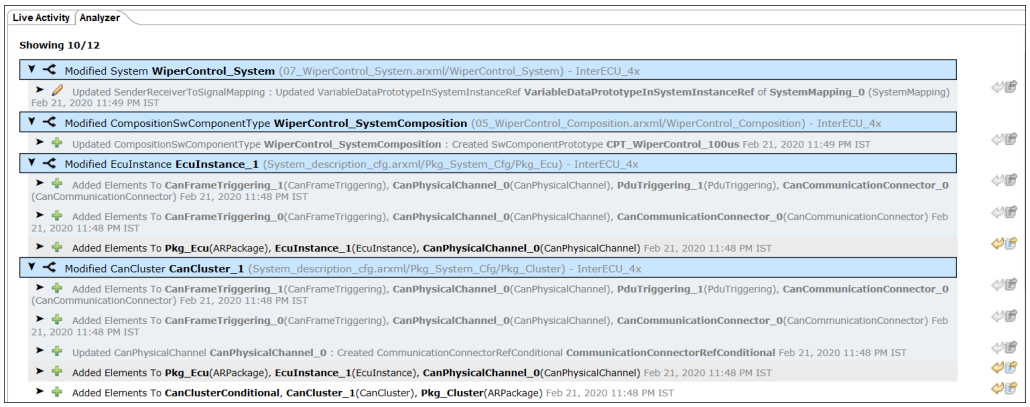

Figure 8.20.Roll back completed (entries disabled)

8.6 Backup Settings

By default, the Configuration Logger displays only 10,000 entries. When this limit is reached, the oldest entries are moved to a backup file with a name derived from the name of the workspace. This backup is stored by default in your local folder:

AppData\SOLAR\ToolName\WorkspaceName

You can configure both the entry limit (100 – 10,000) and the backup location.

Figure 8.21.Accessing Backup Settings

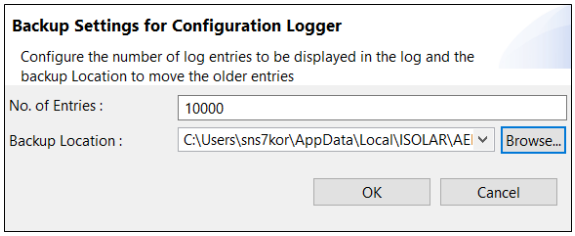

Figure 8.22.Backup Settings dialog

9 Server Mode Interactive

9.1 Introduction

The Server Mode Interactive application is a command line application that allows you to execute ISOLAR-A CLI features in an interactive mode, making the use of the command line interface similar to the ISOLAR GUI application. The interactive application allows you to do the following:

- One-time project load in the workspace.
- Multiple projects can be maintained in the workspace.
- CLI feature execution on a selective/active project in the workspace.
- Support for both human and machine interaction.

9.2 Invocation and Execution

The Server Mode Interactive application is invoked via a cmd file (*ISOLAR-Interactive.cmd*) which you'll find in the ISOLAR product folder. There are several ways to execute the file:

User Mode: Double click on the .cmd file (or run it from a Windows command prompt).

External Mode: Open the product location in a Windows command prompt, type *ISOLAR-Interactive.cmd* followed by the *‹CLI command›* to be executed.

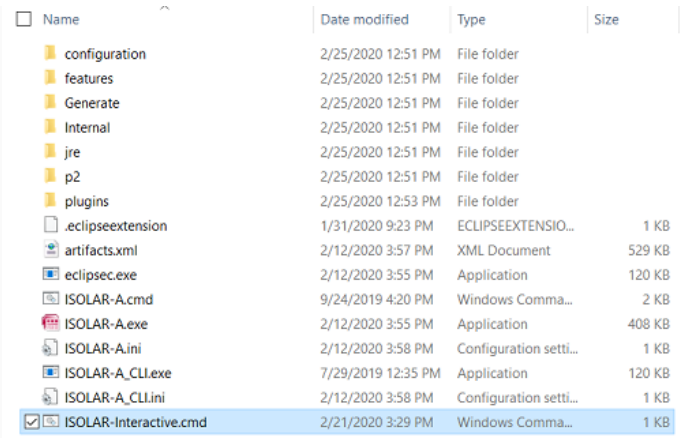

Figure 9.1.ISOLAR-Interactve.cmd file

9.3 Mode Types

As outlined above, based on the usage, the interactive application has two different modes:

- User Mode
- External Mode

9.3.1 User Mode

In user mode, the *ISOLAR-Interactive.cmd* file gets executed only once, as shown here.

Figure 9.2.Invocation of interactive in internal mode

ISOLAR-Interactive.cmd starts the interactive application, and the application waits for you to enter the CLI command. It then completes its execution and waits for the next CLI command, continuing to do so until you provide the *terminate* command to close the session. Third party applications cannot run/execute CLI commands in this mode because it is designed for human interaction only.

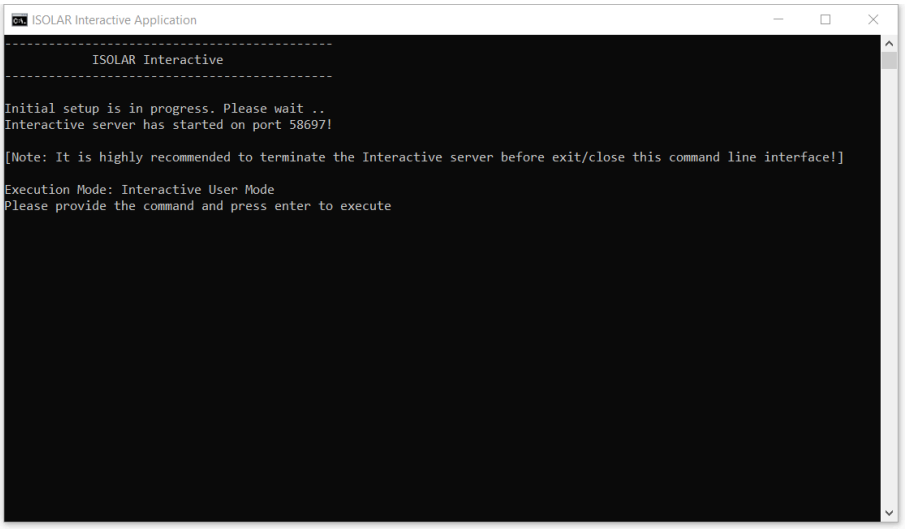

Figure 9.3.Interactive - User mode

9.3.2 External Mode

The *ISOLAR-Interactive.cmd* file needs to be invoked in the Windows command prompt whenever a CLI command is provided to execute in interactive mode. When a command is given to the Interactive application in external mode, it executes the command, displays the execution status and leaves the control to the command prompt where *ISOLAR-Interactive.cmd* can be invoked with the next CLI command. This mode supports both human and machine interaction, which means that third party applications can run/execute CLI commands of ISOLAR using this External Mode of Interactive application.

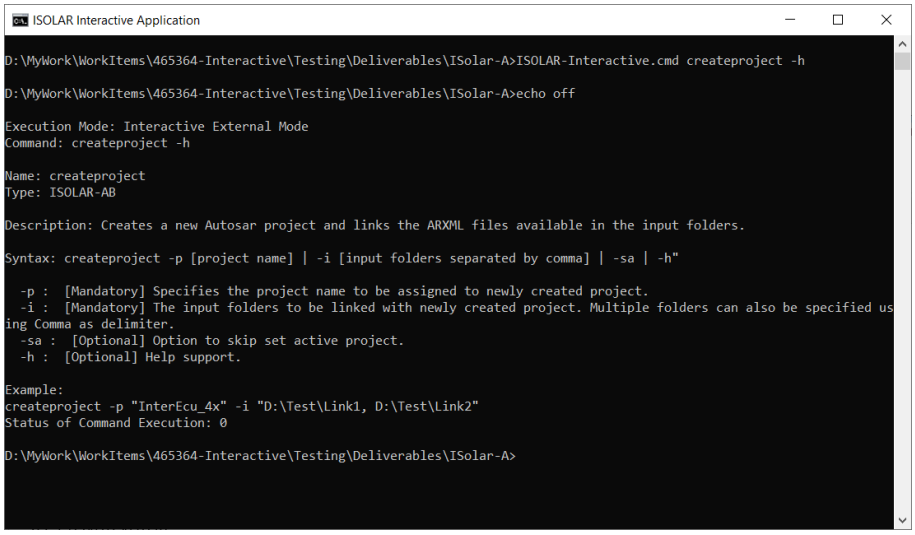

Figure 9.4.Interactive - External mode

10 ASW Auto Generation

10.1 Introduction

The Auto Generate ASW feature allows you to generate, with just a single click, an ASW configuration from Communication Matrix information.

The following use cases are supported with this feature.

- Generating ASW for the first time for a Communication Matrix.
- Incorporating newly added Communication Matrix information into an existing ASW configuration.

10.2 Invocation

The Auto Generate ASW feature can be opened in the AR Explorer by right-clicking on **ECUs** and selecting **Auto Generate ASW**.

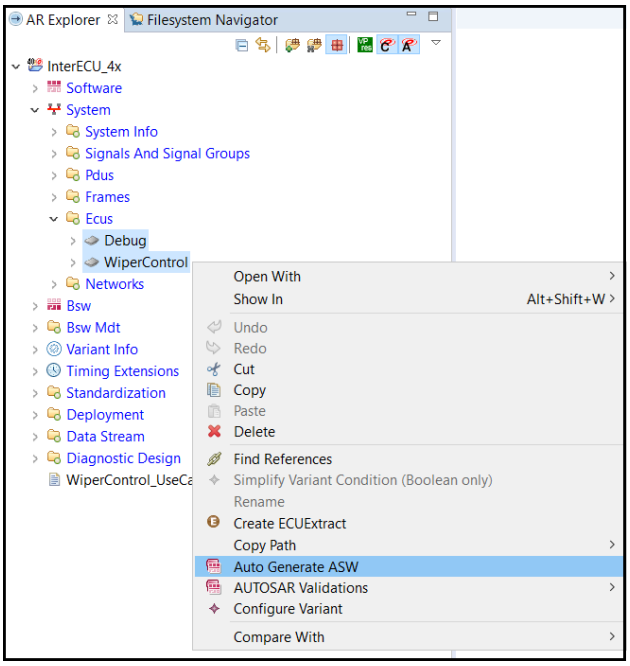

Figure 10.1.Invoking ASW Auto Generation

10.3 Frame Selection

The “Frame Selection” page displays the frames connected to the ECU(s). The default grouping of the frames is based on their direction.

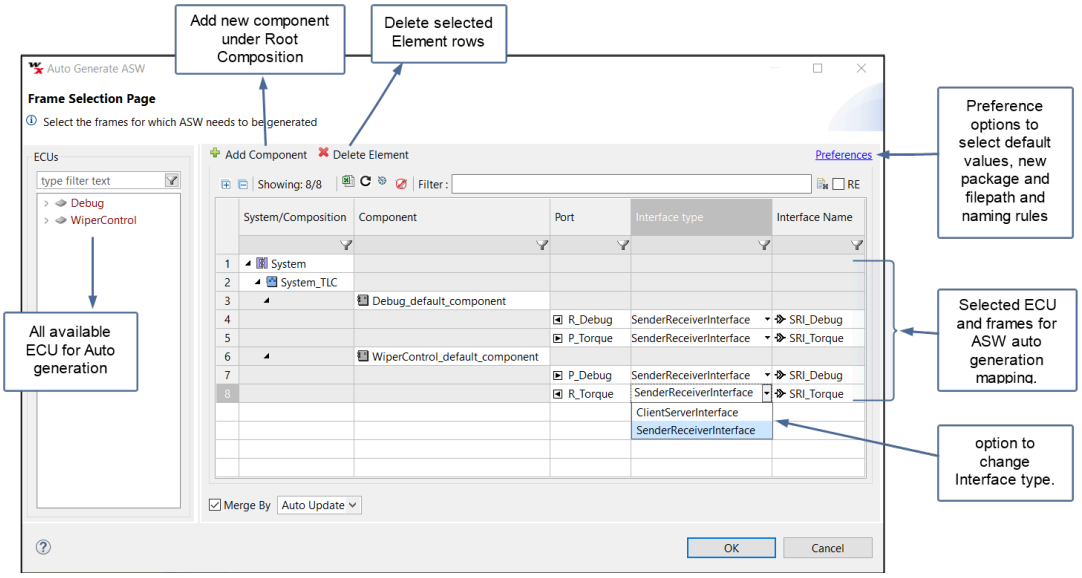

Figure 10.2.Frame Selection

10.4 Preferences

The File and Package preferences section allows you to define the package structure of the auto-generated content and the file names. It also provides the option of generating both System and Software configuration as a single file or as separate files.

11 Adapter Generation

11.1 Introduction

Adapter Generation is a concept that involves automatic creation of Adapter Components, Compositions and System information, which can be used for testing of the developed components by a successful generation of RTE. The following use cases are supported with this feature.

- Creating adapter(s) for Software Components
- Creating adapter(s) for Compositions
  - Close all open ports in the selected composition
  - Creating adapter(s) for the composition
- SDK Adapter generation

11.2 Invocation

There are 2 ways in which Adapter Generation can be invoked.

- On a Software Component
- On a Software Composition

When you right click on a Software Component or Composition in the AR Explorer, the following pop-up menu appears.

Figure 11.1.Invoking Adapter Generation

To invoke the feature, select the highlighted menu item.

11.3 Adapter Generation - Component

If the selected item is a Software Component (Atomic Software Component or Parameter Software Component) the following wizard will appear.

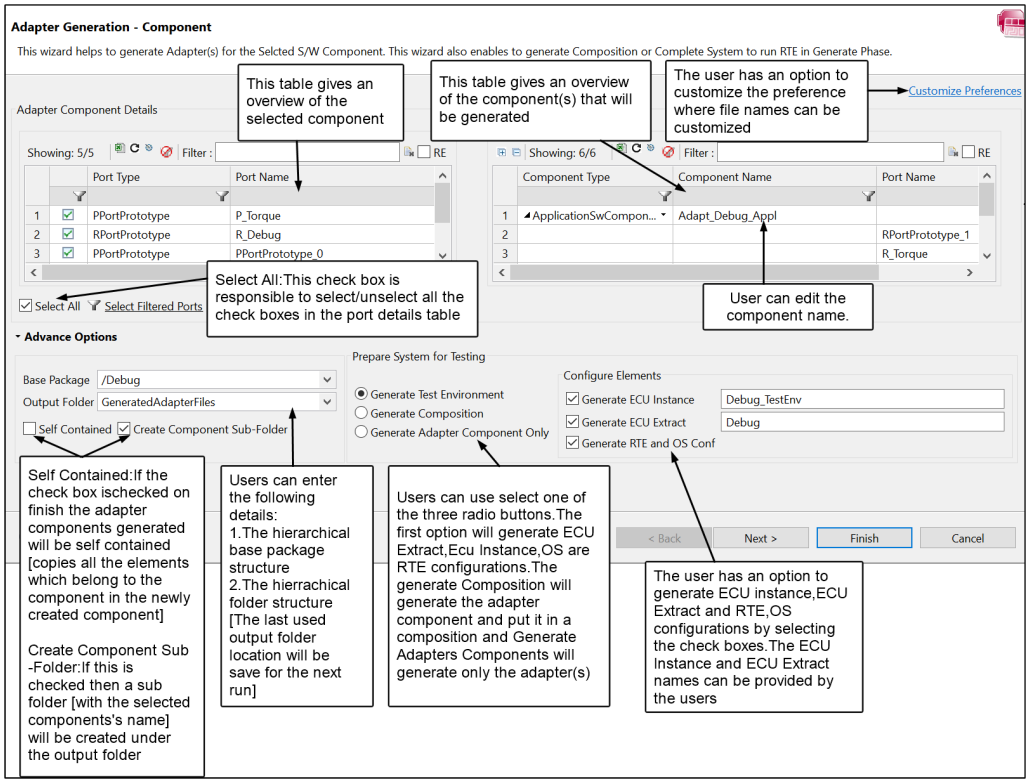

Figure 11.2.Adapter Generation - Component

Enter the following details:

- **Base Package:** The hierarchical Base Package structure under which all the adapter components need to be generated.
- **Output Folder:** The hierarchical folder structure, under which the adapter generation arxml files need to be generated.

The wizard is automatically populated with default values, and the most recently used hierarchical folder structure is saved and re-displayed as the output folder on the subsequent invocation (you can change the path if you wish).

This first page of the wizard provides you with several options that can be selected when generating adapters for the selected Software Component, as described in the sub-sections below.

11.3.1 Generate ECU Instance, ECU Extract, OS and RTE Configurations

When the following options are selected, you will be required to provide some additional information:

- **Generate ECU instance**. You will need to provide the ECU Instance name.
- **Generate ECU Extract.** You will need to provide the ECU Extract name.
- **Generate RTE and OS Configurations.** You can optionally provide the ECU Instance name and ECU Extract name.

11.3.2 Generate Composition

When the “Generate Composition” option is selected, you can optionally provide a composition name. When you click **Finish**, the adapter component(s) will be created and placed in a composition.

11.3.3 Generate Adapter Components Only

When the “Generate Adapter Components Only” option is selected, only the adapter component(s) will be created.

11.3.4 Adapter Generation Preferences

As shown below, various defaults can be set in Preferences for the Adapter Generation, including the file names to be used. For “Adapter Component ARXML File”, the <AdapterComponent-ShortName> keyword is mandatory to create multiple component files.

These defaults will be applied at the workspace level, but you can use the **Configure Project Specific Settings** link if you want to set project-specific defaults.

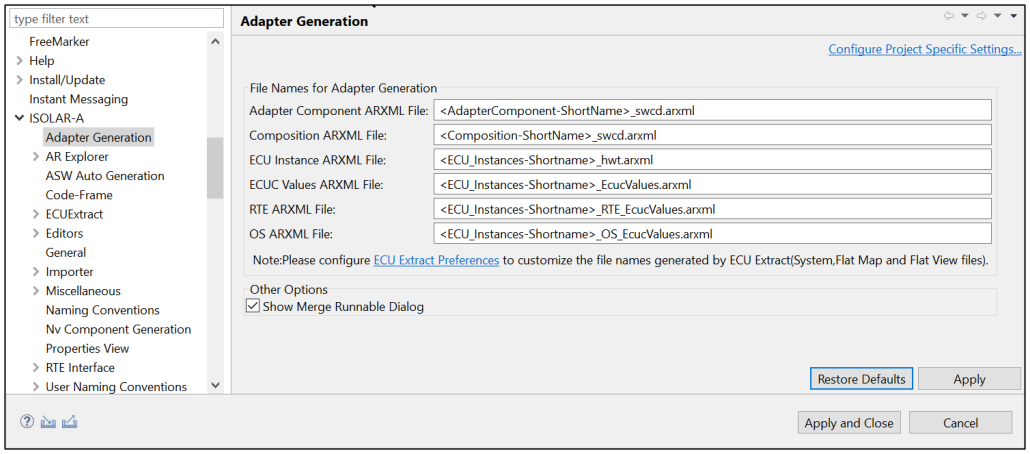

Figure 11.3.Adapter Generation - Preferences

Click the **ECU Extract Preferences** to set preferences for the ECU extract.

The second page of the wizard (“Runnable Information”) gives you an overview of the Runnable Entities that will be created as part of the adapter generation.

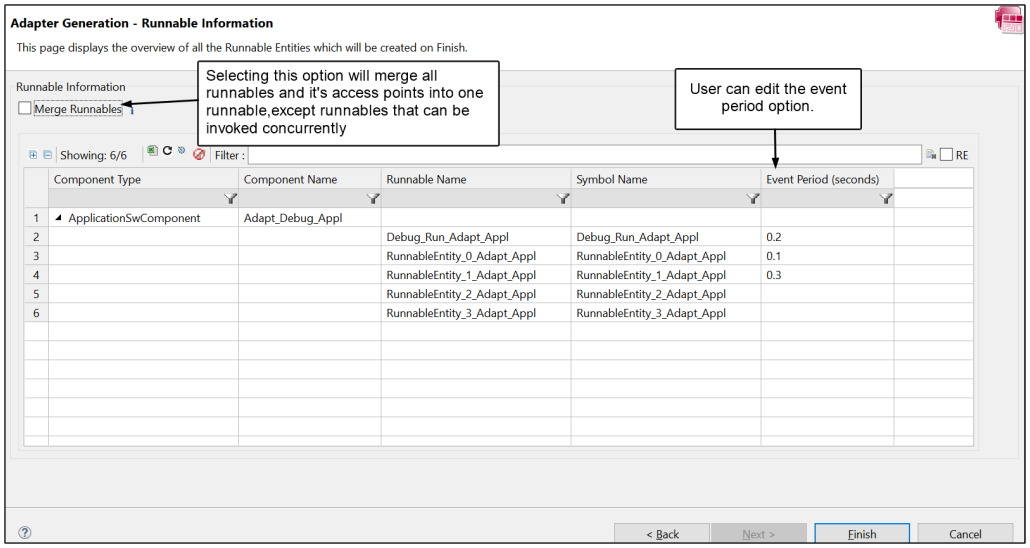

Figure 11.4.Adapter Generation - Runnable Information

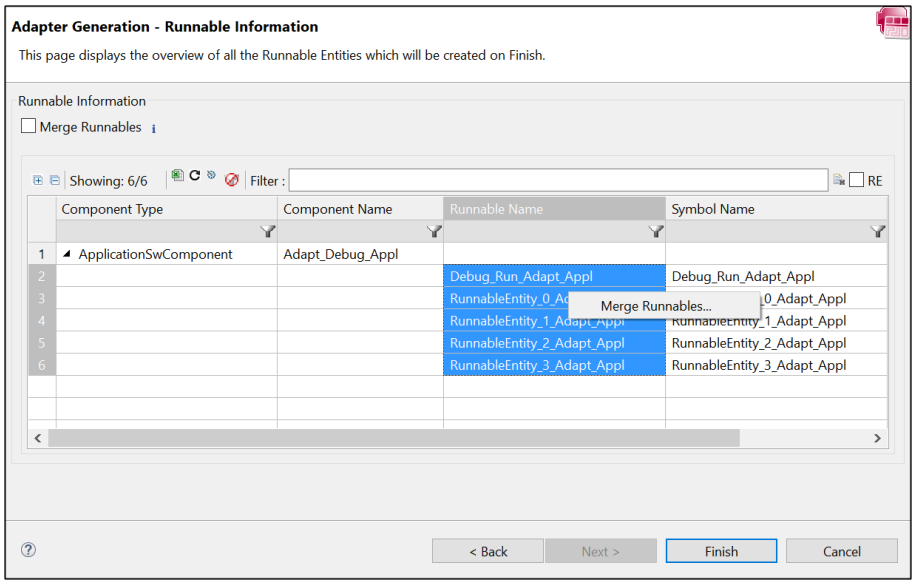

Figure 11.5.Adapter Generation - Runnable Information on Component

You can merge multiple runnables into a single runnable by right-clicking and selecting those that you wish to merge, then choosing the Merge Runnables... option. When you click Finish, the adapters will be generated.

11.4 Adapter Generation - Composition Level

When the selected item is a Software Composition, the following wizard appears.

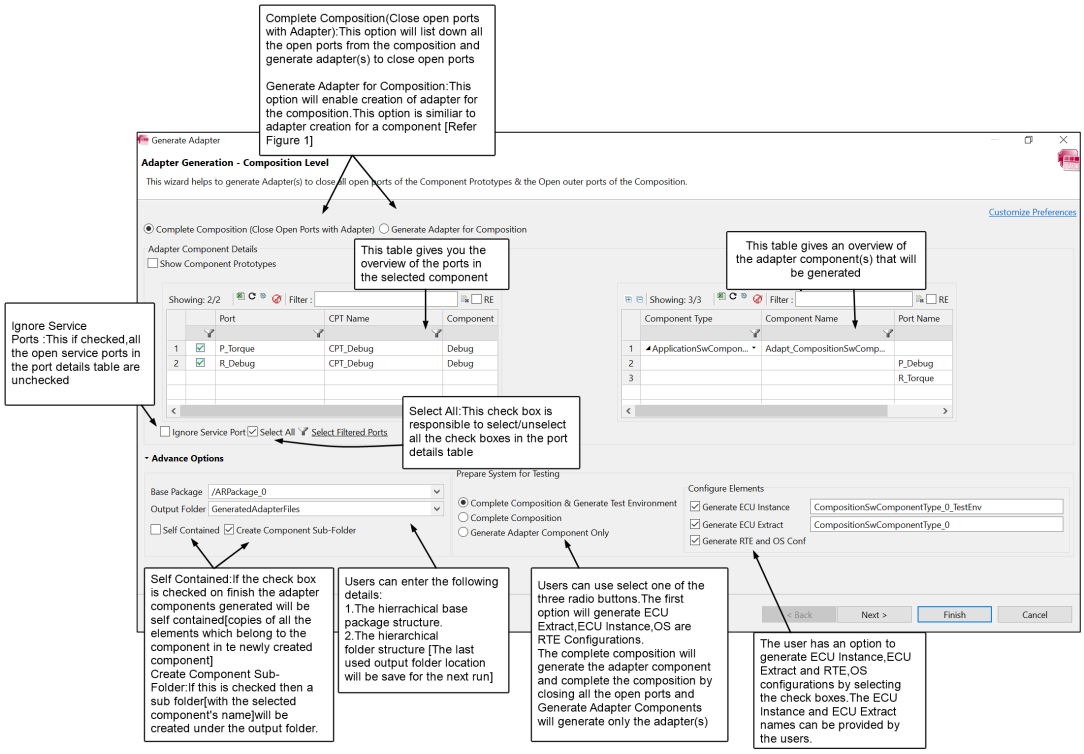

Figure 11.6.Adapter Generation - Composition Level

This first page of the wizard provides you with several options that can be selected when generating adapters for the selected composition, as described in the sub-sections below.

11.4.1 Complete Composition (Close Open Ports with Adapter)

The “Complete Composition” option allows you to create adapter component(s) to close all the open ports in the selected composition. When you click Finish, based on the options you’ve selected, all the required elements will be generated along with the adapter component(s).

11.4.2 Generate Adapter for Composition

The “Generate Adapter for Composition” option allows you to create adapter component(s) for the composition. It displays all the outer ports of the selected composition, and its functionality is similar to when you create adapter component(s) for composition.

11.4.3 Auto Generation Preferences

As shown below, various defaults can be set for the Auto Generation in Preferences, including the file names to be used for the Root Composition and Sub Composition.

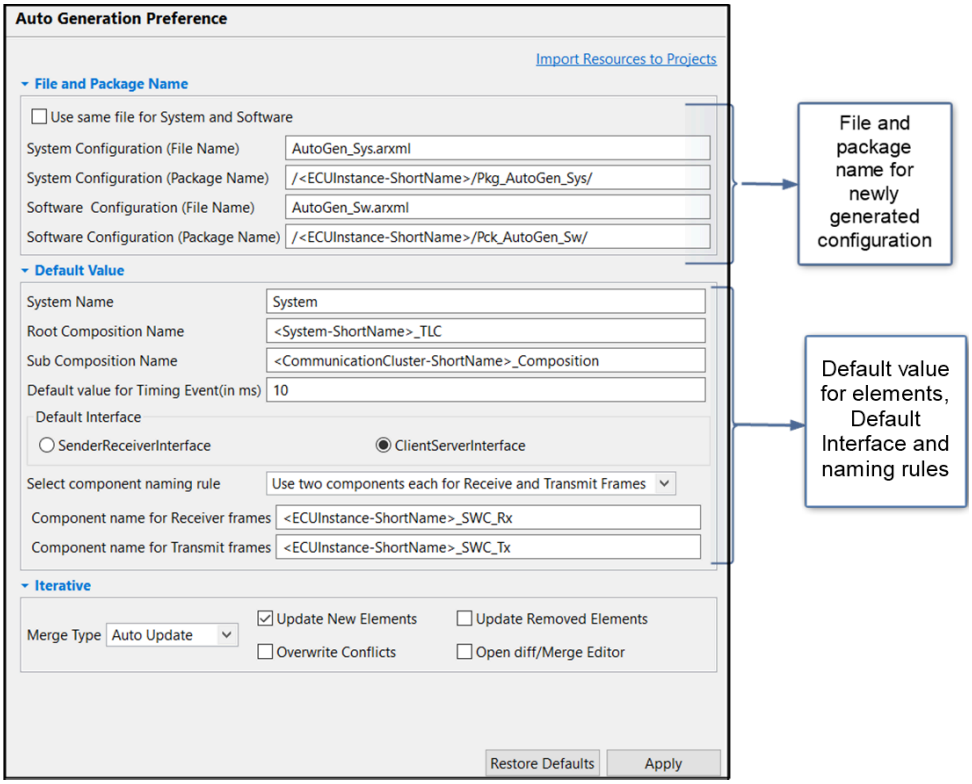

Figure 11.7.Auto Generation Preferences

As previously shown for Adapter Generation of Components, the second page of the wizard (“Runnable Information”) will give you an overview of the Runnable Entities that will be created as part of the adapter generation.

12 Query Search

12.1 Introduction

The Query Search feature allows you to search for an AUTOSAR element based on its properties and references. The search is similar to a database search, so queries are the key point for executing the search. The result of this search is generated in the form of a table.

12.2 Invocation

The Query Search feature can be invoked from two separate views:

- The **Advanced Query Search** view
- The **Query Search** view

12.2.1 Advanced Query Search view

Go to **Window** > **Show View** > **Other**, and then select **Advanced Query Search**.

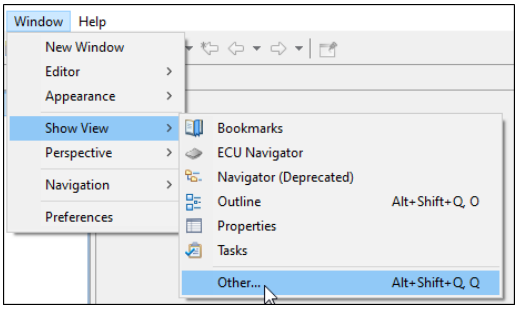

Figure 12.1.Invoking Advanced Query Search

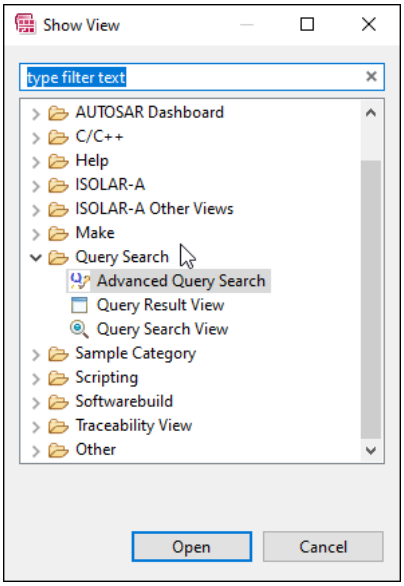

Figure 12.2.Invoking Advanced Query Search

12.2.1.1 Advanced Query Search UI

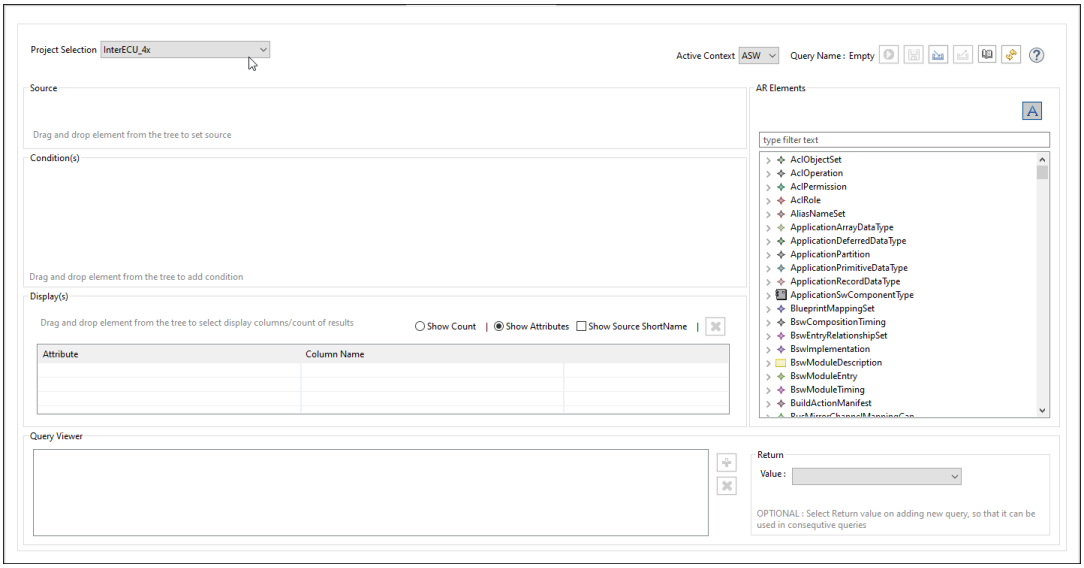

Figure 12.3.Advanced Query Search UI

12.2.1.2 Writing Queries

When you drag-and-drop items from the “AR Elements” view on the right into the respective query sections (Source, Condition, and Display) on the left, the query will be updated in the specified format and displayed in the read only “Query Viewer” section at the bottom of the UI.

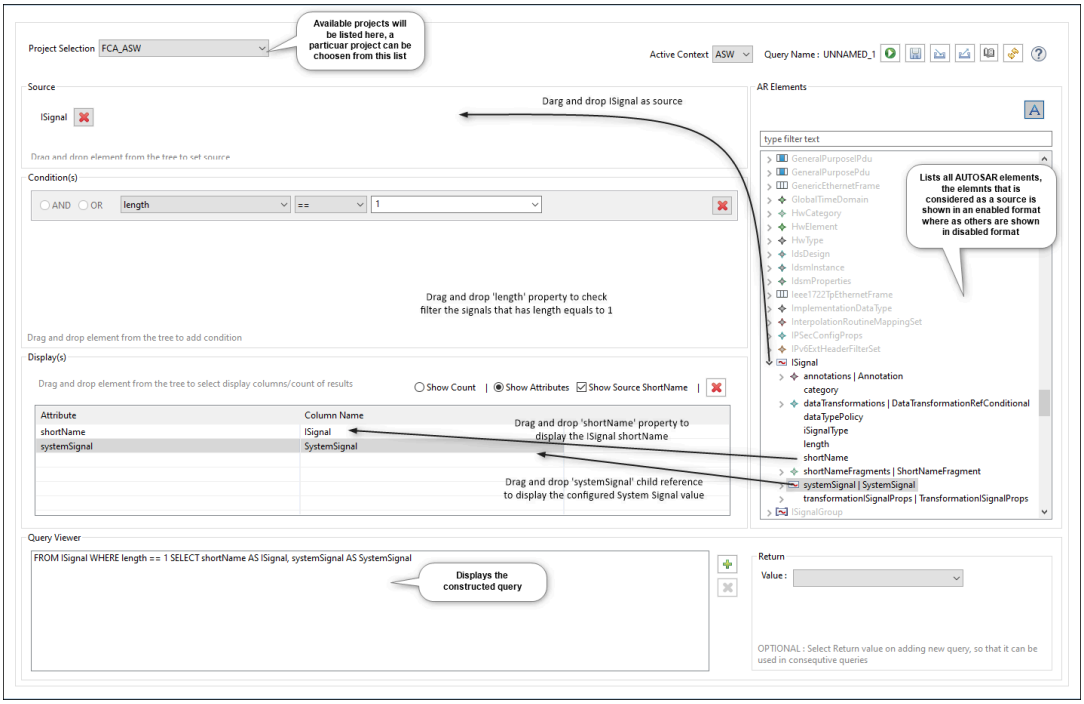

Figure 12.4.Writing Queries

12.2.1.3 Query Results

After you've constructed the query, click the green "play" icon to execute the query, and the results will be displayed in a separate Query Results view.

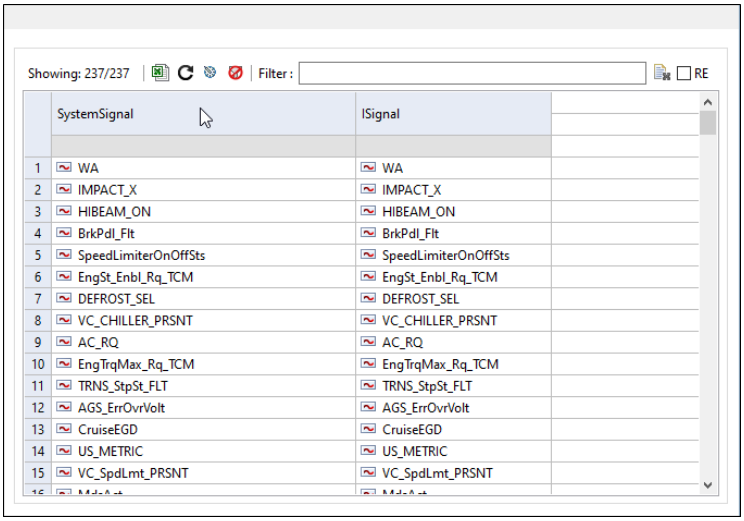

Figure 12.5.Query Results

Any of the AUTOSAR objects in the results table can be opened in the AR Explorer, which is accessible via the context menu.

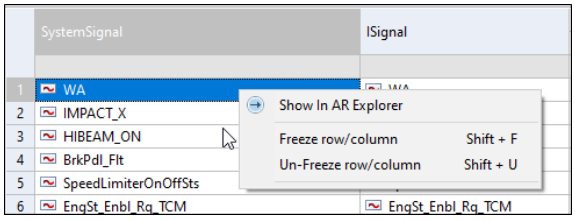

Figure 12.6.Query Results (Show in AR Explorer)

12.2.1.4 Writing Multiple Queries

If you want to search for child elements you can do this by pipelining multiple queries with the RETURN keyword.

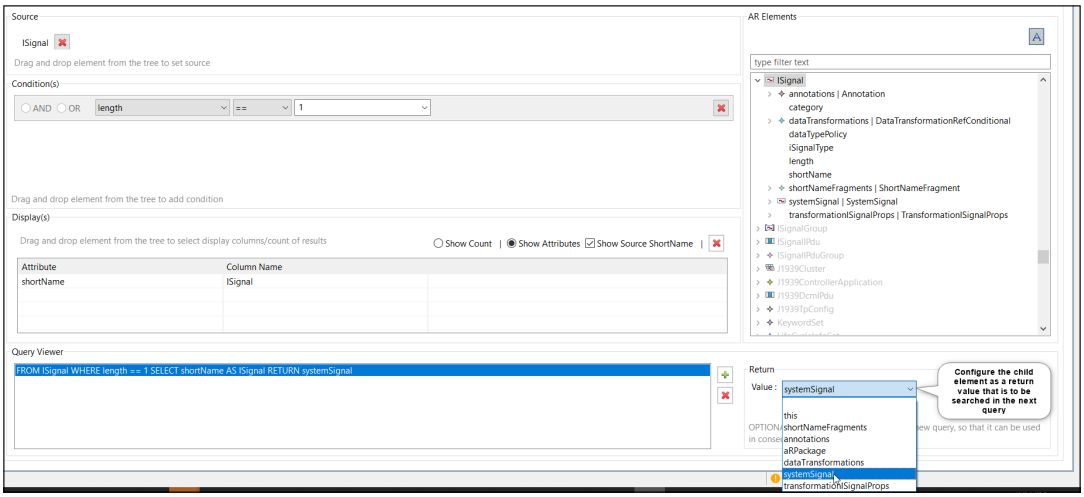

Figure 12.7.Writing multiple queries

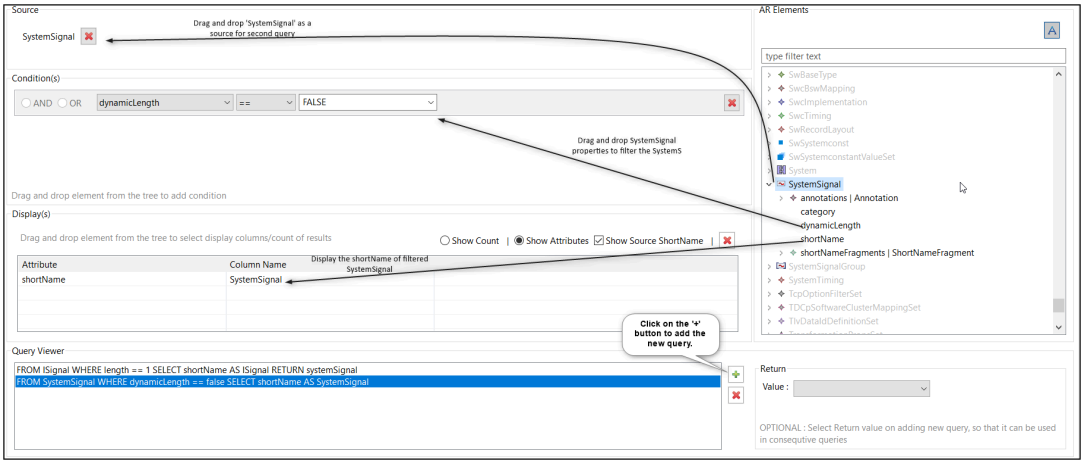

Figure 12.8.Writing multiple queries

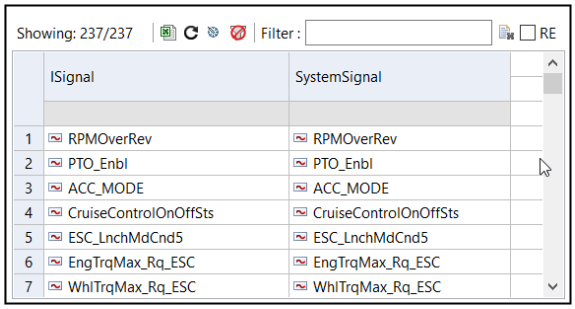

Figure 12.9.Writing multiple queries

12.2.1.5 Writing Multiple Queries (multiple sources)

In the pipelined query, if you want to use a source element that is different from the previous query, then multiple source elements can be used. To specify the secondary sources for the existing query, drag and drop the respective AUTOSAR element into the source section.

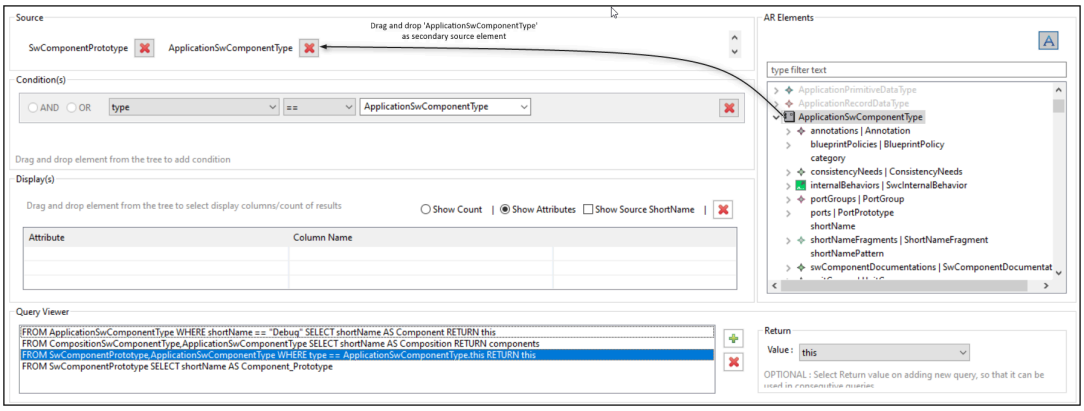

Figure 12.10.Writing multiple queries (multiple sources)

Secondary sources of the query should be a type of return value of any of the previous queries, and they can be equated with the same type of references found in the primary source.

- **Primary source:** First source element of the query
- **Secondary source:** Source element other than first source of the query

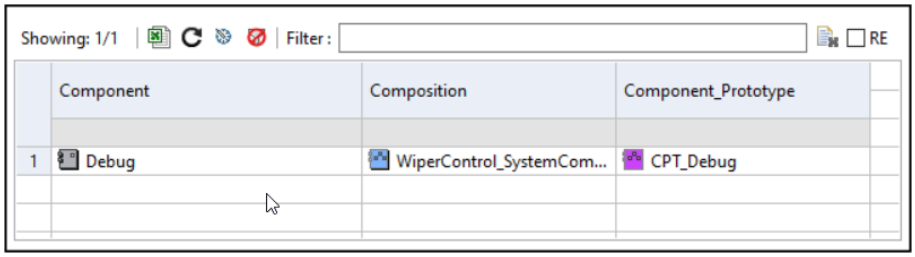

Figure 12.11.Writing multiple queries (multiple sources)

12.2.2 Query Search view

Go to **Window** > **Show View** > **Other**, and then select **Query Search View**.

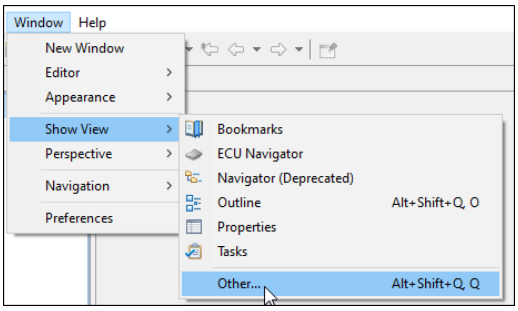

Figure 12.12.Invoking Query Search

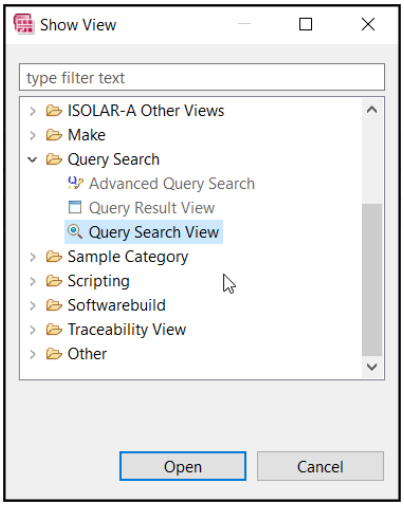

Figure 12.13.Invoking Query Search

12.2.2.1 Query Search UI

All the queries that can be constructed using the Advanced Query Search view can also be constructed using the Query Search View, but it is achieved via a

content assist feature rather than drag and drop.

Pressing Ctrl+Space after a query keyword (FROM, WHERE, SELECT, RETURN) will display a list of possible elements that could come next.

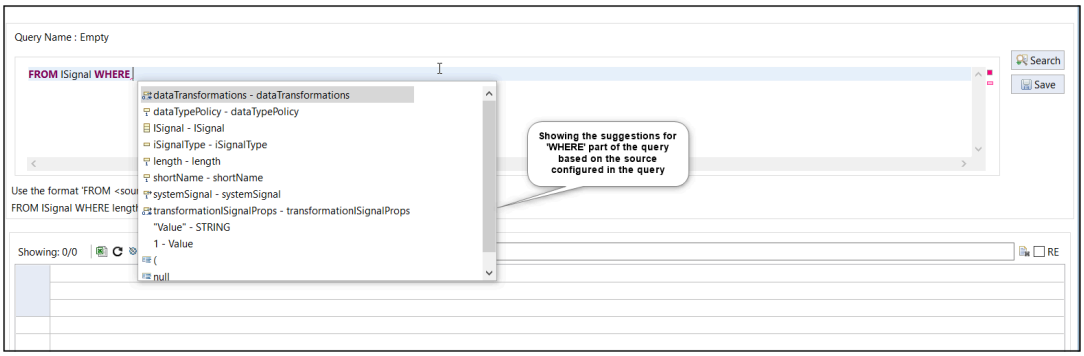

Figure 12.14.Query Search UI

12.2.2.2 Query Results

After you've constructed the query, click the "search" button to execute the query, and the results will be displayed in a table at the bottom of the UI.

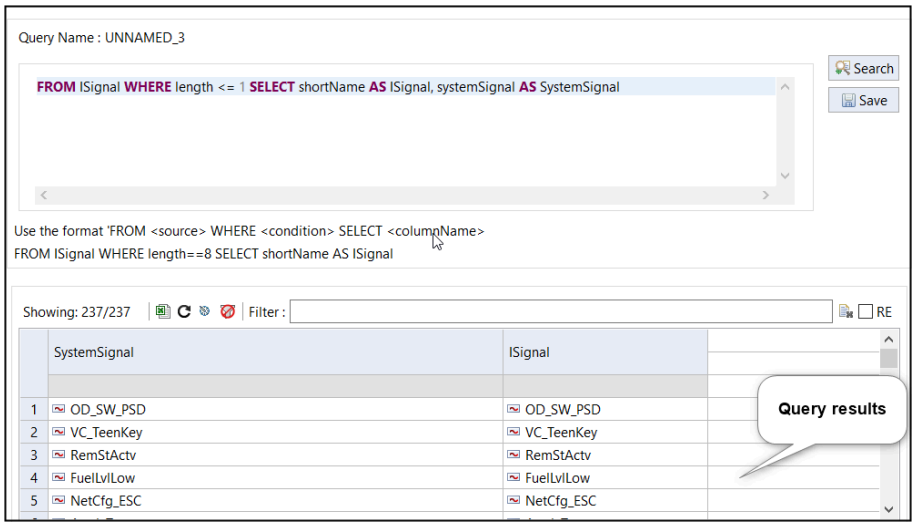

Figure 12.15.Query Results

12.3 Additional Features

The following additional features are available in both the Advanced Query Search view and the Query Search View:

- Save Query
- Import Query
- Export Query

- Restore a recent Query

12.3.1 Save Query

When executing the query, it will be automatically saved to a workspace preference with a name in the following format:

`UNNAMED_<some_random_number>`

You can also manually save a query in workspace preference by clicking on the “Save” icon in the respective query view. In the Save Query dialog, you can give the saved query a meaningful name.

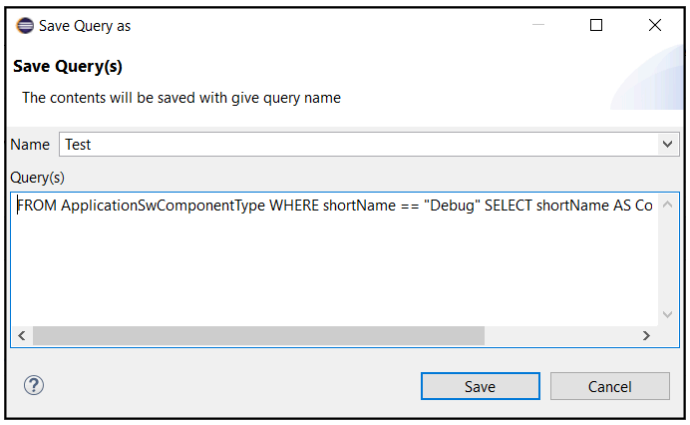

Figure 12.16.Save Query

12.3.2 Import Query

You can import a previously exported query by clicking on the “Import” icon in the respective query views. In the Import Query dialog, browse and locate the query you wish to import, and click OK. The contents of the imported query will be populated into the relevant sections.

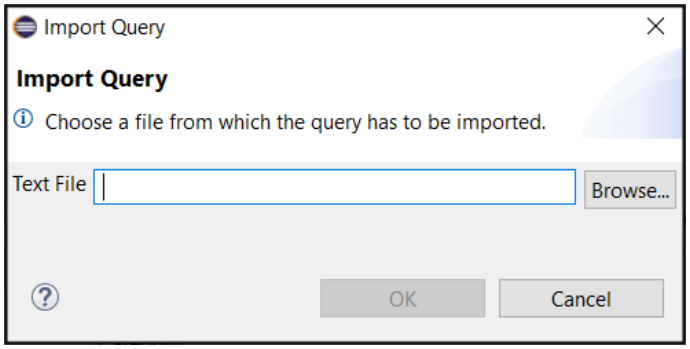

Figure 12.17.Import Query

12.3.3 Export Query

You can export a query by clicking on the “Export” icon in the respective query views. In the Export Query dialog, specify the name and location of the export file, and click OK.

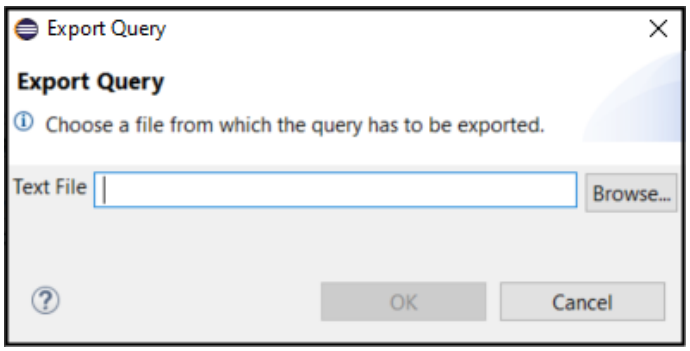

Figure 12.18.Export Query

12.3.4 Restore recent Query

All queries that you execute and/or save are available in the workspace preference, and you can “restore” any of these saved queries by clicking on the “Recent Queries” icon in the respective query views. In the “Restore from Recent Query” dialog, click one of the following two buttons:

- **Clear and Restore:** This will the restore the query with the original timestamp.
- **Restore:** This will create a copy of the query with a new timestamp.

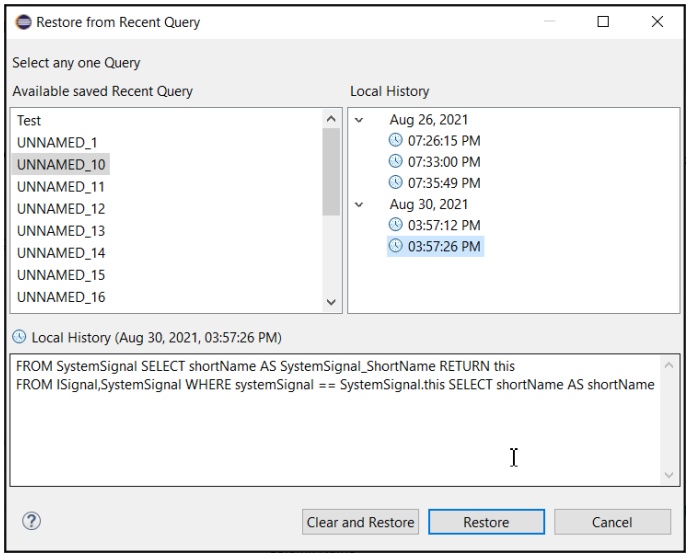

Figure 12.19.Restore a recent Query

13 Hints and Advice

13.1 Preferences

We recommend browsing through Preferences because ISOLAR-A includes lots of options for Advanced Editing. Here are just a few examples:

- Name-based Auto Connection can be configured via Preferences, which helps with the easy connection of ports based on the name rules.
- By default, ISOLAR-A shows ECU, Software and System Template related AUTOSAR elements. To view elements from other templates, you will need to enable the option in Preferences.
- For the file name to be displayed in the AR Explorer, you need to enable the "Show File Name" option under **ISOLAR-A** > **AR Explorer** > **AUTOSAR 4x**.
- Advanced delete can be optionally enabled in Preferences to delete an object reference if the referred object is deleted.
- To enable the service port on SwCompositionType, you need to enable the "Allow Service ports on SwCompositionType" option under **ISOLAR-A** > **General** > **Enable Service ports**.

13.2 Merging two AUTOSAR Elements with the Compare Editor

To compare and merge two AUTOSAR elements, ISOLAR-A provides an AUTOSAR Compare Editor, which is available by right-clicking an AUTOSAR element in the AR Explorer.

13.3 ISOLAR-A Help

For detailed help on individual feature-specific functionality, please read the built-in Help available in ISOLAR-A.

13.4 External Code Generation

It is possible to run external Code Generation in ISOLAR-A, allowing you to run any command-line code generator.

13.5 Importing an Example project

Follow these steps to import an example project into your workspace.

1. Select **File** > **New** > **Example**.

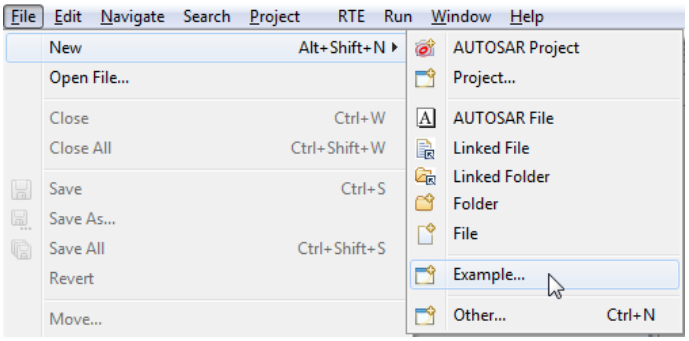

Figure 13.1.Invoking Example selection

2. In the “New Example” dialog, select **ISOLAR-A** > **ISOLAR-A AUTOSAR Examples** and click **Next**.

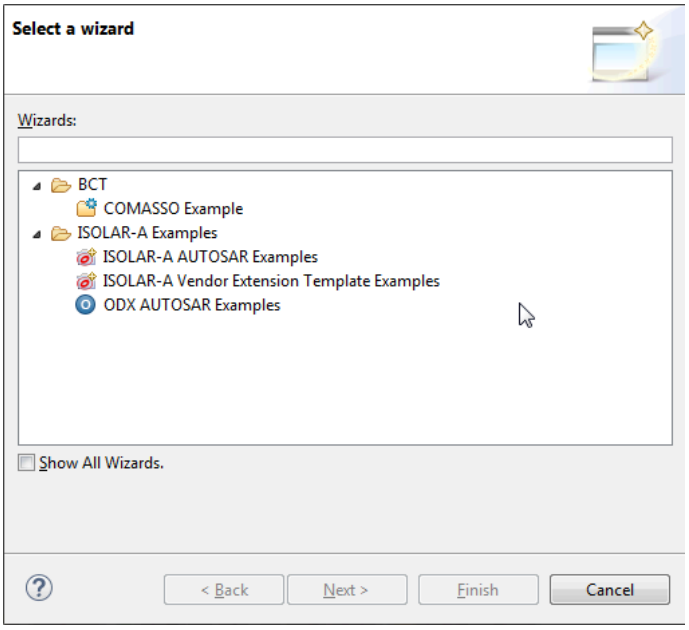

Figure 13.2.Select an Example wizard

3. Select the project that you wish to import, and then click **Finish**.

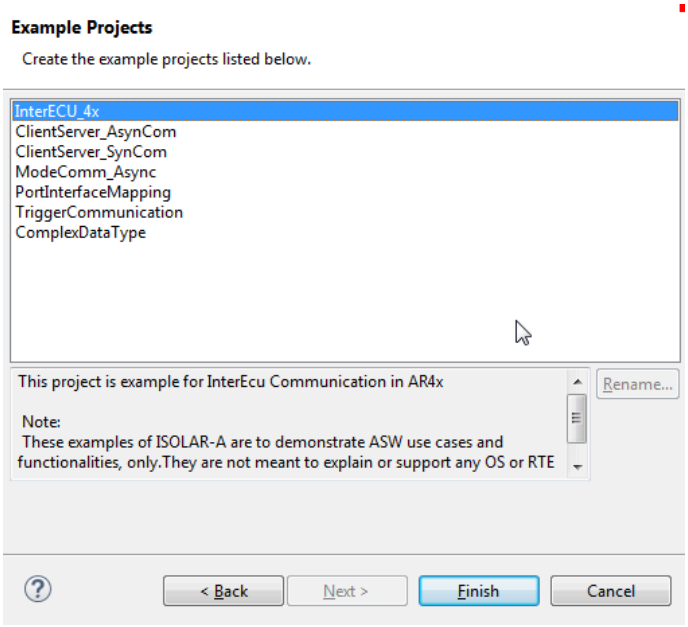

Figure 13.3.Import a project

13.6 Importing a Vendor Extension template

Follow these steps to import a vendor extension template.

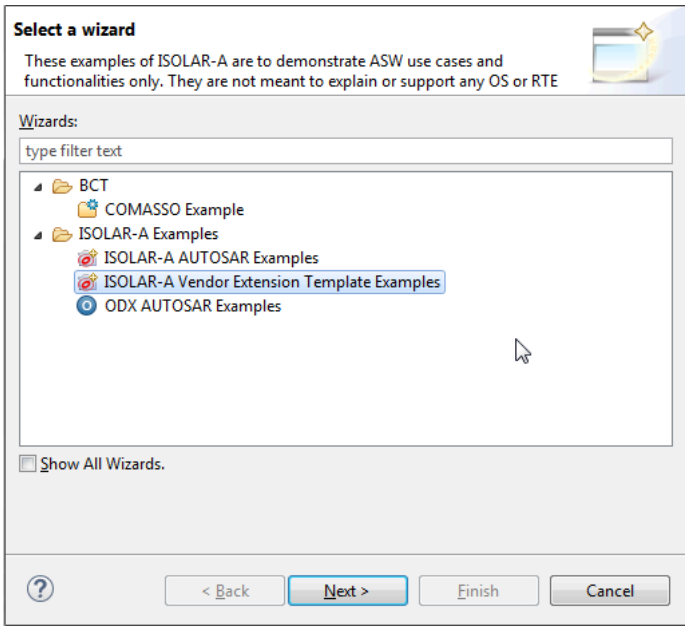

Figure 13.4.Import a Vendor Extension template

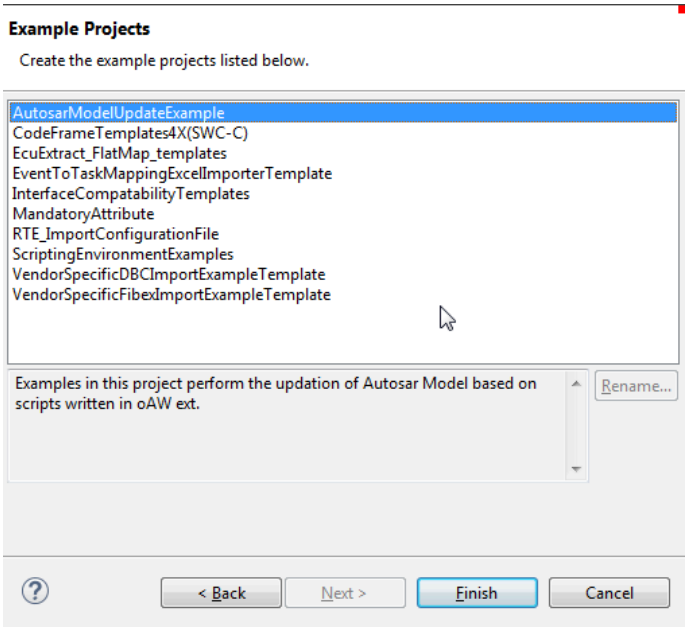

Figure 13.5.Template examples

14 ETAS Contact Addresses

14.1 Technical Support

Technical support is available to all users with a valid support contract. If you do not have a valid support contract, please contact your regional sales office.

- https://www.etas.com/en/contact.php

The best way to get technical support is using the online support portal RTA Hotline:

- https://rtahotline.etas.com/

It is helpful if you can provide technical support with the following information (where relevant):

- Your support contract number
- The version of the ETAS tools you are using
- The version of the compiler tool chain you are using
- The command line (or reproduction of steps) that result in an error message
- The error messages or return codes you received (if any)
- Your RTA-CAR Project or if not possible the arxml file(s) containing the error(s).
- The *Diagnostic.dmp* if it was generated

14.2 ETAS HQ

ETAS GmbH Borsigstraße 24 70469 Stuttgart Germany	Phone:	+49 711 3423-0
	Fax:	+49 711 3423-2106
	WWW:	www.etas.com